

从持续学习到递归自我改进

AI 如何在使用中持续成长?

周 杰 青年研究员
计算机科学与技术学院/上海人工智能实验室

華東師範大學
EAST CHINA NORMAL UNIVERSITY



華東師範大學
EAST CHINA NORMAL UNIVERSITY

AI 如何持续成长?

数据驱动终身学习

新数据到来，怎样学新而不忘旧?

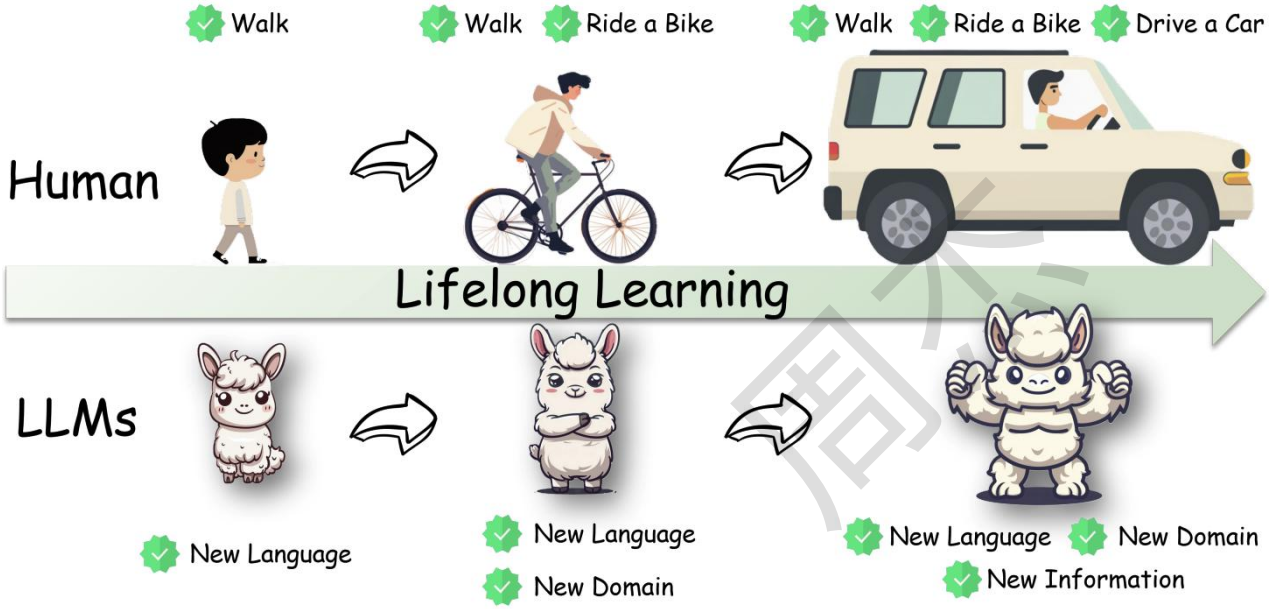
经验驱动终身学习

做过一次，怎样帮助下一次行动?

循环自我进化

积累能力后，学习方法能否一起改进?

终身学习：新任务学得会，旧能力记得住



数据驱动的典型设置

任务和数据依次到来，
模型随新数据持续更新。

旧数据难以完整保存，
只能有限回放或不回放。

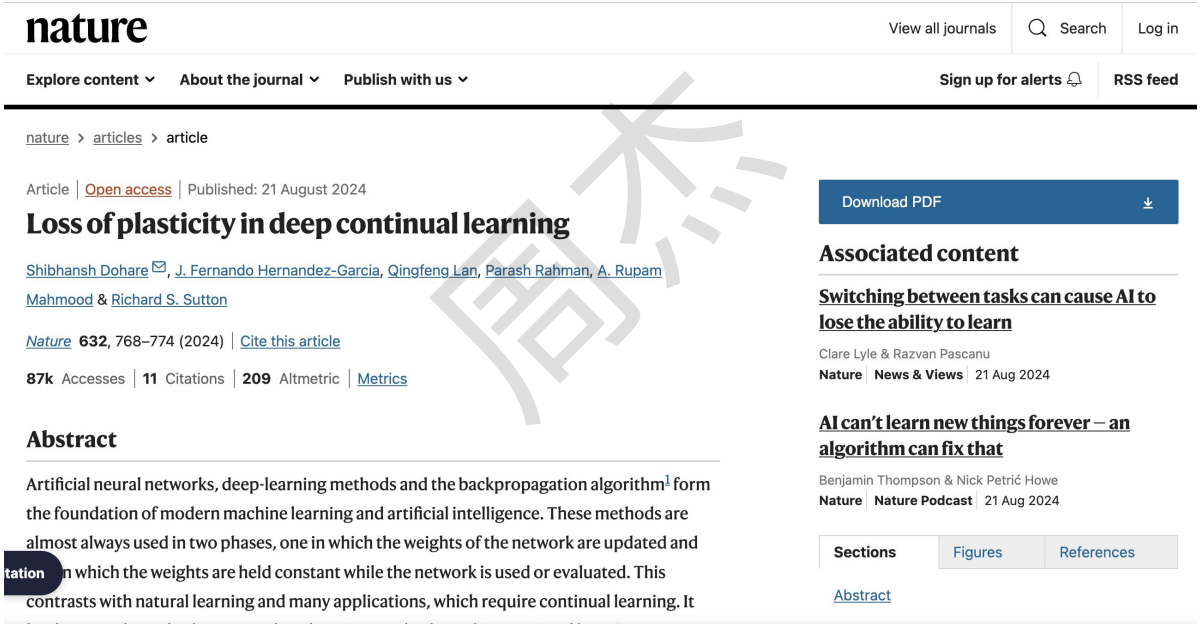
目标：知识持续积累
约束：历史数据受限

抗遗忘：学新任务时，尽量保留旧任务表现。
知识迁移：让已学知识帮助新任务，也让新学习反哺旧任务。

Junhao Zheng, Shengjie Qiu, et al. Towards Lifelong Learning of Large Language Models: A Survey, 2025.

持续学习既怕“忘了”，也怕“学不动”

长期连续训练后，模型可能越来越难学会新任务，这称为可塑性下降。
Nature 的研究在监督学习与强化学习任务中观察到了这一现象。



nature

View all journals | Search | Log in

Explore content | About the journal | Publish with us | Sign up for alerts | RSS feed

nature › articles › article

Article | Open access | Published: 21 August 2024

Loss of plasticity in deep continual learning

Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood & Richard S. Sutton

Nature **632**, 768–774 (2024) | Cite this article

87k Accesses | **11** Citations | **209** Altmetric | Metrics

Abstract

Artificial neural networks, deep-learning methods and the backpropagation algorithm[1] form the foundation of modern machine learning and artificial intelligence. These methods are almost always used in two phases, one in which the weights of the network are updated and [illegible] in which the weights are held constant while the network is used or evaluated. This contrasts with natural learning and many applications, which require continual learning. It

Download PDF

Associated content

Switching between tasks can cause AI to lose the ability to learn

Clare Lyle & Razvan Pascanu
Nature | **News & Views** | 21 Aug 2024

AI can't learn new things forever – an algorithm can fix that

Benjamin Thompson & Nick Petrić Howe
Nature | **Nature Podcast** | 21 Aug 2024

Sections | Figures | References

Abstract

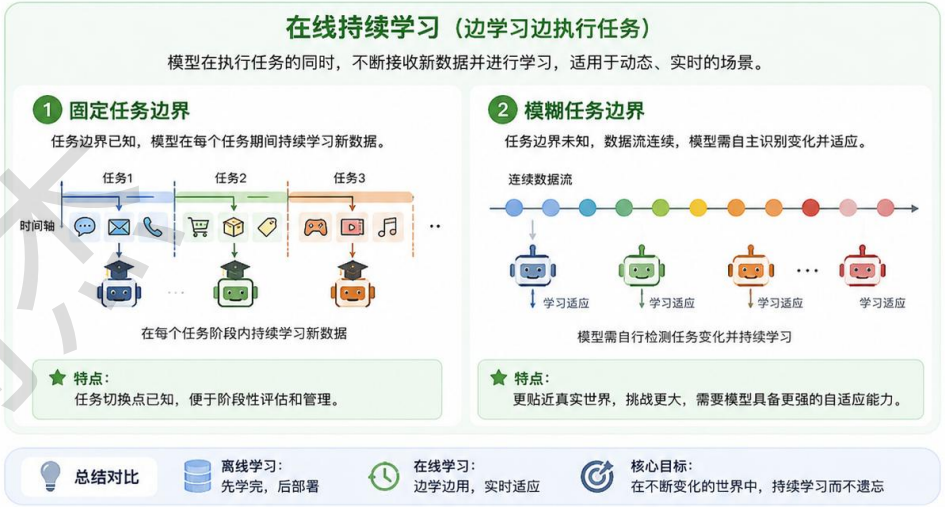
保住旧知识之外，还要维持学习新知识的能力

Shibhansh Dohare, J. Fernando Hernandez-Garcia, et al. Loss of plasticity in deep continual learning, 2024.

数据逐步走向在线，学习条件也更加复杂

成批任务考察知识保留，在线数据流还要求及时更新与可靠反馈。





EASYCL
Continual Learning for Large Language Models

ICALK

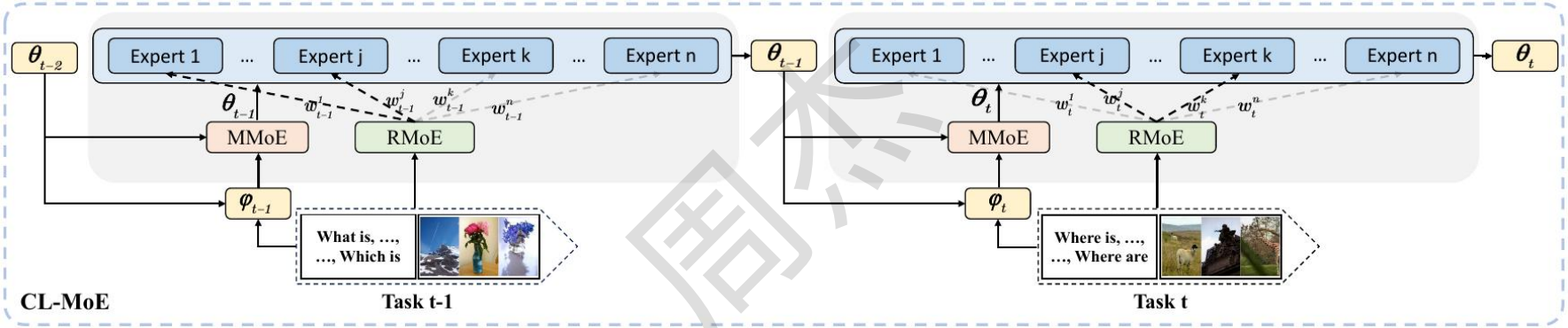
EasyCL 支持统一的持续学习实验
多模态、多基座模型与十余种方法。

https://github.com/ECNU-ICALK/EasyCL

Yutao Yang, Jie Zhou, et al. Recent Advances of Foundation Language Models-based Continual Learning: A Survey, 2025.

CL-MoE：用专家分工减少新旧任务干扰

面对持续到来的任务，关键是让新知识有地方学，同时少改动旧任务依赖的能力。
CL-MoE 从视觉问答切入，用多位 LoRA 专家分担任务，并控制共享专家的更新。



一个问题需要多种能力

同一任务中的样本有细节差异，
不同任务又可能共享识别、定位等能力。

共享能力最需要谨慎更新

统一更新会损伤旧任务依赖的专家，
完全冻结又限制新知识的吸收。

持续学习既要决定哪些专家参与，也要决定新知识应该改动哪些专家

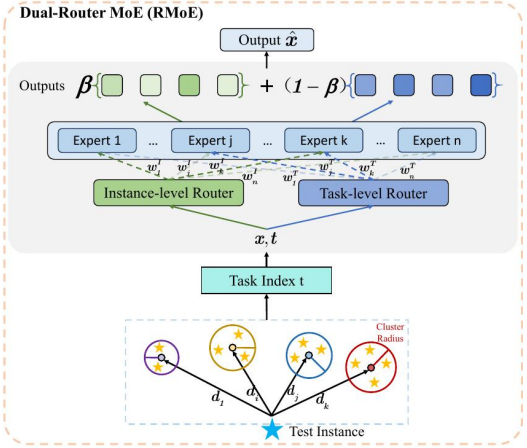
Tianyu Huai, Jie Zhou, et al. CL-MoE: Enhancing Multimodal Large Language Model with Dual Momentum Mixture-of-Experts for Continual Visual Question Answering, CVPR, 2025.

CL-MoE：先选对专家，再控制更新幅度

任务层面的共性可以稳定专家分工，样本层面的差异又需要灵活选择。
选好专家后，再根据它是否服务过旧任务，决定更多保留旧参数还是吸收新参数。

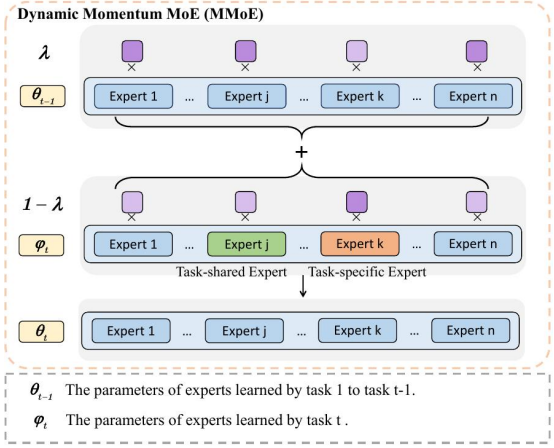
双路由：结合全局共性与局部需求

任务级权重结合实例级权重，
测试时通过最近任务锚点推断任务。



动态动量：按专家角色融合新旧参数

新旧任务共享的专家偏向旧参数，
仅对当前任务重要的专家偏向新参数。



共享专家多保留历史，新任务专属专家多吸收变化，学习与记忆各有侧重

Tianyu Huai, Jie Zhou, et al. CL-MoE: Enhancing Multimodal Large Language Model with Dual Momentum Mixture-of-Experts for Continual Visual Question Answering, CVPR, 2025.

周杰

CL-MoE：选对专家，还要保护已有能力

在 VQA v2 的十类问题上依次学习，比较同一 LLaVA-7B 基座。
AP 衡量学完全部任务后的平均表现，AF 衡量旧任务的平均遗忘，越低越好。

Methods	$AP(\uparrow)$	$AF(\downarrow)$
LLaVA-7B based methods		
Vanilla	32.51	20.69
EWC	37.28	15.27
ER	41.95	10.20
VQACL	43.49	9.10
CL-MoE	**51.34**	**-0.02**

	Method		AP	AF
	MMoE	RMoE		
a	×	×	32.51	20.69
b	√	×	48.43	3.02
c	×	√	41.23	11.09
d	√	√	**51.34**	**-0.02**

相对 VQACL，AP 提高 7.85 个百分点

AF 从 9.10 降至 −0.02，平均遗忘接近零。

单有路由，遗忘仍然明显

仅 RMoE 的 AF 为 11.09，加入 MMoE 后大幅下降。

在这组视觉问答任务中，专家路由与动量保护共同减少了遗忘

Tianyu Huai, Jie Zhou, et al. CL-MoE: Enhancing Multimodal Large Language Model with Dual Momentum Mixture-of-Experts for Continual Visual Question Answering, CVPR, 2025.

TASER：能力不足时，按需扩展参数技能

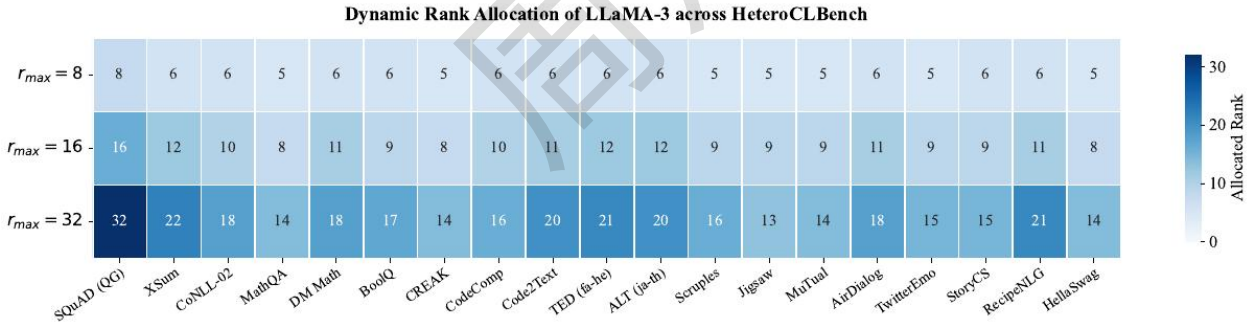
当任务扩展到数学、代码与对话，除了调用已有能力，还要判断哪些能力尚未具备。
TASER 按任务缺口扩展可组合的参数技能，在 19 项异质任务中兼顾学习与复用。

固定容量，难兼顾专业化与复用

只保护旧参数，可能没有空间学习新能力。
每个任务都大量扩展，又会增加冗余与干扰。

原子技能是可组合的参数单元

把 LoRA 的每个秩维度视为一个原子技能，
新增模块的秩，就决定新增技能的数量。



原文图 4：相同容量预算下，不同任务实际获得的新增技能数并不相同。

新任务需要多少新增技能，应由已有能力覆盖了什么、还缺什么来决定

周杰

Jiacheng Wang, Xinjia He, et al. Task-Differentiated Atomic Skill Expansion and Routing for Continual Learning Across Highly Heterogeneous Tasks, ECML-PKDD, 2026.

TASER：按能力缺口扩展，再组合新旧技能

新技能应补足已有技能库的缺口，同时保持可独立复用，避免学成重复的能力。
训练时冻结底座和历史技能，仅更新新增技能与路由器，并只使用当前任务数据。

新增多少：先试学，再估计

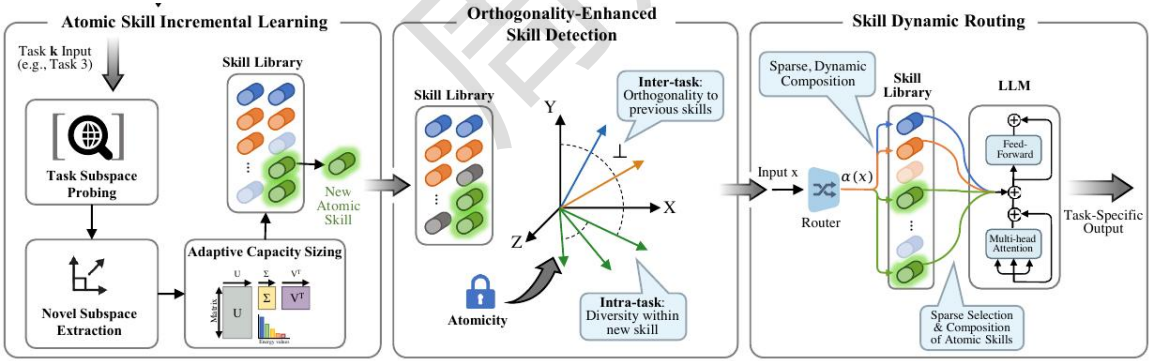
短暂训练探测适配器，
去掉旧技能已覆盖的方向，
保留主要新方向来确定秩。

如何学好：减少技能重叠

新技能与旧技能保持正交，
新增技能内部也保持多样，
减少跨任务干扰与内部冗余。

如何使用：按输入选组合

轻量路由器给技能分配权重，
允许多个技能同时参与，
稀疏约束抑制无关技能。



按缺口扩展容量，用正交约束减少干扰，再让路由器组合相关的新旧技能

Jiacheng Wang, Xinjia He, et al. Task-Differentiated Atomic Skill Expansion and Routing for Continual Learning Across Highly Heterogeneous Tasks, ECML-PKDD, 2026.

TASER：按需扩展与路由支持异质任务学习

在 Llama-3-8B 的 19 项异质任务上，TASER 的 AP 达到 63.31，联合训练为 65.70。
BWT 为 −0.94，表示旧任务平均下降不足 1 分，优于表中其他持续学习方法。

Task	Continual Learning Baselines							Ours	Upper Bound
	Seq FT	EWC	Replay	LAMOL	I-LoRA	MoE-LoRA	CL-MoE	TASER	MTL
AP	50.76	53.56	62.66	53.67	61.82	55.17	54.69	**63.31**	65.70
BWT	-13.54	-13.13	-2.03	-14.45	-1.56	-11.47	-15.13	**-0.94**	-

Model Variant	AP (↑)	Δ_{AP}	BWT (↑)	Δ_{BWT}
Full TASER framework	**63.31**	-	**-0.94**	-
w/o Atomic Skill Incremental Learning	57.14	-6.17	-2.69	-1.75
w/o Orthogonality-enhanced Skill Detection	62.74	-0.57	-1.4	-0.46
w/o Skill Dynamic Routing	44.82	-18.49	-5.39	-4.45

动态路由贡献最明显

移除路由，AP 下降 18.49 分。
取消自适应扩展，下降 6.17 分。

两者共同支持高异质任务学习。

在这些异质任务中，扩展与路由兼顾了新任务学习和旧能力保留

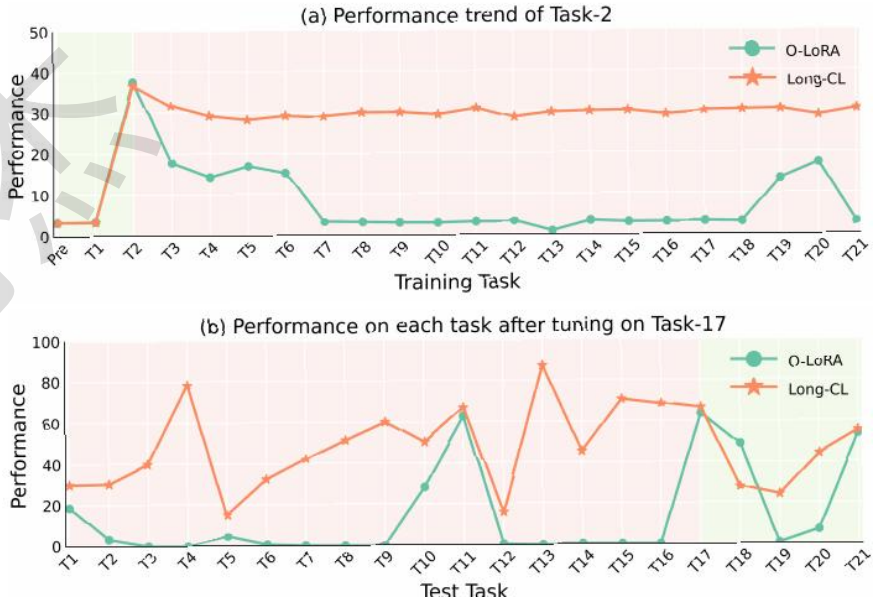
Jiacheng Wang, Xinjia He, et al. Task-Differentiated Atomic Skill Expansion and Routing for Continual Learning Across Highly Heterogeneous Tasks, ECML-PKDD, 2026.

Long-CL：任务流拉长，记忆如何留得更久

能力可以不断扩展，但任务流拉长后，仍需让关键知识经得起后续更新。
Long-CL 把问题转向长期巩固：哪些参数要保护，哪些历史样本值得反复学习。

更长、更多样的任务流

多模态：21 个数据集，4 类任务
文本：30 个数据集，3 类任务



任务 2 学过之后，仍可能迅速遗忘

原文示例中，O-LoRA 学到任务 7 后，
任务 2 的表现已经接近零。

长期持续学习需要同时回答：哪些参数值得保护，哪些样本值得反复学习

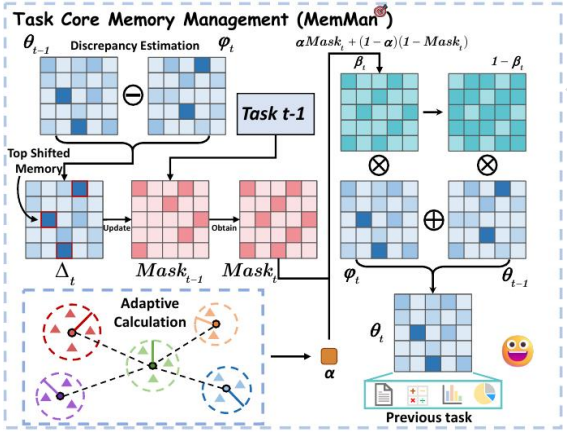
Tianyu Huai, Jie Zhou, et al. Task-Core Memory Management and Consolidation for Long-term Continual Learning, 2025.

Long-CL：保护关键参数，精选样本巩固

一视同仁地更新参数，会覆盖旧知识；随机保存样本，又可能错过最值得复习的内容。
Long-CL 在冻结底座、训练 LoRA 的基础上，分别管理参数中的记忆和用于复习的样本。

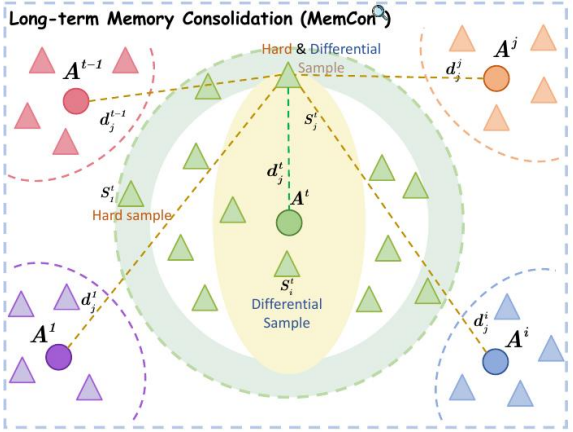
MemMan：找到关键位置，再控制更新

记录各任务训练时变化最大的参数位置，
按任务新颖度和历史重要性融合新旧参数。



MemCon：挑选难样本与跨任务样本

保留偏离当前任务中心的难样本，
以及兼顾多种历史任务的样本用于回放。



参数融合控制知识如何变化，精选回放强化值得长期保留的知识

周杰

Tianyu Huai, Jie Zhou, et al. Task-Core Memory Management and Consolidation for Long-term Continual Learning, 2025.

Long-CL：长任务流中，旧能力仍能受益

Long-CL 在多模态与文本基准上都取得最高的持续学习平均表现。
AP 越高越好，AF 越低越好；负 AF 表示旧任务的平均表现超过刚学完这些任务时。

	多模态		文本	
Methods	$AP(\uparrow)$	$AF(\downarrow)$	$AP(\uparrow)$	$AF(\downarrow)$
Vanilla	15.35	28.42	44.20	15.55
ER [34]	43.11	-0.74	53.58	5.89
CL-MoE [6]	44.53	-2.21	45.04	14.64
Long-CL	**51.93**	**-9.93**	**60.12**	**-0.89**
Multitask	54.96	-	61.48	-

AP 分别超过最优基线 7.40、6.54 个百分点
AF 为 −9.93、−0.89，均有平均后向提升。

两种机制配合效果最好

	Method		AP	AF
	MemMan	MemCon		
a	✗	✗	15.35	28.42
b	✓	✗	39.91	2.31
c	✗	✓	46.63	-4.90
d	✓	✓	**51.93**	**-9.93**

多模态 AP：仅复习 46.63
加入参数管理后升至 51.93。

参数保护与精选回放相互配合，长任务流中仍有平均后向提升

Tianyu Huai, Jie Zhou, et al. Task-Core Memory Management and Consolidation for Long-term Continual Learning, 2025.

LifeAlign：持续学会新偏好，保留已有对齐

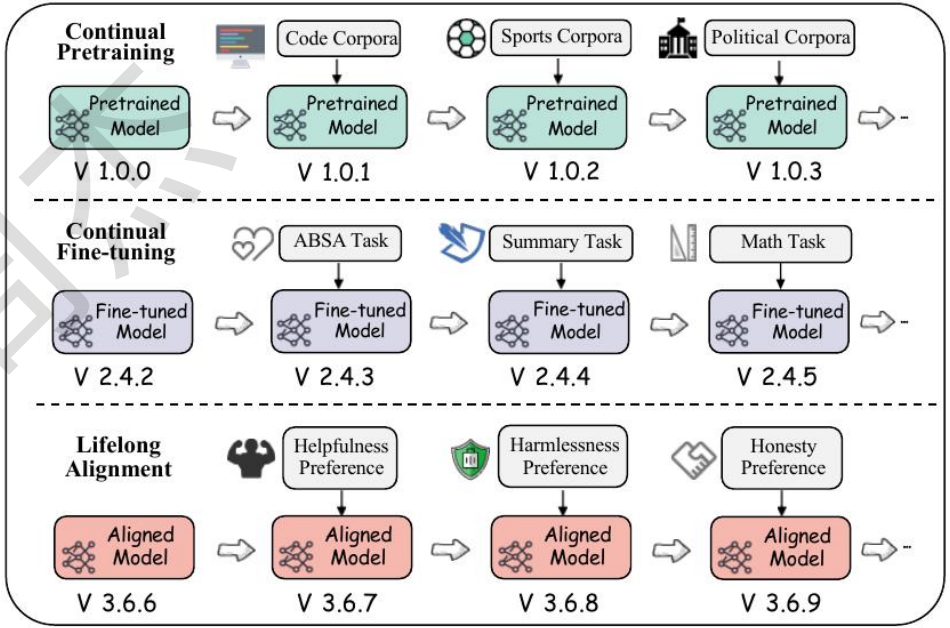
长期保留的不只有任务知识，还有用户对“哪个回答更好”的判断。
LifeAlign 将持续学习扩展到偏好对齐，吸收新要求时，也保留此前形成的行为准则。

学习信号是“哪个回答更好”

偏好数据比较两个回答，
新任务的优化可能改变旧任务的排序。
保存知识的方案需要适配这种学习信号。

优化力度和参数合并都要控制

已学会的偏好不必反复大幅调整，
新学到的参数变化也需要去噪和整合，
减少对历史对齐的干扰。



终身对齐需要持续吸收新偏好，同时保留已经建立的行为准则

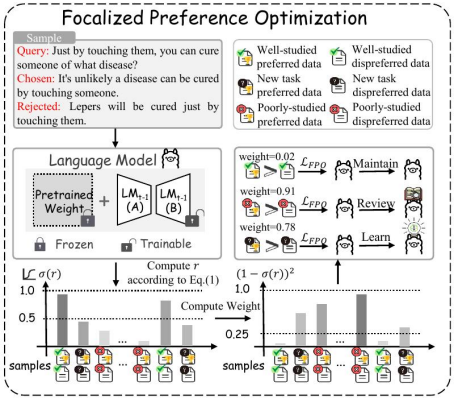
Junsong Li, Jie Zhou, et al. LifeAlign: Lifelong Alignment for Large Language Models with Memory-Augmented Focalized Preference Optimization, AAAI, 2026.

LifeAlign：按需强化偏好，稳定合并更新

新旧样本的学习需求不同，而一次训练得到的参数变化还可能包含噪声与冲突。
LifeAlign 先调节每对偏好样本的学习力度，再把精炼后的 LoRA 更新并入长期参数。

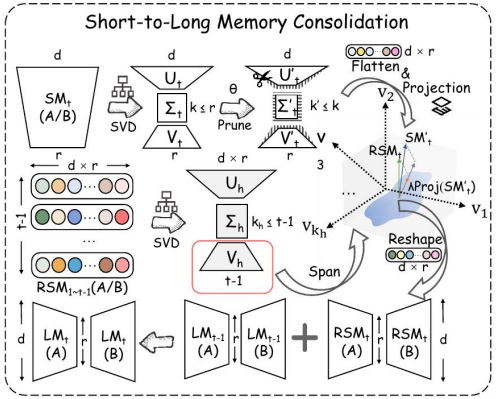
FPO：不确定的偏好多学一些

在 DPO 损失上加入置信度权重。
已掌握的样本降低权重，新样本与遗忘样本
获得更强的学习信号，并混合回放历史数据。



SLMC：更新先精炼，再融入历史

用 SVD 保留主要变化、滤除噪声。
减弱与历史更新子空间重叠的分量，
再把精炼后的更新加入长期 LoRA 参数。



FPO 控制哪些偏好需要重点学，SLMC 控制哪些参数变化值得长期保留

周木

Junsong Li, Jie Zhou, et al. LifeAlign: Lifelong Alignment for Large Language Models with Memory-Augmented Focalized Preference Optimization, AAAI, 2026.

LifeAlign：学好新偏好，也能保住历史对齐

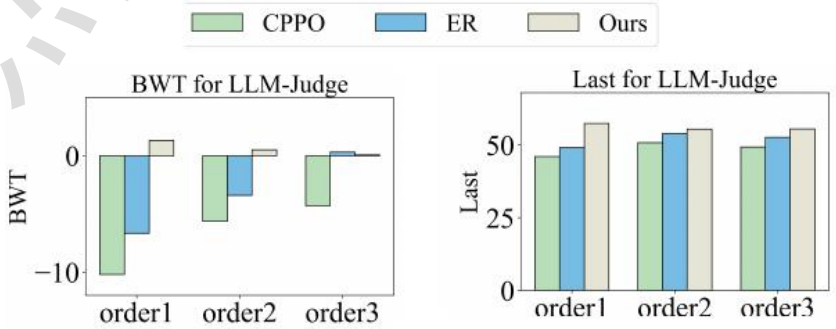
在 Qwen-2.5-7B 的六项顺序对齐任务中，LifeAlign 兼顾最终质量与历史偏好保留。
下表采用 LLM-Judge：Last 是学完后的平均质量，BWT 为正表示旧任务平均受益。

Methods	LLM-Judge		
	BWT	Last	AP
SeqFT	-9.21	39.72	39.42
L2P	**3.26**	34.04	34.03
ER	-6.66	49.11	52.03
CPPO	-10.2	46.05	45.64
LifeAlign (Ours)	1.31	**57.42**	53.91
MTL (Upper Bound)	–	57.01	–

L2P 的 BWT 更高，但 Last 仅为 34.04。
LifeAlign 的 Last 为 57.42，接近联合训练。

逆序、随机序下仍保持优势

order1 正序，order2 逆序，order3 随机序



逆序时 BWT 为 +0.51，ER 为 −3.41。
改变学习顺序后，仍能有效保留历史偏好。

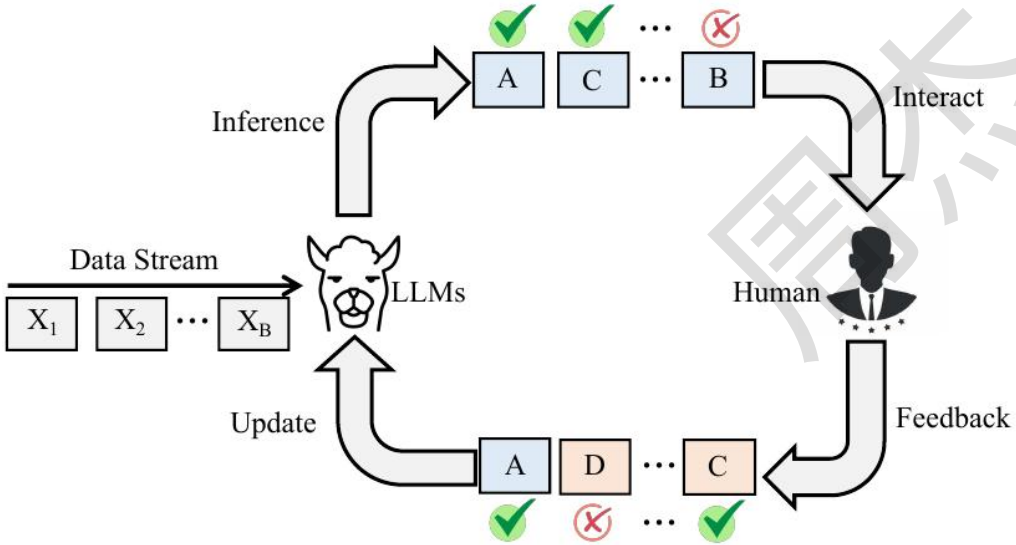
在不同任务顺序下，LifeAlign 兼顾最终质量与历史偏好保留

周杰

Junsong Li, Jie Zhou, et al. LifeAlign: Lifelong Alignment for Large Language Models with Memory-Augmented Focalized Preference Optimization, AAAI, 2026.

RiCL：反馈随交互到来，错误纠正也要筛选

学会偏好还需要可信的反馈，但真实用户的纠正随交互到来，也可能出错。
RiCL 研究模型边预测、边接收反馈、边更新时，如何同时应对标签噪声与知识遗忘。



数据在变化，标签也不完全可靠

模型需要随数据流及时更新，
既保留历史能力，
又减少错误纠正带来的影响。

不可信的标签，不等于无用的输入

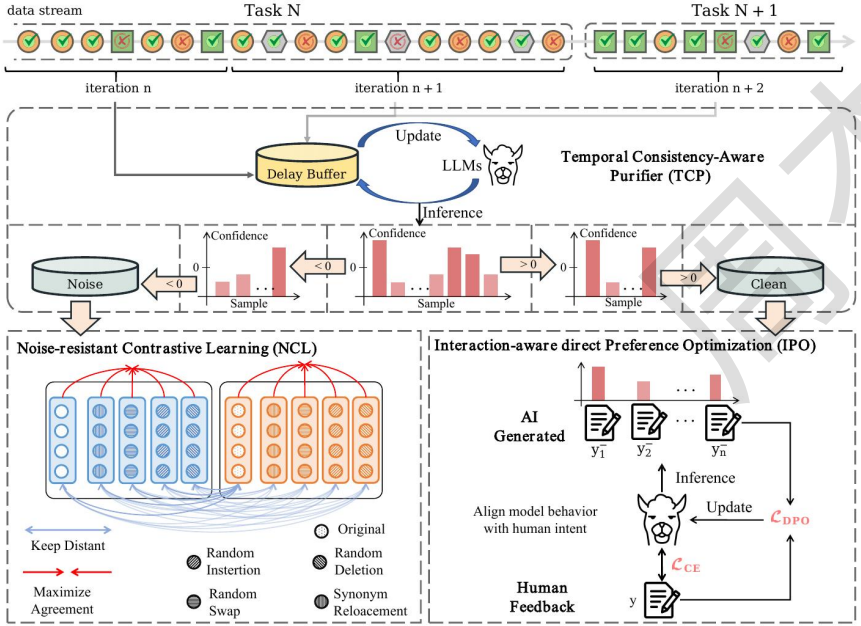
可信反馈可以指导答案选择，
含噪样本仍能帮助学习稳定的表示，
两类数据需要采用不同学习方式。

交互式持续学习需要同时处理反馈噪声与知识遗忘，判断可信度是学习的一部分

Yutao Yang, Jie Zhou, et al. Reinforced Interactive Continual Learning via Real-time Noisy Human Feedback, ESWA, 2026.

RiCL：可信标签指导答案，噪声样本学习表示

RiCL 先用置信度区分反馈，再通过时间上的重复判断筛选回放数据。
随后分别采用偏好优化与对比学习，让干净样本和含噪样本都能发挥作用。



TCP：用前后判断的一致性筛选

先按标签置信度分流，缓存满后复查。
只有前后判断一致的样本，
才进入对应的干净或含噪回放库。

IPO：把可信纠正变成偏好对

把用户标签与模型可能预测的错误标签配对，
用 DPO 式目标提高正确标签的相对概率。

NCL：绕开不可靠的标签

对原句及增强版本做对比学习，
从含噪样本中学习更稳定的特征。

同一条数据流按可靠性分开学习，再通过回放兼顾当前改进与历史保留

Yutao Yang, Jie Zhou, et al. Reinforced Interactive Continual Learning via Real-time Noisy Human Feedback, ESWA, 2026.

RiCL：筛选反馈抗噪，偏好优化减少遗忘

在 LLaMA-7B 上，用 FewRel、TACRED 的标签扰动模拟人类反馈，持续学习 10 个任务。
AP 衡量最终平均准确率，AF 衡量平均遗忘；原文分别考察噪声强度与模块贡献。

FewRel：噪声从 20% 增至 40%

Methods	Fewrel			
	20% Noise		40% Noise	
	AP ↑	AF ↓	AP ↑	AF ↓
SeqFT	68.09	28.19	57.11	28.41
ER + JoCoR	81.74	11.54	63.08	18.28
MIR + JoCoR	78.89	14.79	64.45	18.89
RiCL	84.77	7.90	83.78	8.86

RiCL 的 AP 仅下降 0.99 个百分点。
ER+JoCoR 则从 81.74 降至 63.08。

去掉 IPO，遗忘明显增加

TCP	NCL	IPO	Tacred		Fewrel	
			AP	AF	AP	AF
✓	✓	✓	**82.70**	**1.69**	**84.77**	**7.9**
✓	✓	✗	82.53	6.32	84.12	10.1
✓	✗	✓	82.41	4.51	84.22	9.2
✗	✓	✓	81.16	6.82	83.05	9.61

TACRED 的 AP 几乎不变，
AF 却由 1.69 升至 6.32。

在模拟含噪反馈下，可靠性筛选保持准确率，偏好优化减少遗忘

同杰

Yutao Yang, Jie Zhou, et al. Reinforced Interactive Continual Learning via Real-time Noisy Human Feedback, ESWA, 2026.

持续学习的局限与思考

遗忘仍是难题　持续微调仍可能覆盖旧知识，已有能力需要长期保护。

标注难以备齐　预先收集完整的任务或场景标注数据，
与真实应用中的持续变化难以匹配。

回放依然关键　经验回放（Replay）仍是缓解遗忘的有效方法。

强化学习（RL）能否用行动反馈支持持续学习？



華東師範大學
EAST CHINA NORMAL UNIVERSITY

经验驱动终身学习

行动所得，怎样帮助下一次？

已有基础　新数据可以持续更新模型，仍要控制遗忘与反馈噪声。

接下来　从交互中获得经验，再保存、复用并内化有效做法。

RL 提供反馈学习方法，经验还需要保存与复用。

人如何学习知识

The 70:20:10 Framework



数据驱动

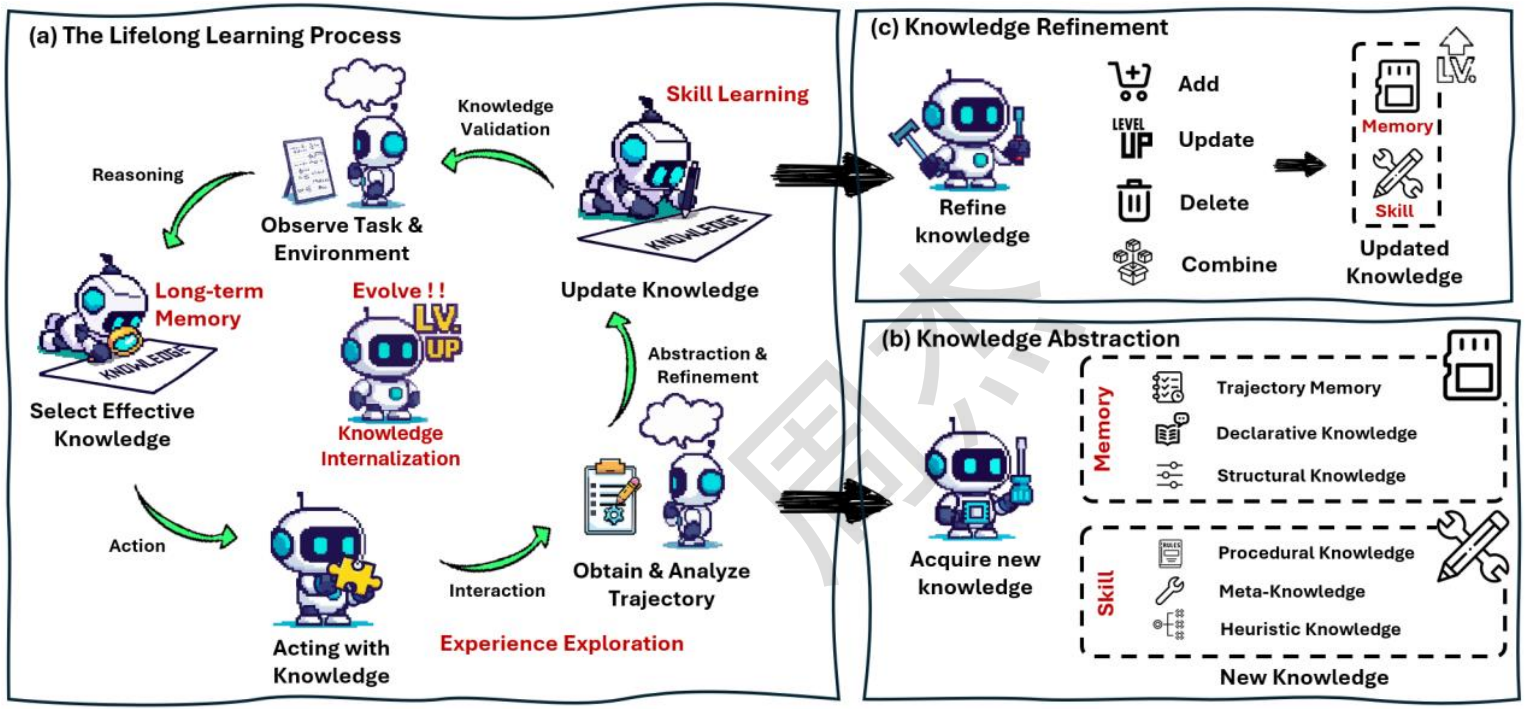
↓

经验驱动

Lombardo, Michael M; Eichinger, Robert W (1996). The Career Architect Development Planner (1st ed.). Minneapolis: Lominger. p. iv. ISBN 0-9655712-1-1.

经验驱动的终身学习

Github：https://github.com/ECNU-ICALK/ELL-StuLife
Webpage：https://ecnu-icalk.github.io/ELL-StuLife/



经验探索

长期记忆

技能学习

知识内化

- 核心理念：机器应从第一人称视角积累**经验并学习**。
- 范式转变：从模仿人类知识输出到**主动探索**和互动。
- 持续进化：通过与环境的持续互动，不断**进化和改进**。
- 关键机制：持续的经验学习机制，包括**探索、记忆、技能和内化**。

Cai, Yuxuan, Hao, Yipeng, Zhou Jie, et al. "Building Self-Evolving Agents via Experience-Driven Lifelong Learning: A Framework and Benchmark." EMNLP, 2026.

StuLife-把AI送去上大学

StuLife

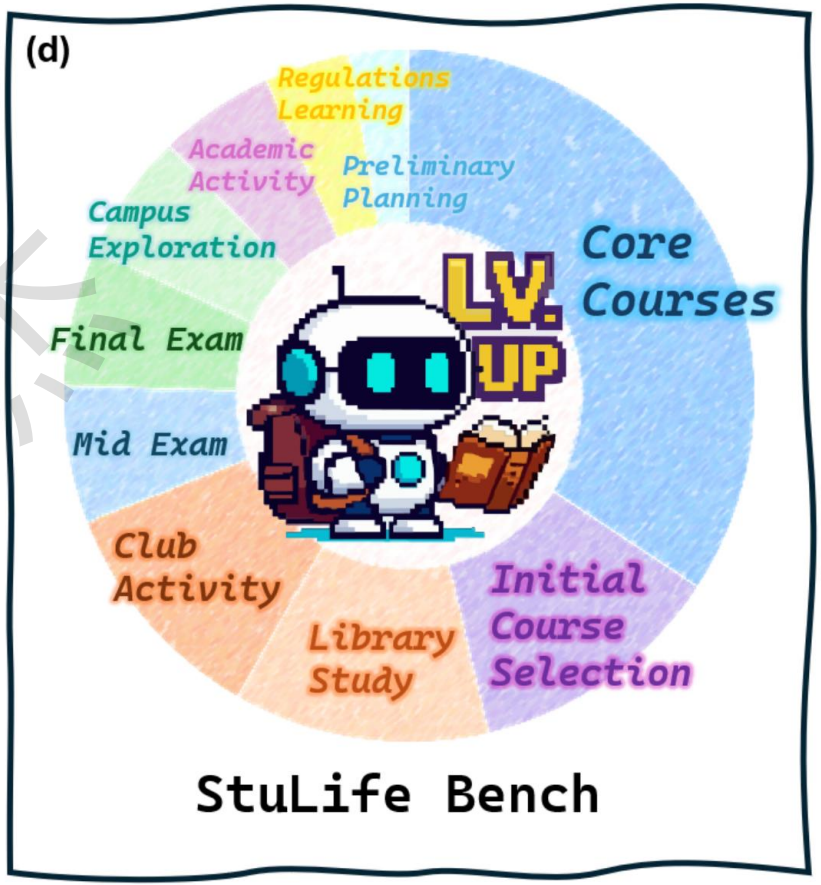
大学生活模拟环境

场景: 模拟学生从入学到学术和个人发展的完整大学旅程。

特点:

- 三个核心阶段（课堂内、日常校园、考试）
- 动态环境，状态变量随时间演变

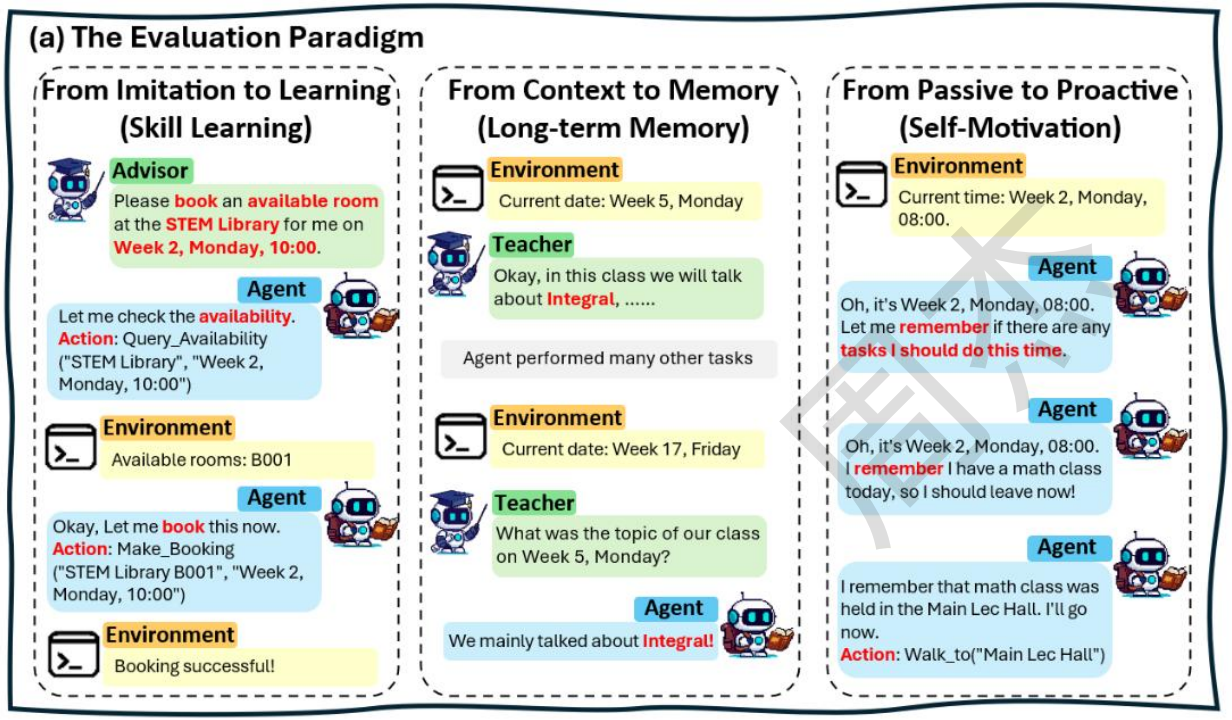
- 课堂内任务：规章制度学习；核心课程教学
- 日常校园任务：校园探索、初步选课；初步规划；学术活动；图书馆资源管理；学生社团参与
- 考试任务：期中考试；期末考试

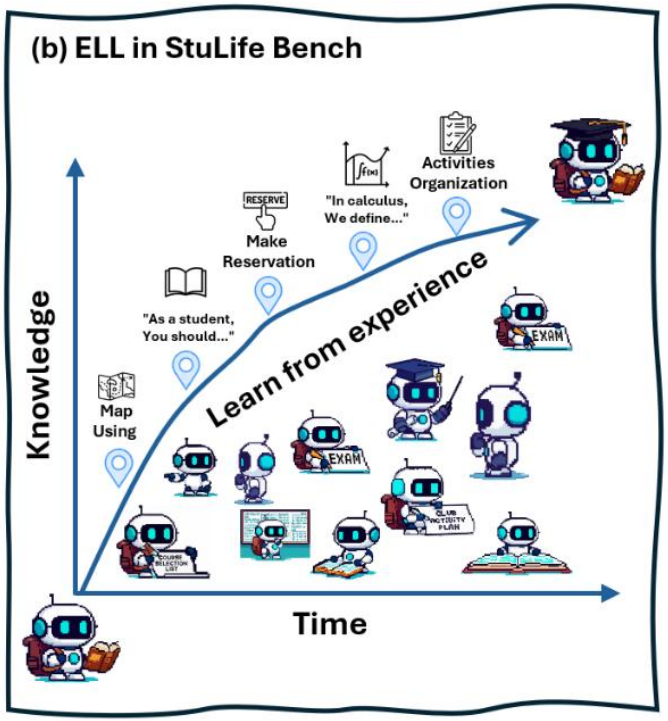


周杰

Cai, Yuxuan, Hao, Yipeng, Zhou Jie, et al. "Building Self-Evolving Agents via Experience-Driven Lifelong Learning: A Framework and Benchmark." EMNLP, 2026.

StuLife-把AI送去上大学





从模仿到学习　　从上下文到记忆　　从被动到主动

Cai, Yuxuan, Hao, Yipeng, Zhou Jie, et al. "Building Self-Evolving Agents via Experience-Driven Lifelong Learning: A Framework and Benchmark." EMNLP, 2026.

现有大模型是否具有持续学习能力？-顶尖AI集体“挂科”

	StuGPA	LTRR	PIS	In-Class		Daily Campus		Exam		Total	
				Success	AvgTurn	Success	AvgTurn	Success	AvgTurn	Success	AvgTurn
Llama-3.1-8B	5.81	3.30	0.90	0.90	61.34	0.00	35.91	10.63	28.46	2.13	44.62
Qwen3-8B	13.31	4.33	0.54	0.90	10.12	8.31	10.25	14.38	6.31	6.71	9.71
Qwen3-30B-A3B	16.30	5.05	0.72	0.60	9.45	10.79	11.75	17.50	5.46	8.31	10.09
Qwen3-32B	7.36	3.97	0.54	0.60	7.80	2.25	13.79	13.13	4.88	3.51	10.41
Qwen3-32B	12.67	5.42	1.26	1.80	8.31	7.64	10.74	17.50	4.94	7.24	9.10
QwQ-32B	13.21	5.78	3.42	4.79	7.72	6.97	13.25	16.88	4.52	7.88	10.06
DeepSeek-V3	11.22	6.14	2.88	3.59	5.84	6.74	11.87	16.25	4.26	7.24	8.64
DeepSeek-R1	14.25	8.30	3.96	5.09	8.04	13.26	13.02	18.13	4.56	11.18	10.08
DeepSeek-V3.1	14.26	4.51	0.54	0.90	14.03	12.81	12.62	15.00	6.78	8.95	12.43
DeepSeek-V3.1	17.04	6.14	3.78	6.29	9.83	12.58	13.03	17.50	5.54	11.18	10.88
Qwen3-235B-A22B	16.03	5.42	1.80	2.10	18.71	10.34	17.17	16.88	10.75	8.52	16.95
Gemini-2.5-Pro	16.43	7.04	3.24	5.39	14.94	18.88	12.78	15.63	9.51	13.53	13.19
Grok4	17.38	10.65	4.50	4.79	6.31	21.80	11.25	18.75	5.69	15.23	8.68
GPT-5	17.90	6.50	4.68	7.78	12.70	14.16	14.31	16.88	6.24	12.35	12.69
Human	85.24	84.91	88.13	88.62	-	91.13	-	82.50	-	88.81	-

- **StuGPA** = 20% Daily + 30% In-Class + 50% Exam
- LTRR：长期记忆任务上效果
- PIS：主动任务上效果

- 即使是最强的模型 (GPT-5) 在 StuLife 上得分仅为 17.9/100
- 揭示了当前AI与人类水平自主学习之间存在巨大鸿沟。
- 现有模型在长期记忆保留和自我驱动行为方面存在根本缺陷。

Cai, Yuxuan, Hao, Yipeng, Zhou Jie, et al. "Building Self-Evolving Agents via Experience-Driven Lifelong Learning: A Framework and Benchmark." EMNLP, 2026.

挑战一：经验探索和奖励获取

- 挑战1: 高效探索与经验获取
  - 设计**内在动机**机制引导的交互
  - 奖励往往**稀疏、延迟或难以定义**



伊利亚（Ilya Sutskever）

这些模型的**泛化能力远远不如人类**。像是具备某些能力的神童，但整体认知上仍然是小孩。

- 现有方法
  - 基于反思的经验探索（一边做一边思考）
  - 基于强化学习的探索（暴力搜索）
  - 基于多智能体的经验探索（众人拾柴火焰高）
  - 基于人在回路的经验探索（交互中学习）

- 如何少样本高效搜索路径？
- 如何构建类人价值函数（如活下去）？
- 如何一直保持好奇心？

ReAct: Synergizing Reasoning and Acting in Language Models, ICLR, 2023

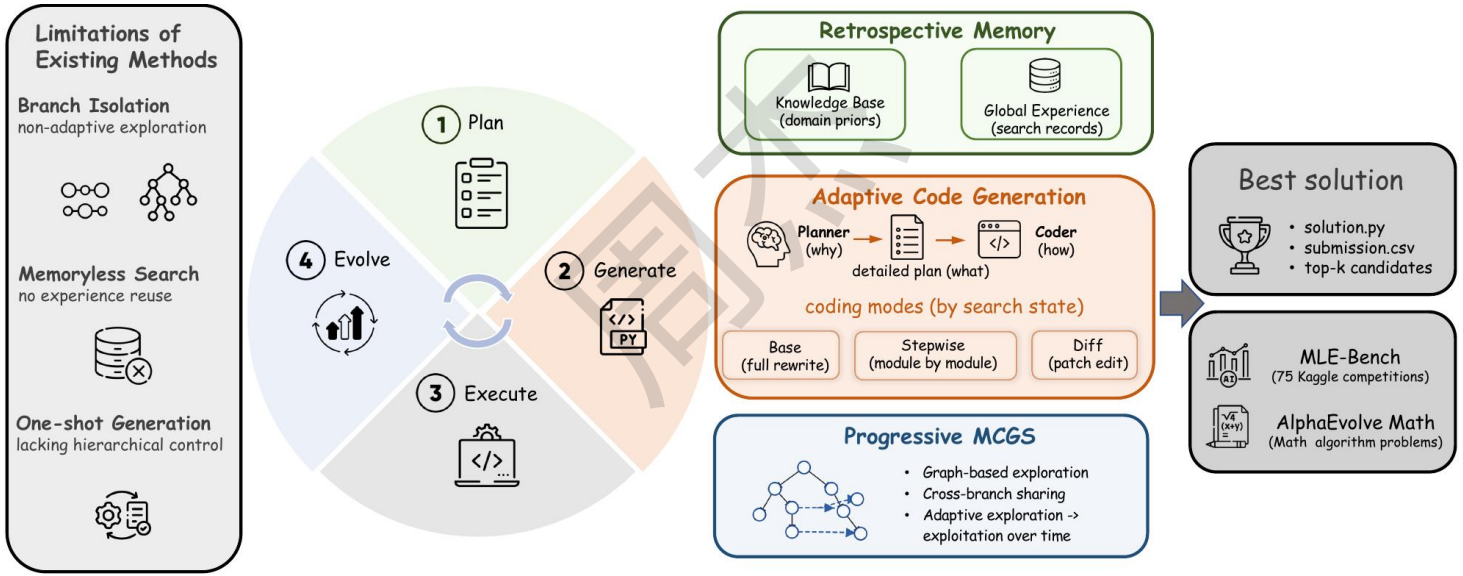
Reflexion：Language Agents with Verbal Reinforcement Learning，NeurIPS 2023

The Landscape of Agentic Reinforcement Learning for LLMs: A Survey, 2025

周木

MLEvolve：每次试验都应帮助下一次改进

在算法发现中，每次试验都会留下代码、分数与失败原因。
MLEvolve 利用这些记录搜索和修改候选算法，让各分支共享成败经验。



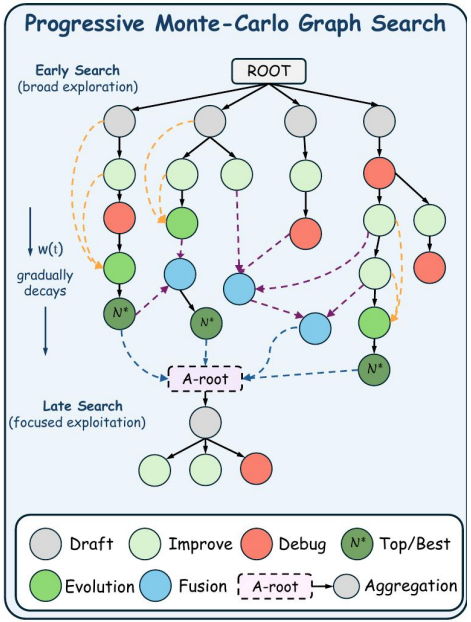
分支各自搜索，容易重复试错；只看分数，也难以知道下一步怎么改。

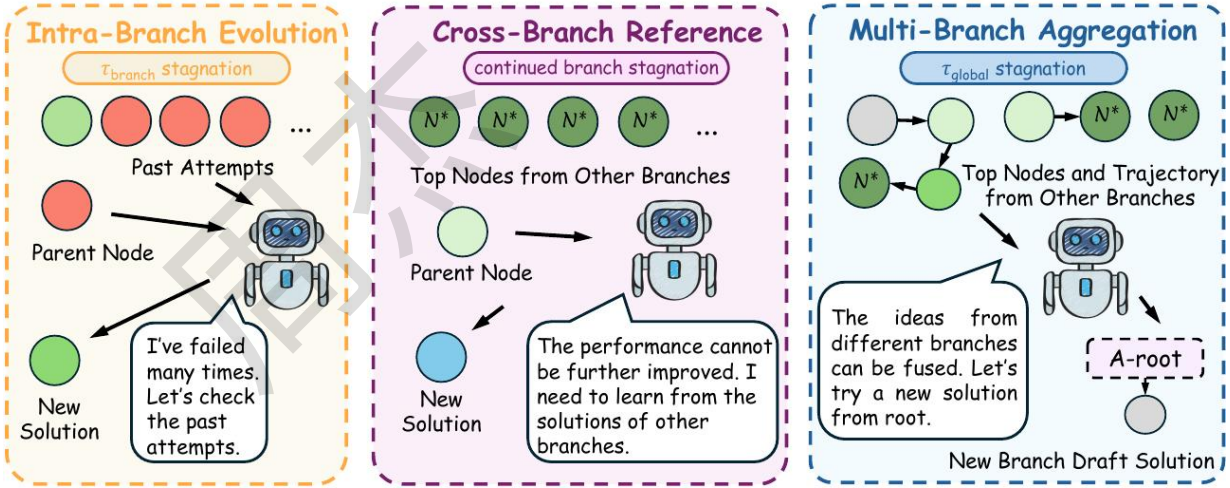
让一次算法试验留下可借鉴的经验，再用经验改进后续候选

Shangheng Du, Xiangchao Yan, et al. MLEvolve: A Self-Evolving Framework for Automated Machine Learning Algorithm Discovery, 2026.

MLEvolve：共享搜索经验，逐步聚焦强方案

不同分支各自摸索，容易浪费已经获得的经验。
MLEvolve 用图连接可相互借鉴的方案，并随搜索推进逐步收拢探索范围。





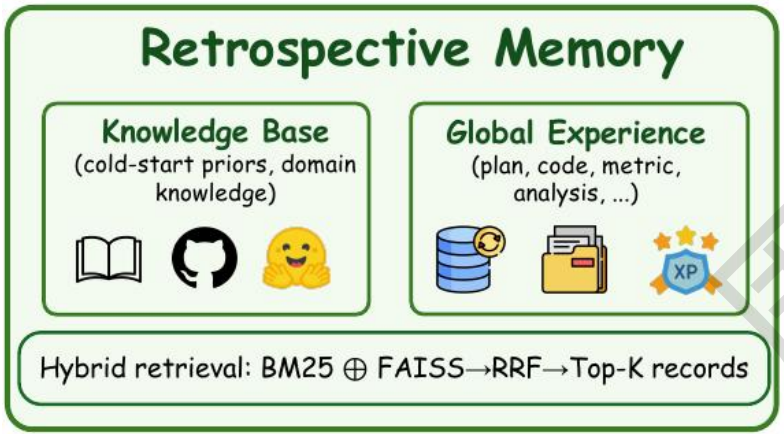
分支内回看 → 跨分支借鉴 → 聚合多分支重新出发

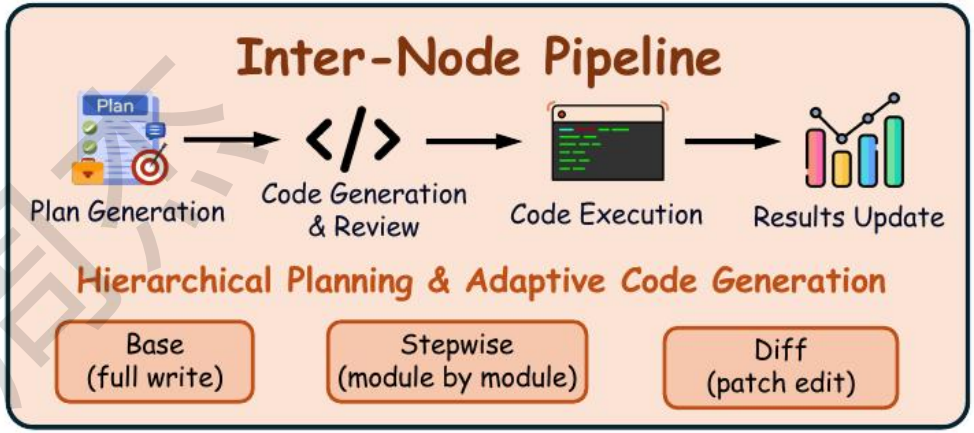
前期广泛尝试，后期聚焦强方案，停滞时借鉴其他分支的经验

Shangheng Du, Xiangchao Yan, et al. MLEvolve: A Self-Evolving Framework for Automated Machine Learning Algorithm Discovery, 2026.

MLEvolve：先判断改什么，再决定怎么改

分数只能说明结果好坏，不能指出哪些代码值得保留或修改。
因此先检索成败经验、形成修改计划，再按问题选择重写、分步改动或局部补丁。





经验用在两个时刻

规划时参考成败记录，
报错时寻找已解决的相似问题。

改动幅度由问题决定

无可靠基础才重写；复杂变更分步做；
局部优化尽量只改相关代码。

经验指导修改方向，分层代码改动减少重复试错与无关变化

Shangheng Du, Xiangchao Yan, et al. MLEvolve: A Self-Evolving Framework for Automated Machine Learning Algorithm Discovery, 2026.

MLEvolve：有限预算下，后续探索仍有收益

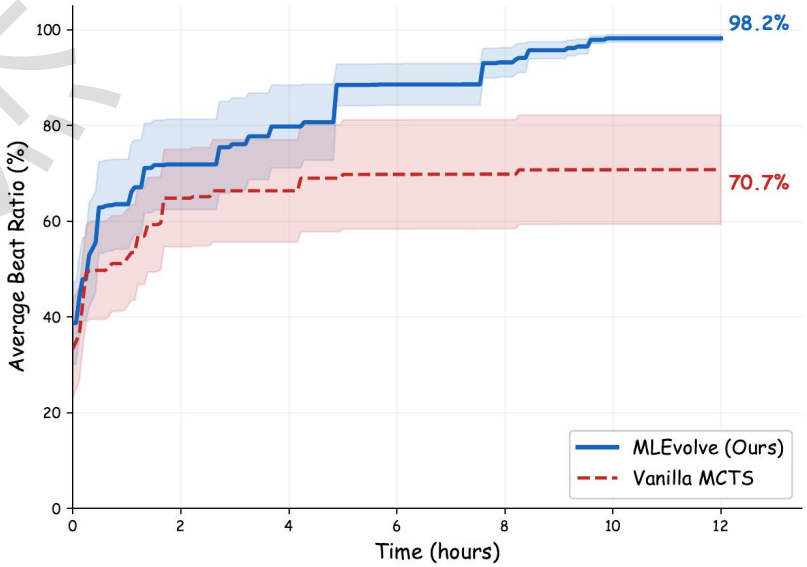
在 MLE-Bench 的 75 项任务上，MLEvolve 的获奖率达到 65.3%。
代表性任务的搜索曲线还显示，增加搜索时间后仍能持续找到更好的方案。

12 小时，65.3% 的任务获奖

原文表1节选 · 不同模型配置的系统比较

Agent	Time (h)	All (%)
MARS+ [18]		
Gemini-3-Pro-preview	24	62.7±0.8
AIBuildAI [25]		
Claude-Opus-4.6	24	63.1±0.4
MLEvolve (ours)		
Gemini-3.1-Pro-preview	**12**	**65.3±0.8**

后半程继续超过更多人类参赛者



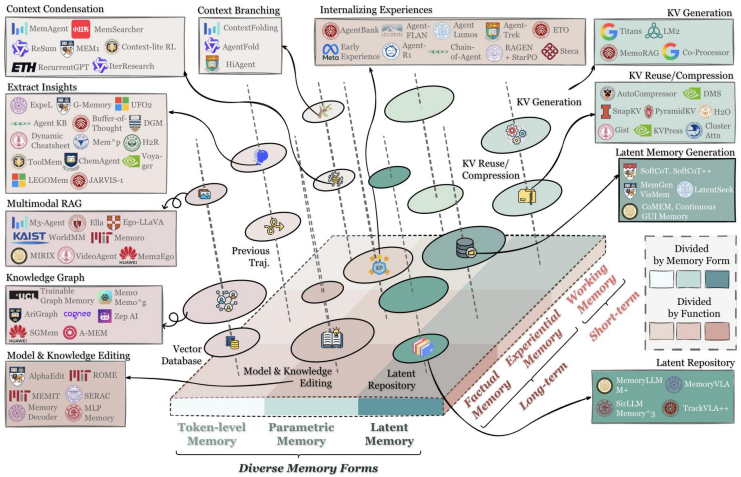
一次搜索中的成败经验能够反复利用，帮助后续试验继续改进候选算法

周木

Shangheng Du, Xiangchao Yan, et al. MLEvolve: A Self-Evolving Framework for Automated Machine Learning Algorithm Discovery, 2026.

挑战二：长期记忆

- 挑战2: 长期记忆与关联回忆
  - 如何构建可扩展、易访问的**长期记忆**?
  - 如何实现跨事件的**关联回忆**，克服灾难性遗忘?
- 现有方法
  - 上下文工程：将记忆以文本等形式存储
    - 外部知识库（RAG、MemOS）
    - 自进化上下文（"全面、演变的操作手册"）
  - 参数化记忆：将记忆转化为参数化记忆
    - MemGen



功能：事实记忆、经验性记忆、工作记忆

记忆形式：Token级别，参数记忆、隐式记忆

类人联想记忆？

主动回忆？

周杰

Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models, 2025, Stanford
MemGen: Weaving Generative Latent Memory for Self-Evolving Agents, 2025
Hu Y, Liu S, Yue Y, et al. Memory in the Age of AI Agents: A Survey

MemPro：记忆越积越多，读写流程也要改进

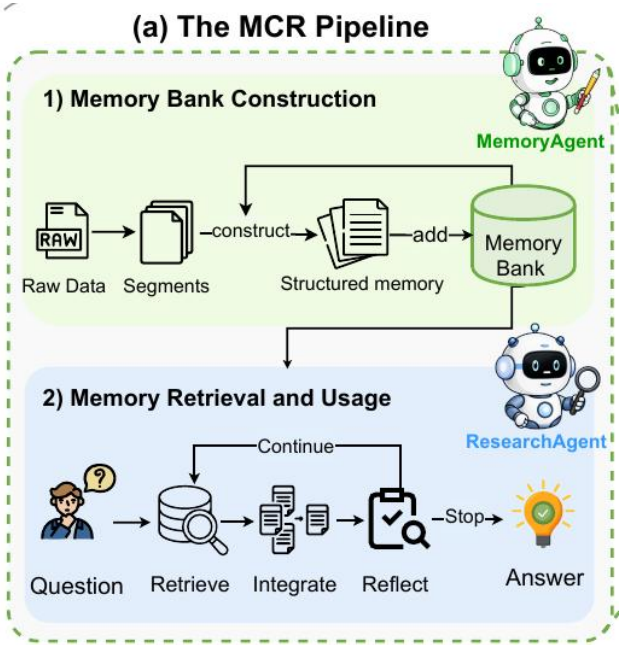
探索留下了经验，长期使用还需要可靠的存储、检索与证据整合。
MemPro 把整套记忆流程当作可修改的程序，让读写方法适应变化的任务与记忆库。

不同问题，需要不同的记忆策略

问“后来发生了什么”，要理清时间。
问“跨几次对话得出的结论”，要串联证据。

记忆库变了，旧流程可能跟不上

记忆规模和结构持续变化，
检索深度、证据整合方式也需要调整。



长期记忆既要更新内容，也要改进组织与使用这些内容的方法

Qingshan Liu, Guoqing Wang, et al. MemPro: Agentic Memory Systems as Evolvable Programs, EMNLP, 2026.

MemPro：从失败出发，逐步改好记忆程序

一次修改可能修好当前问题，也可能损害已有能力，因此需要保留可比较的版本。
MemPro 在离线样例上诊断、修改和评估，再从有潜力的历史版本继续探索。

1 选版本，也选改进方向

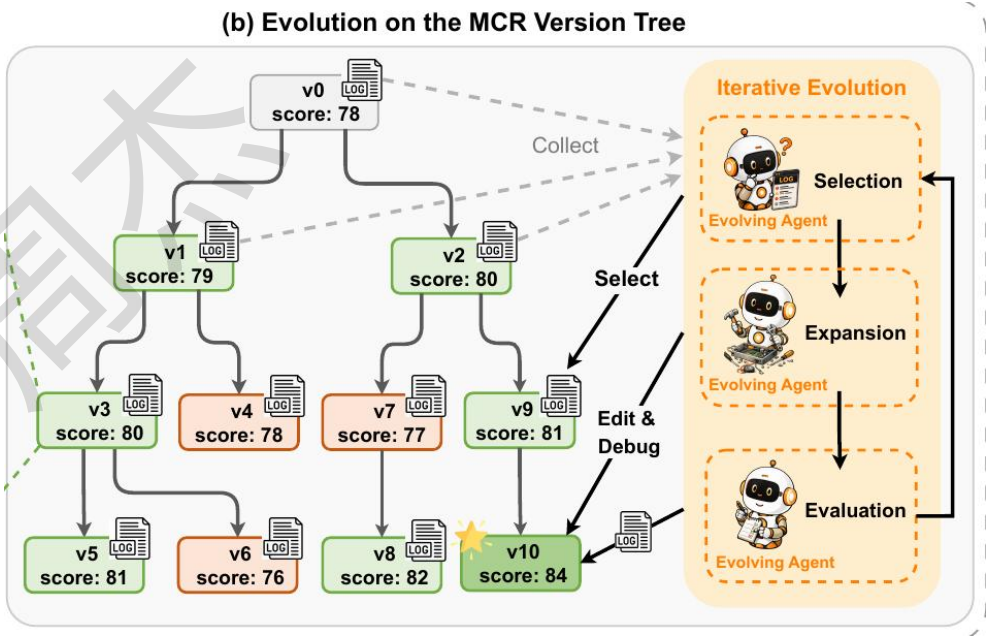
结合总分、分类表现和失败记录，
找到值得继续改的版本。

2 改提示和代码，边改边调试

用典型失败样例运行和定位问题，
反复修改后形成新版本。

3 完整评估，写回版本树

新版本的效果与诊断日志，
共同决定下一轮从哪里出发。

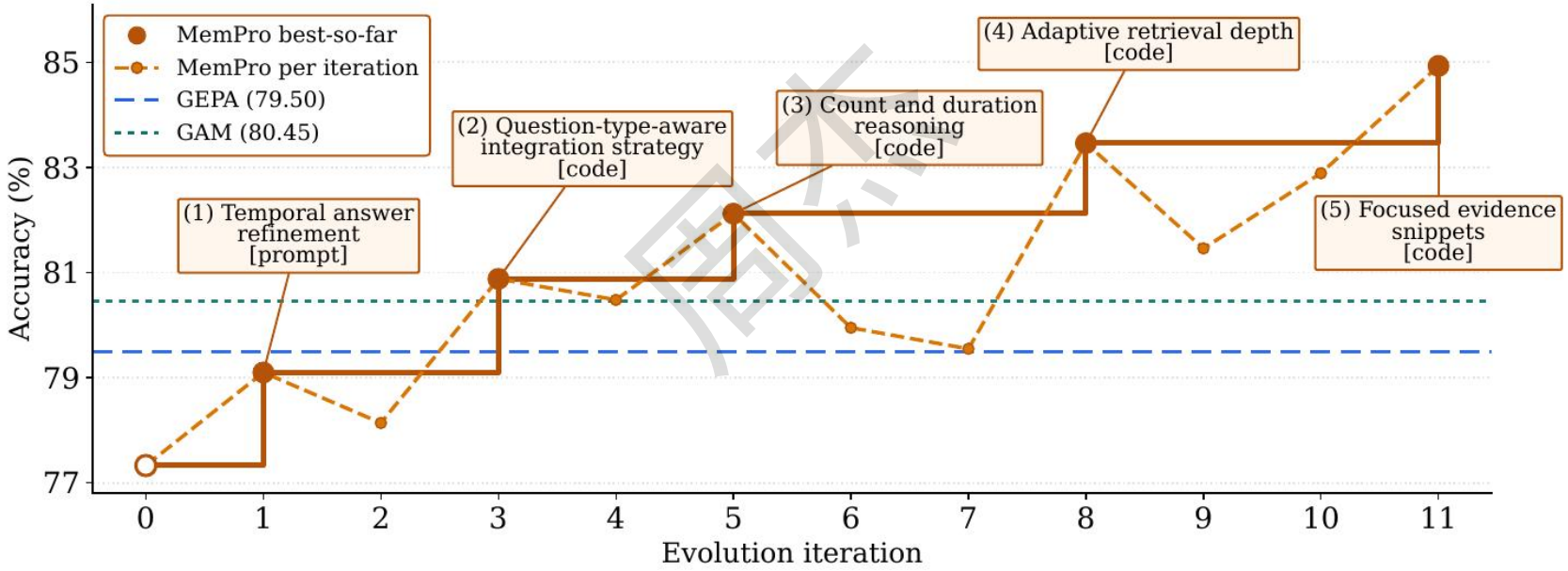


版本树保存有效进展，失败诊断为下一次程序修改提供方向

Qingshan Liu, Guoqing Wang, et al. MemPro: Agentic Memory Systems as Evolvable Programs, EMNLP, 2026.

MemPro：记忆收益来自具体的流程改进

LoCoMo 的演化轨迹显示，改进逐步从时间提示深入到可执行的处理流程。
按问题整合证据、计算次数与时长、调整检索深度，都能成为下一次修改的方向。



单轮修改可能起伏，保留有效版本才能把局部进步积累下来

Qingshan Liu, Guoqing Wang, et al. MemPro: Agentic Memory Systems as Evolvable Programs, EMNLP, 2026.

MemPro：改进整套流程，收益超过只改提示

长期记忆实验中，5 轮进化已在两种基座、两个基准的平均得分上领先。
消融进一步表明，代码修改、版本分支与反复迭代都对效果有贡献。

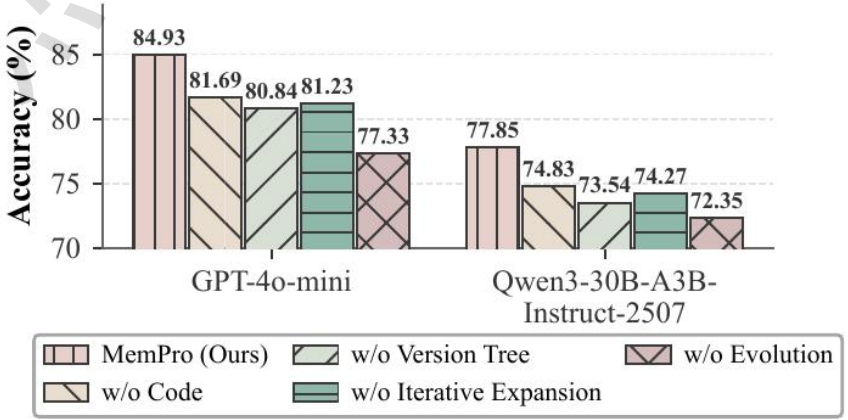
从第 5 轮到第 15 轮仍在提升

原文表1节选 · GPT-4o-mini · 平均得分

方法	LongMemEval	LoCoMo
GAM	72.40	80.45
GEPA	75.60	79.50
MemPro-5	75.77†	81.94†
MemPro-10	77.11	83.29
MemPro-15	**79.00**	**84.93**

只改提示、只沿最新版本改，都更弱

原文图3 · LoCoMo 消融实验

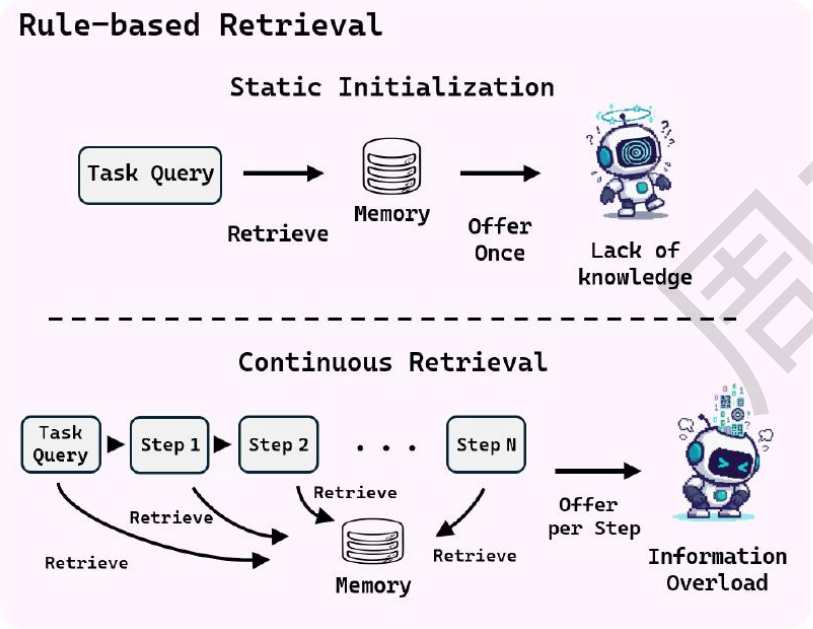


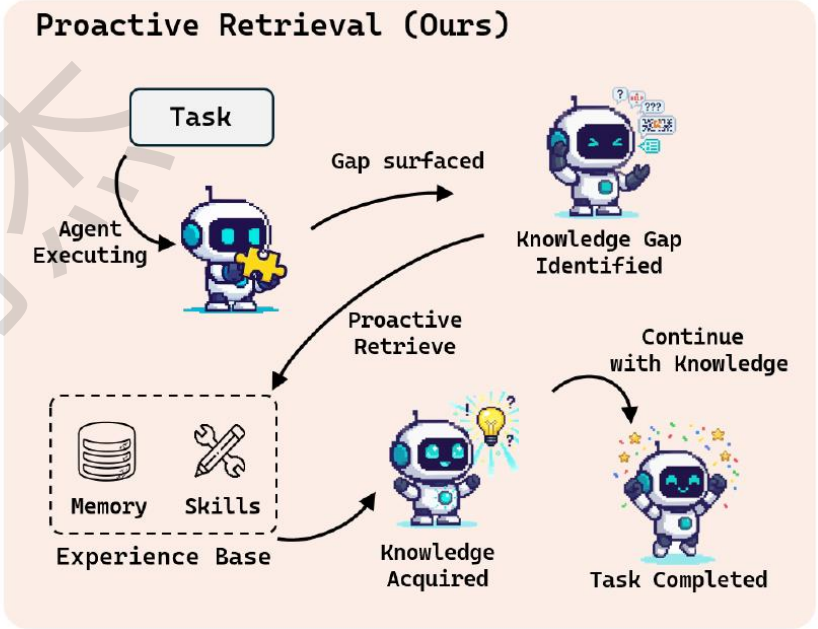
代码修改、版本分支与多轮迭代共同提升记忆效果，收益超过只改提示

Qingshan Liu, Guoqing Wang, et al. MemPro: Agentic Memory Systems as Evolvable Programs, EMNLP, 2026.

ProactAgent：需要经验时，智能体应主动查找

记忆系统能够检索，不代表每次任务都能在合适的时刻用上它。
ProactAgent 把“何时查、查什么”作为可学习的动作，避免只在开头查或每步都查。



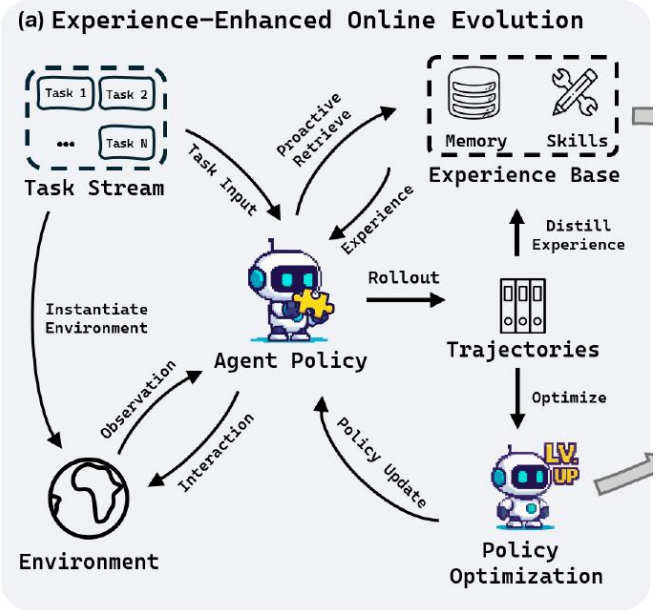


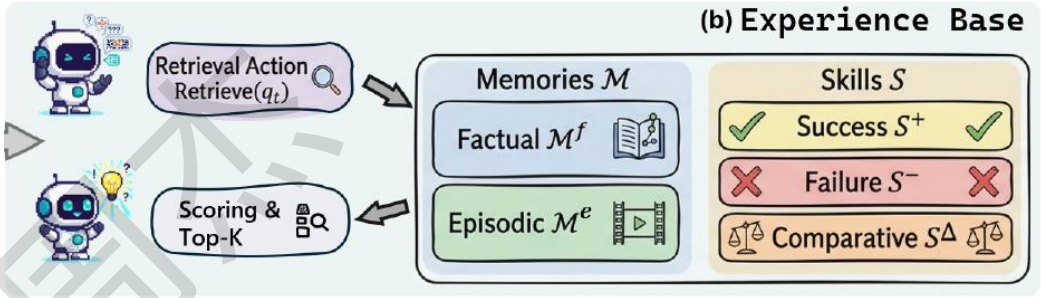
先识别当前缺少什么经验，再主动检索来支持下一步行动

Yuxuan Cai, Wei Li, et al. Ask Only When Needed: Proactive Retrieval from Memory and Skills for Experience-Driven Lifelong Agents, EMNLP, 2026.

ProactAgent：事实补信息，技能给做法

事实回答“发生过什么”，技能帮助判断“接下来怎么做”。
经验库分别组织这些内容，同时用交互更新策略，让可用经验与行动能力共同积累。





一次检索，兼顾证据和做法

事实记忆与经历记忆：现在是什么、以前发生过什么。
成功、失败与对比技能：怎么做、避开什么、为何更好。

完成交互 → 提炼经验 → 强化学习更新策略

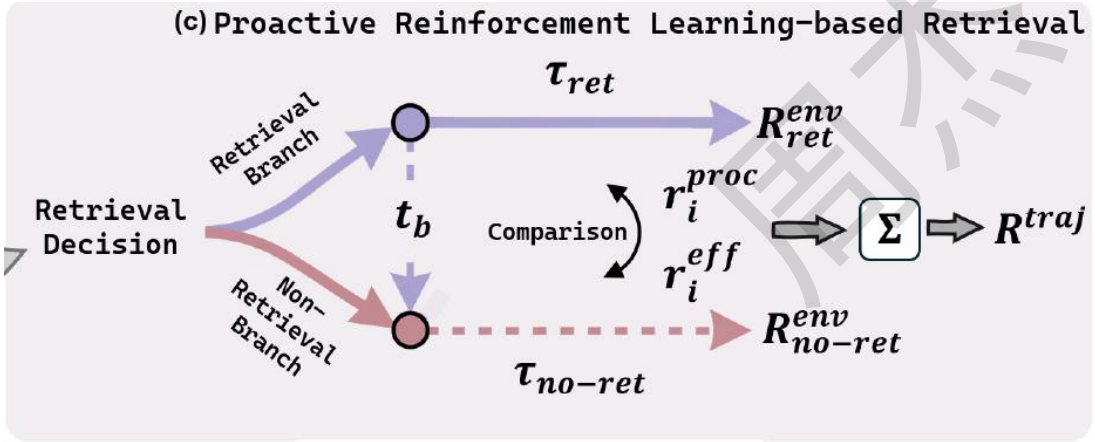
经验库提供依据，策略更新改善行动，两者共同支持后续任务

Yuxuan Cai, Wei Li, et al. Ask Only When Needed: Proactive Retrieval from Memory and Skills for Experience-Driven Lifelong Agents, EMNLP, 2026.

ProactAgent：检索是否有用，要有同起点对照

最终任务成功，并不能直接证明中途那次检索值得做。
从同一段交互历史出发，对照检索与不检索的后续结果，才能为检索动作分配奖励。

同一历史，只改变分支点的检索选择



任务结果更好，或完成得更快

按结果和步数计算分支差异，
收益为正才给正向过程奖励。

效果变差就扣分，持平为零

同时抑制重复查询，
避免检索本身成为额外负担。

用 GRPO 联合训练检索与任务动作

根据检索带来的实际收益学习时机，减少无帮助的重复查询

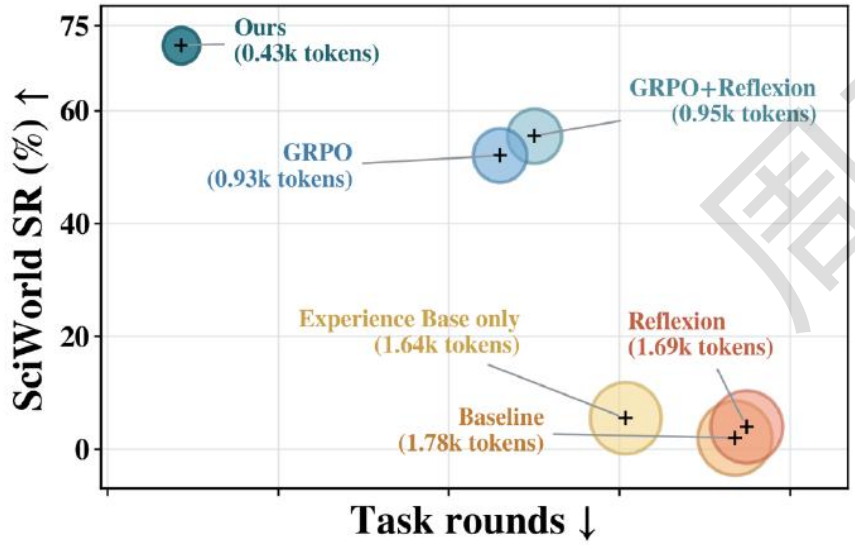
周杰

Yuxuan Cai, Wei Li, et al. Ask Only When Needed: Proactive Retrieval from Memory and Skills for Experience-Driven Lifelong Agents, EMNLP, 2026.

ProactAgent：有选择地检索，任务更准也更快

在 SciWorld 上，7B 模型成功率达到 73.5%，比 GRPO+Reflexion 高 18 个百分点。
交互轮数同时减少 33.2%，3B 模型也能从主动检索中获益。

成功率更高，轮数与上下文开销更低



3B 模型接近 7B 被动检索基线

原文表4 · SciWorld · SR 为成功率

Method	SR ↑	Rounds ↓
GRPO (3B)	24.00	24.14
GRPO+Reflexion (3B)	29.50	20.09
GRPO+Reflexion (7B)	55.50	27.52
ProactAgent (3B)	**53.50**	**14.35**

53.5% 对 55.5%

接近 7B 基线，交互轮数减少约 48%

在这组实验中，主动检索让模型用更少交互获得更高成功率

周杰

Yuxuan Cai, Wei Li, et al. Ask Only When Needed: Proactive Retrieval from Memory and Skills for Experience-Driven Lifelong Agents, EMNLP, 2026.

挑战三-技能抽象和管理

- 挑战3: 技能抽象与管理
  - 如何**动态管理技能**生命周期（获取、验证、调用、演化）？
  - 如何将显式规则知识转化为直觉、泛化的**内化**能力?
- 现有方法
  - 技能学习（如何抽取可泛化的技能？）
    - Reasoningbank，从失败和成功的轨迹抽取经验
  - 技能验证和评测（如何验证技能有效性？）
  - 技能内化（参数化技能）



Andrej Karpathy

AI是无法成长的“幽灵”，因为**缺乏从经验中提取核心再写入自身参数**的机制。

周杰

Contextual Experience Replay for Self-Improvement of Language Agents, ACL 2025
ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory, 2025, Google

AutoSkill：用户教过的做法，下次就能复用

有了按需检索，还需要把用户反复强调的做法整理成真正可用的技能。
AutoSkill 从对话中提炼措辞、格式与工作习惯，让后续任务少一次重复纠正。

专业改写中，用户在意什么？

保留原意，数字、姓名、日期不能改
语言专业流畅，但不要堆砌术语
只给改写后的正文，不附带解释

这些要求会在相似任务中反复出现，
每次重新交代，会增加协作成本。

沉淀成一项“专业改写”技能

何时用：遇到同类改写请求
怎么做：保留信息，改善表达
怎样交付：遵守已确认的输出规范

把聊天中的要求写成可执行的规则，
在新任务到来时按需检索与调用。

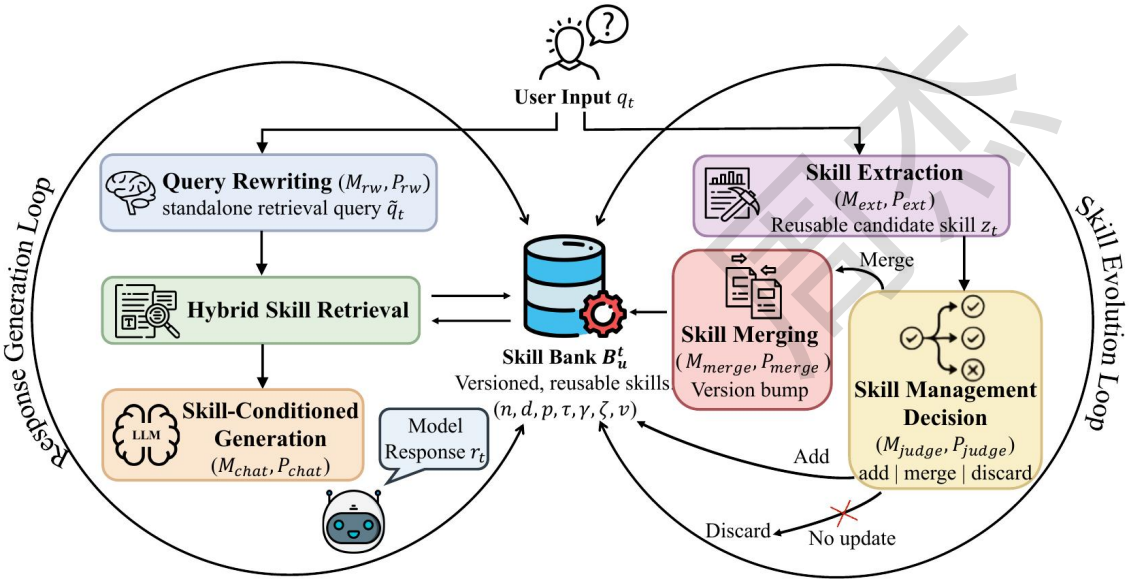
把反复确认的做事规则保存为技能，让下一次任务接着受益

周杰

Yutao Yang, Junsong Li, et al. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution, Lifelong Learning Agent Workshop@COLM, 2026.

AutoSkill：完成当前任务，也持续维护技能

提炼出技能之后，还要让它在合适的任务中被调用，并吸收新的用户反馈。
AutoSkill 分开处理当前响应与后台维护，使日常交互能够持续改进技能库。



回答时：只带入相关技能

结合当前问题和近期对话检索，
匹配到的技能作为行动指导。

交互后：判断经验如何保留

从用户要求中提炼候选技能，
决定新增、合并或丢弃；
修订已有技能时保留版本记录。

后台异步维护，无需训练模型参数。

技能在任务中被调用，在反馈后被修订，才能长期融入工作流程

Yutao Yang, Junsong Li, et al. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution, Lifelong Learning Agent Workshop@COLM, 2026.

AutoSkill：同一项技能，持续吸收 34 次修订

每条新反馈都单独保存，容易让同一项能力分散在重复甚至冲突的规则里。
论文中的改写技能更新到 0.1.34，展示了持续维护同一项能力的过程。

原文案例：professional_text_rewrite

Skill Card: professional_text_rewrite

ID: `a407043f-d6b0-4760-821e-86b538c149c1` **Version:** `0.1.34`

Description: Rewrites user-provided text to enhance fluency, professionalism, and grammatical correctness, strictly preserving the original meaning, intent, and all details by rephrasing in new words.

Tags:

rewrite | editing | professional | paraphrase | fluency | clarity

Triggers:

rewrite this professionally | improve this text | make this more formal | rewrite this in your own words | paraphrase this text

周杰

版本保留演化轨迹，技能文件可查看、可修改。

把容易遗漏的要求写清楚

内容边界
保留事实、数字、姓名和日期。

表达方式
清楚、专业，不改变原意。

交付规范
只输出改写正文；按需给备选。

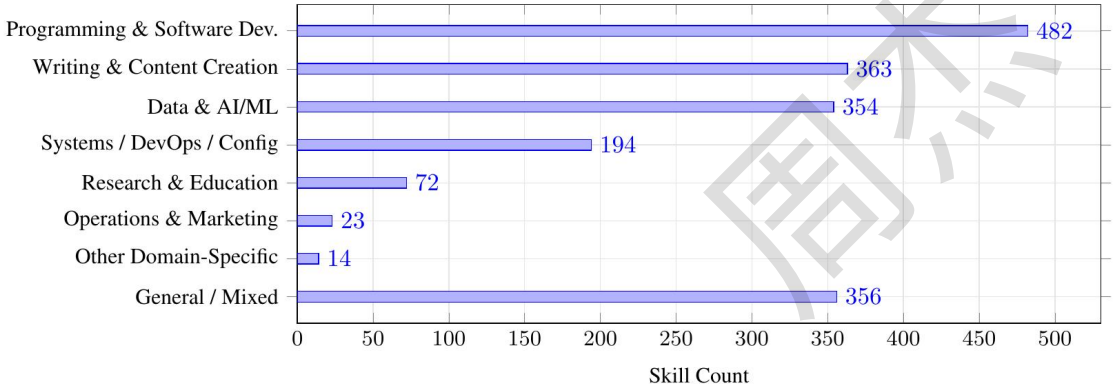
后续纠正应回到已有技能中，让规则逐步完善，而不是不断重复新建

Yutao Yang, Junsong Li, et al. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution,, Lifelong Learning Agent Workshop@COLM 2026.

AutoSkill：已有对话可以启动长期技能积累

在 WildChat 真实对话中，AutoSkill 提取出 1,858 项技能，覆盖多类工作需求。
历史对话可以用于启动技能库，后续交互再持续补充与修订。

真实交互能够提供多样的技能来源



编程 482 项 · 写作 363 项 · 数据与 AI 354 项

github.com/ECNU-ICALK/AutoSkill

接入已有应用

SDK 与兼容 OpenAI 的接口，
让技能层进入现有调用流程。

让经验成为可维护的文件

SKILL.md 保存规则与版本；
支持个人库、共享库与导出，
便于检查、修改和迁移。

对话经验能长期保留和复用，专业文档中的操作经验也可以进一步整理

周杰

Yutao Yang, Junsong Li, et al. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution, Lifelong Learning Agent Workshop@COLM, 2026.

Anything2Skill：专业文档也是技能来源

技能不必只靠用户逐次纠正，手册、案例与操作记录中也有成熟的专业做法。
Anything2Skill 将这些分散知识整理为可检索、可执行、可持续维护的技能。

找到相关内容，仍有执行上的距离

以命令行操作为例：
文档提供命令、参数和用法，
任务却要求把它们组织成完整流程。

先做什么、何时调用、怎样检查，
仍需要模型在每次任务中重新推断。

提前把“这件事怎么做”整理好

何时适用，何时不该用
按什么顺序执行，遵守哪些约束
需要产出什么，依据来自哪里

任务到来时，检索相应技能，
结合当前资料指导智能体执行。

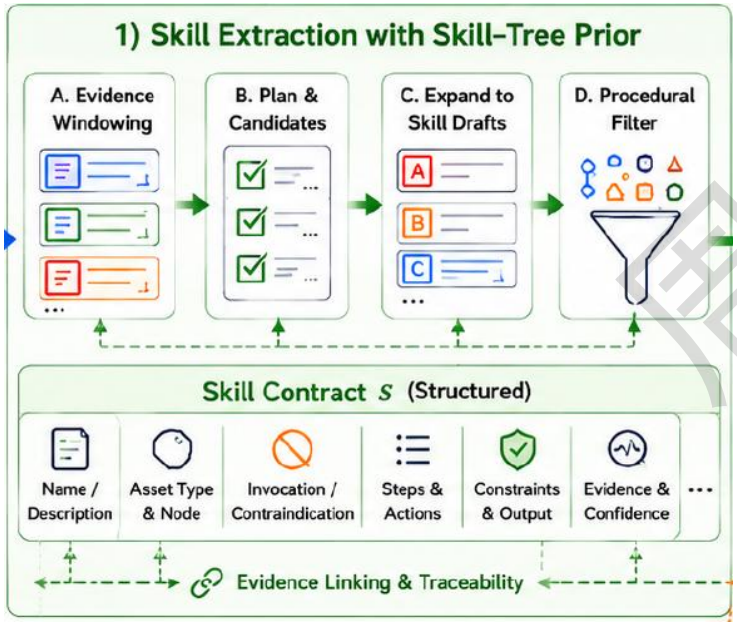
周杰

知识库除了提供资料，还可以为智能体提供可复用的操作方法

Qianjun Pan, Yutao Yang, et al. Anything2Skill: Compiling External Knowledge into Reusable Skills for Agents, Lifelong Learning Agent Workshop@COLM, 2026.

Anything2Skill：技能要写清适用条件与做法

文档摘要并不等于可执行技能，关键步骤、适用边界与交付要求都不能缺失。
先按技能层级规划候选，再依据原文逐项展开，让每项做法都能追溯到证据。



先分清层次，避免混成一团

总体流程、具体操作与约束规则各有位置，
技能树帮助确定“这条经验属于哪类能力”。

再展开做法，保留可执行的内容

明确触发条件、禁用条件、动作与输出；
缺少原文支持的候选，不补写成完整流程。

每项技能关联来源片段，
便于后续检查、纠错和更新。

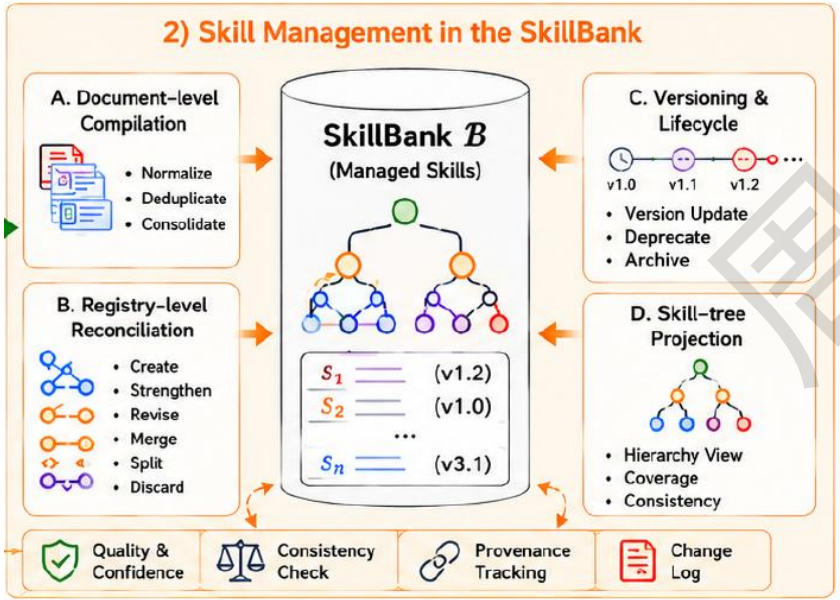
可用的技能需要明确的执行步骤、适用边界与来源依据

周杰

Qianjun Pan, Yutao Yang, et al. Anything2Skill: Compiling External Knowledge into Reusable Skills for Agents, Lifelong Learning Agent Workshop@COLM, 2026.

Anything2Skill：技能越多，越需要版本管理

110 篇 GitHub-CLI 文档被整理为 179 项技能，覆盖总体流程与具体命令操作。
规模扩大后，去重、版本更新与层级组织决定了这些技能能否长期使用。



相同能力归并，新的证据持续补充

先合并文档中的重复描述，再与已有技能比对；
决定新建、增强、修订、合并或拆分。

内容有版本，变化有来源

保留证据、状态与修改历史；
旧技能可以更新或退役，避免重复堆积。

技能树便于查找与检查

按领域与层级组织，清楚展示已有的能力。

技能库要能合并重复内容、吸收新证据，并保留可追溯的版本变化

周杰

Qianjun Pan, Yutao Yang, et al. Anything2Skill: Compiling External Knowledge into Reusable Skills for Agents, Lifelong Learning Agent Workshop@COLM, 2026.

Anything2Skill：资料与技能组合，执行更可靠

在 CSV 处理与 GitHub 仓库操作中，原文比较了文档、技能及两者组合。
RAG 与技能提供不同帮助，组合后两项任务的成功率都最高。

任务成功率 · 基础模型：GPT-5.4-mini

qsv：CSV 操作　GitHub-CLI：仓库管理

Method	qsv	GitHub-CLI
Base Agent	81.60%	64.70%
RAG	95.41%	76.50%
ANYTHING2SKILL	91.95%	82.30%
ANYTHING2SKILL + RAG	**98.85%**	**94.10%**

qsv 中，单独使用 RAG 已达到 95.41%；
加入技能后，进一步提高到 98.85%。

复杂操作中，技能的帮助更突出

GitHub-CLI 中：
仅技能 82.30%，高于
仅 RAG 的 76.50%。

组合达到 94.10%

在 RAG 基础上再提高
17.60 个百分点。

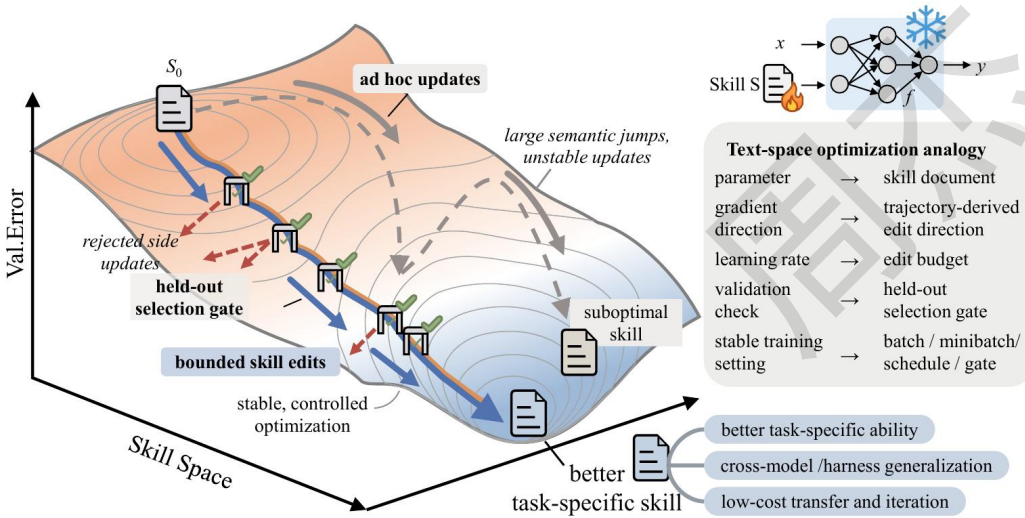
把资料和操作经验一起交给智能体。

资料补充当前信息，技能提供执行方法，后续还要用任务结果检验这些做法

Qianjun Pan, Yutao Yang, et al. Anything2Skill: Compiling External Knowledge into Reusable Skills for Agents, Lifelong Learning Agent Workshop@COLM, 2026.

SkillOpt：技能写好之后，还能继续被优化

从对话与文档中提炼出技能，只是形成了一个可以使用的起点。
SkillOpt 根据执行反馈小步修改技能，再用独立验证集决定是否保留新版本。



Text-space optimization analogy

parameter	→	skill document
gradient direction	→	trajectory-derived edit direction
learning rate	→	edit budget
validation check	→	held-out selection gate
stable training setting	→	batch / minibatch/ schedule / gate

- better task-specific ability
- cross-model /harness generalization
- low-cost transfer and iteration

用真实执行，找到该改的规则

从一批成功与失败轨迹中，
归纳可复用的做法与错误。

每次少改一点，避免越改越乱

限制新增、删除、替换的条数，
保留已经有效的技能内容。

验证确实更好，才保留新版

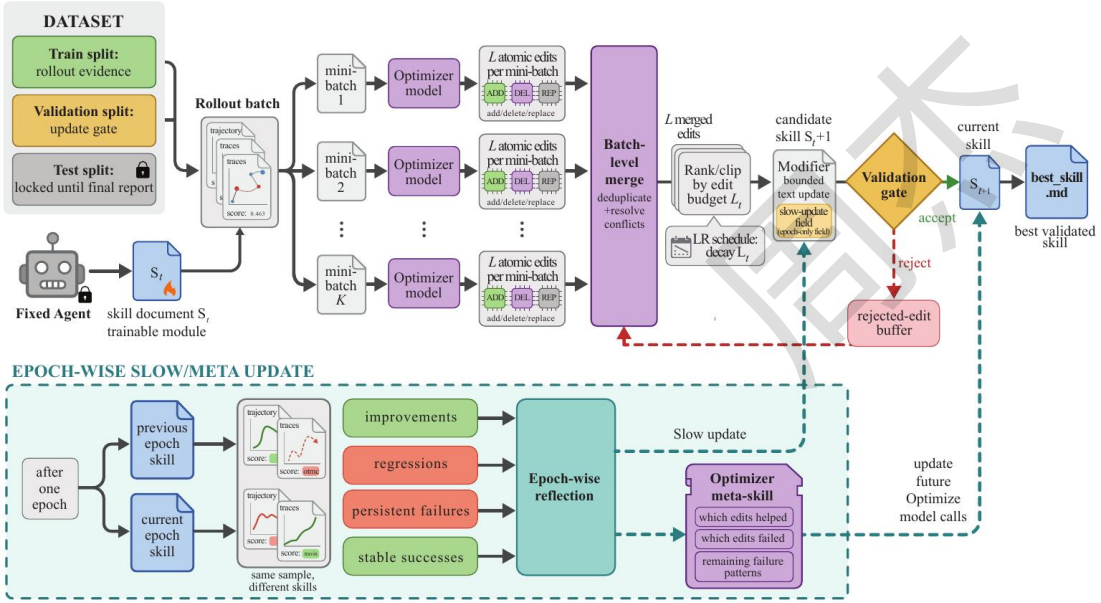
在独立验证集上，分数严格
提高才接受，否则保留原技能。

把技能作为可优化的外部状态，固定模型权重也能持续改进执行方法

Yifan Yang, Ziyang Gong, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills, 2026.

SkillOpt：小步修规则，跨轮保留有效经验

只围绕当前错误反复重写，容易丢掉原先有效的规则。
SkillOpt 在批次内限制改动，跨轮比较新旧技能，把更稳定的经验保存下来。



批次内：有证据地小步改

成功轨迹保留有效做法，
失败轨迹定位缺失规则。
优化模型合并建议、限制改动，
验证更好才更新，失败则记账。

跨轮次：让经验留下来

用相同任务比较新旧技能，
把稳定经验写入受保护区域。
元技能记录哪些编辑有效，
只指导优化，不随技能部署。

局部更新修问题，跨轮更新留经验，部署时仍只使用一份技能文档

Yifan Yang, Ziyang Gong, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills, 2026.

SkillOpt：强模型也受益于更好的执行规则

固定 GPT-5.5 的权重，仅优化技能文档，六项任务平均提高 23.5 个百分点。
表格与办公问答的收益尤其明显，说明执行流程仍有可改进的空间。

Model	Skill source	SearchQA	Spreadsheet	OfficeQA	DocVQA	LiveMath	ALFWorld
No harness / direct chat							
GPT–5.5	No skill	77.7	41.8	33.1	78.8	37.6	83.6
	Human skill	81.8$_{+4.1}$	72.9$_{+31.1}$	66.9$_{+33.8}$	90.1$_{+11.3}$	38.4$_{+0.8}$	91.8$_{+8.2}$
	LLM skill	80.9$_{+3.2}$	43.2$_{+1.4}$	51.7$_{+18.6}$	89.6$_{+10.8}$	40.0$_{+2.4}$	93.3$_{+9.7}$
	Trace2Skill	82.4$_{+4.7}$	49.6$_{+7.8}$	65.7$_{+32.6}$	90.6$_{+11.8}$	52.0$_{+14.4}$	87.3$_{+3.7}$
	TextGrad	81.4$_{+3.7}$	41.1$_{-0.7}$	42.0$_{+8.9}$	87.2$_{+8.4}$	49.2$_{+11.6}$	82.8$_{-0.8}$
	GEPA	84.8$_{+7.1}$	73.6$_{+31.8}$	63.9$_{+30.8}$	89.1$_{+10.3}$	43.2$_{+5.6}$	85.8$_{+2.2}$
	SkillOpt	**87.3**$_{+9.6}$	**80.7**$_{+38.9}$	**72.1**$_{+39.0}$	**91.2**$_{+12.4}$	**66.9**$_{+29.3}$	**95.5**$_{+11.9}$

表格任务的成绩几乎翻倍

41.8% 提高到 80.7%。
工作簿检查、工具操作和输出规范，
都能沉淀成可执行的技能规则。

超过每项任务各自的最强基线

六项任务平均仍高 5.4 个百分点。
持续利用执行反馈，
能进一步提升技能的实际效果。

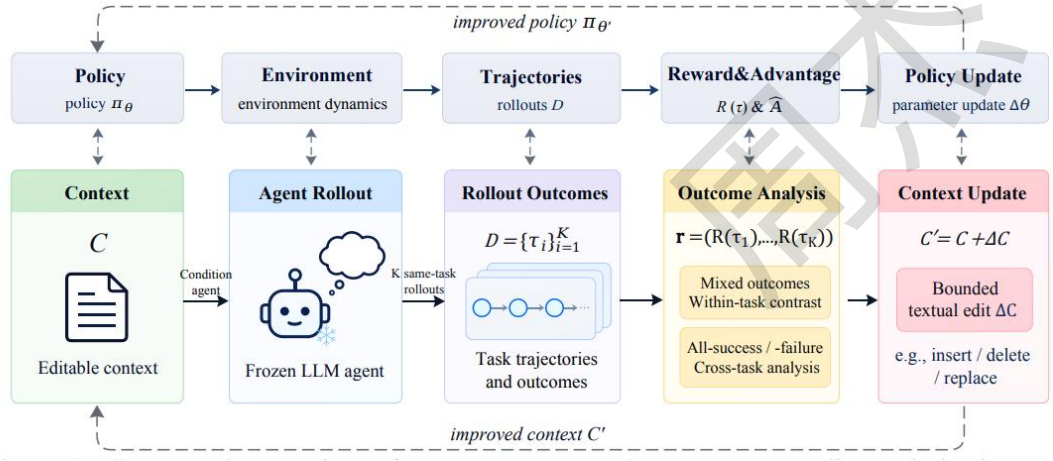
更好的技能规则可以补足执行流程中的短板，强模型同样能受益

Yifan Yang, Ziyang Gong, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills, 2026.

TextualRL：强化学习也能无梯度地更新策略

技能可以随反馈改进，关键还在于让奖励给出可靠的更新方向。
TextualRL 固定模型参数，比较多次执行并修改文本策略，实现无梯度强化学习。

目标仍是更高奖励，策略更新改用文本



知道成败，却不知道该改什么

单次执行缺少对照，
难判断哪种行为更好。

一题改对了，换题可能出错

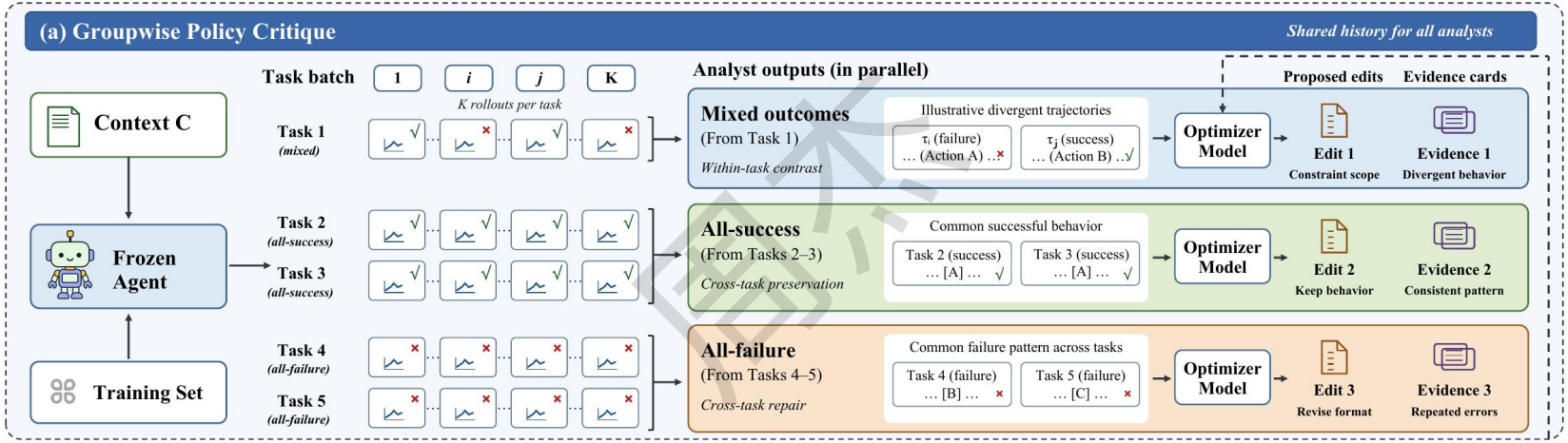
局部修复写成通用规则，
可能限制其他任务的正确做法。

目标仍是提高任务奖励，优化对象可以是指导行动的文本策略

Tianyu Huai, Jie Zhou, et al. TextualRL: Textual Reinforcement Learning for Context Management, 2026.

TextualRL：同题执行对照，才有可靠修改方向

只知道一次执行的成败，往往难以判断策略中的哪条规则应该改变。
同题、同策略执行 4 次，再比较行为与结果，为修改建议提供依据。



有成有败：找行为分歧

固定题目与上下文，
对照成功与失败的行为差异。

全部成功：保留有效做法

缺少同题成败对照，
就跨题寻找反复有效的行为。

全部失败：提出修复假设

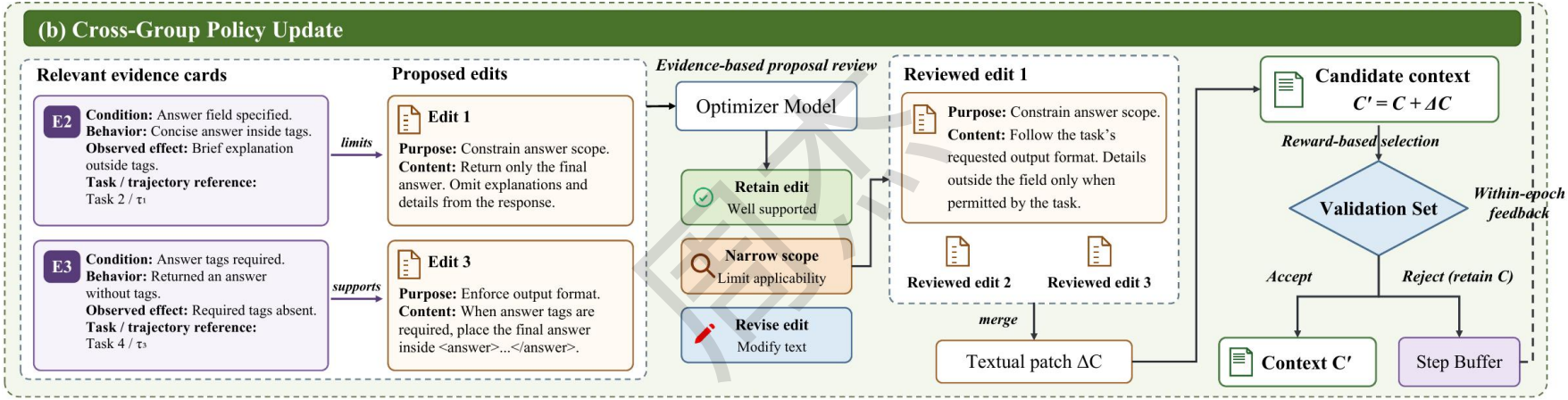
跨题寻找共同的失败原因，
修复是否有效留待验证。

通过组内对照与跨题证据，把执行结果变成有根据的策略修改建议

Tianyu Huai, Jie Zhou, et al. TextualRL: Textual Reinforcement Learning for Context Management, 2026.

TextualRL：跨任务审查，防止局部经验被滥用

修好一道题的建议，可能在其他任务中限制原本正确的行为。
用跨任务证据审查建议的适用范围，再以验证奖励决定是否接受新策略。



无需修改的成功轨迹，也能限制“一律只给答案”这类过宽的建议。

审查后：遵循任务要求，只有允许时才在答案字段外补充解释。

证据限定规则的适用范围，验证奖励决定这次更新是否值得保留

Tianyu Huai, Jie Zhou, et al. TextualRL: Textual Reinforcement Learning for Context Management, 2026.

TextualRL：固定参数，也能持续提高任务奖励

冻结目标模型，只优化文本策略，4B 小模型与 GPT-5.5 智能体都能受益。
原文分别检验小模型的任务完成率，以及成熟执行框架中的进一步提升。

小模型：完成率从三成到八成

Qwen3.5-4B / ALFWorld · 表1节选（%）

Skill Source	ALFWorld
Base Model	30.6
Human skill	28.4
LLM skill	55.2
Trace2Skill	64.9
TextGrad	53.7
GEPA	60.4
SkillOpt	81.3
TextualRL	**82.1** +51.5

+51.5

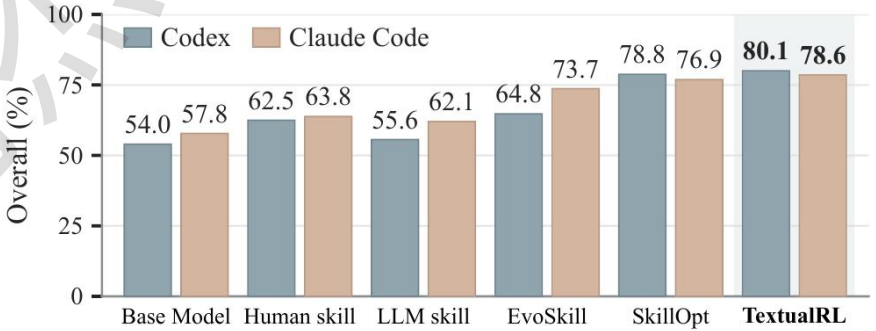
个百分点

30.6% → 82.1%

相对基础模型

执行框架：仍超过最强基线

GPT-5.5 · 五项任务平均分（原文图3）



+1.3 / +1.7 个百分点

Codex / Claude Code，相对 SkillOpt

奖励能够驱动无梯度策略更新，小模型与成熟智能体框架都能受益

周杰

Tianyu Huai, Jie Zhou, et al. TextualRL: Textual Reinforcement Learning for Context Management, 2026.

让外部指导变成模型自己的能力

经验可以保存在外部，也可以通过训练写入参数，两种方式能够配合。
当参数允许更新，关键是把执行中验证有效的做法变成可独立使用的能力。

可验证的实践

SkillGym 把技能变成可执行任务，
通过验证成功的轨迹训练流程能力。

沿自身路径学习

OPSD 与 OPCD 用教师的逐词指导，
让学生沿自己的生成路径吸收外部知识。

部署后继续学习

OEL 与 LifeSkill 持续提炼并巩固新经验，
让学习延伸到部署后的交互过程中。

检验内化的关键：移除原经验提示后，模型能否仍然做得更好

SkillGym：把外部技能变成可练习的任务

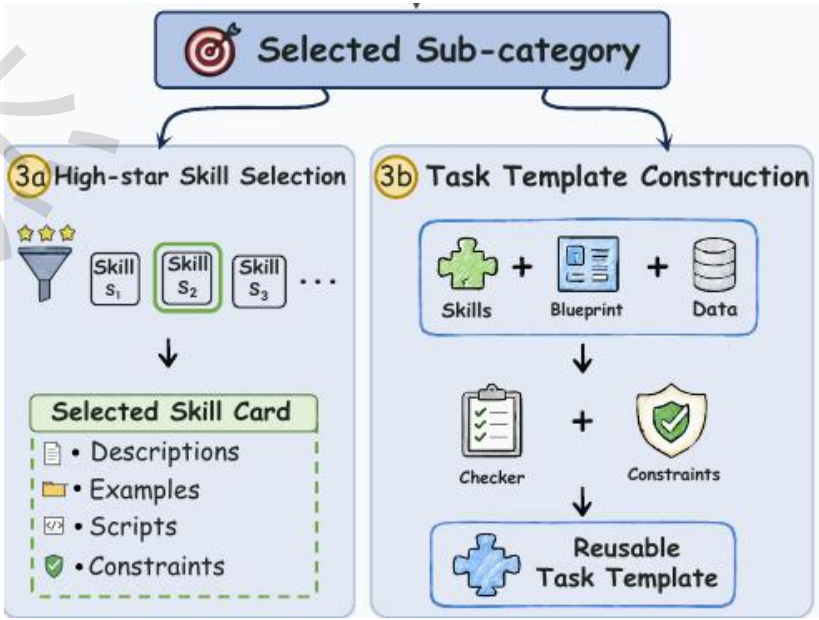
技能文档已经能指导执行，进一步要让模型不带原技能也能独立完成任务。
SkillGym 把人类技能转化为可执行、可验证的练习，再用成功轨迹训练模型。

通用做法，怎样变成具体练习？

技能描述流程；训练还需要明确的
输入、目标、运行环境和完成条件。

任务做得对，怎样可靠地判断？

不能只看回答像不像技能说明，
还要检查最终文件、测试和环境状态。

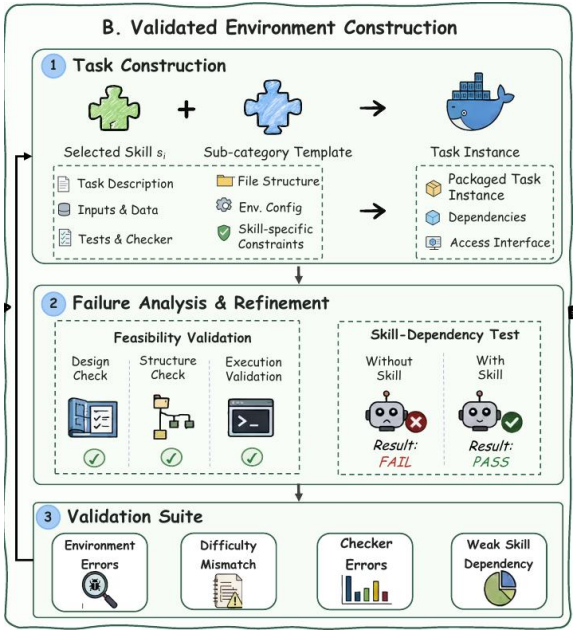


把做法变成任务，把结果变成验证信号，外部技能才有了实践载体

Zhilong Ge, Yuting Shao, et al. SkillGym: Internalizing Human Skills into LLMs for Real-World Problem Solving, 2026.

SkillGym：能执行，还要验证技能确实有帮助

由技能生成的任务，可能环境不完整，也可能根本不需要目标技能。
因此分别检查执行可行性与技能依赖，并根据失败原因修正任务环境。



把技能落到完整任务中

技能卡片结合类别模板，生成输入、依赖和检查器，
打包成可重复运行的 Docker 环境。

通过设计、结构和实际执行检查

用代码核验交付物，并排查改输入、抄答案等捷径。

同一参考智能体，分别带技能和不带技能执行

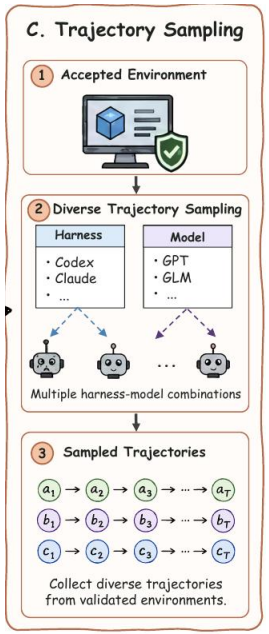
有技能成功、无技能失败，才标为“技能依赖”。
仅通过执行验证的任务仍可保留，用于补充训练。

先确保任务可执行、结果可验证，再用对照判断技能的实际帮助

Zhilong Ge, Yuting Shao, et al. SkillGym: Internalizing Human Skills into LLMs for Real-World Problem Solving, 2026.

SkillGym：完整成功轨迹让流程知识进入参数

长任务的能力体现在工具协调与错误恢复中，只看最终答案无法学到这些过程。
SkillGym 收集不同模型和框架验证通过的完整轨迹，用它们进行监督微调。



2,756

可执行任务环境
覆盖 12 大类工作流程

8,364

验证成功的执行轨迹
来自多种模型与执行框架

49 次

平均工具调用
持续协调与执行

用成功轨迹做全参数监督微调

以 Qwen3.5-35B-A3B 为基础，学习任务中的行动—观察序列，
得到 SkillGym-Agent；论文报告的能力提升来自监督微调。

经验证的行动与反馈提供训练数据，使可复用的工作流程进入模型参数

Zhilong Ge, Yuting Shao, et al. SkillGym: Internalizing Human Skills into LLMs for Real-World Problem Solving, 2026.

SkillGym：去掉技能提示，练出的能力仍然有效

训练后模型在两种执行框架、三类基准上都优于同一基座。
在 SkillsBench 上，不带外部技能的训练后模型，仍超过带技能的基础模型。

原文表4 · GDPval 为 Elo，其余为任务成功率（%）

Harness	Model	GDPval-AA v2	Terminal-Bench 2.1	SkillsBench v1.1	SkillsBench v1.1 (w/o)
Codex	Qwen3.5-35B-A3B (Base)	942	10.11	5.33	0.69
	SkillGym-Agent (Ours)	**979** (+37)	**46.07** (+35.96)	**33.02** (+27.69)	**21.08** (+20.39)
Claude Code	Qwen3.5-35B-A3B (Base)	974	39.33	23.34	12.13
	SkillGym-Agent (Ours)	**1173** (+199)	**58.43** (+19.10)	**51.47** (+28.13)	**26.81** (+14.68)

Claude Code：26.81% > 23.34%

训练后不带技能　　基础模型带技能

Codex：21.08% > 5.33%

训练后不带技能　　基础模型带技能

模型学到了可独立使用的流程能力，外部技能仍能继续补充帮助

Zhilong Ge, Yuting Shao, et al. SkillGym: Internalizing Human Skills into LLMs for Real-World Problem Solving, 2026.

SkillGym：教师经验有互补，组合仍要看任务

混合教师经验提升了终端执行与技能任务表现，但并非所有评测都越混越好。
专业交付物评测中，单一教师仍可能得分更高，训练数据需要匹配目标能力。

Harness	Teacher Setting	GDPval-AA v2	Terminal-Bench 2.1	SkillsBench v1.1	SkillsBench v1.1 (w/o)
Codex	Base	942	10.11	5.33	0.69
	GPT-5.4	969 (+27)	24.72 (+14.61)	14.00 (+8.67)	6.96 (+6.27)
	Nex-N2-Pro	**1074** (+132)	33.71 (+23.60)	13.59 (+8.26)	12.61 (+11.92)
	GPT+Nex	976 (+34)	40.45 (+30.34)	19.91 (+14.58)	13.59 (+12.90)
	All Teachers	979 (+37)	**46.07** (+35.96)	**33.02** (+27.69)	**21.08** (+20.39)
Claude Code	Base	974	39.33	23.34	12.13
	DeepSeek V4 Pro	1106 (+132)	43.82 (+4.49)	28.81 (+5.47)	19.02 (+6.89)
	GLM-5.2	**1212** (+238)	55.06 (+15.73)	45.50 (+22.16)	25.10 (+12.97)
	Deepseek+GLM	1161 (+187)	57.30 (+17.97)	47.33 (+23.99)	**28.41** (+16.28)
	All Teachers	1173 (+199)	**58.43** (+19.10)	**51.47** (+28.13)	26.81 (+14.68)

教师经验具有互补性，最佳组合仍取决于要训练的能力

Zhilong Ge, Yuting Shao, et al. SkillGym: Internalizing Human Skills into LLMs for Real-World Problem Solving, 2026.

OPSD：看过解法的模型，指导独立作答的自己

SkillGym 从成功轨迹中学习，OPSD 则关注如何指导模型自己生成的推理路径。
在数学任务中，看过参考解法的教师为学生的每一步提供更细的学习信号。

	SFT/Off-Policy Distillation	GRPO	On-Policy Distillation	On-Policy Self-Distillation (Ours)
On-Policy Data	✗	✓	✓	✓
Dense Learning Signal	✓	✗	✓	✓
Low Sampling Cost	✓	✗	✓	✓
No External Teacher	✓	✓	✗	✓

周杰

学生自己的推理路径，也需要指导

在线蒸馏能纠正模型自己生成的路径，
但通常依赖一个额外、更强的教师。

论文的切入点：让上下文带来能力差

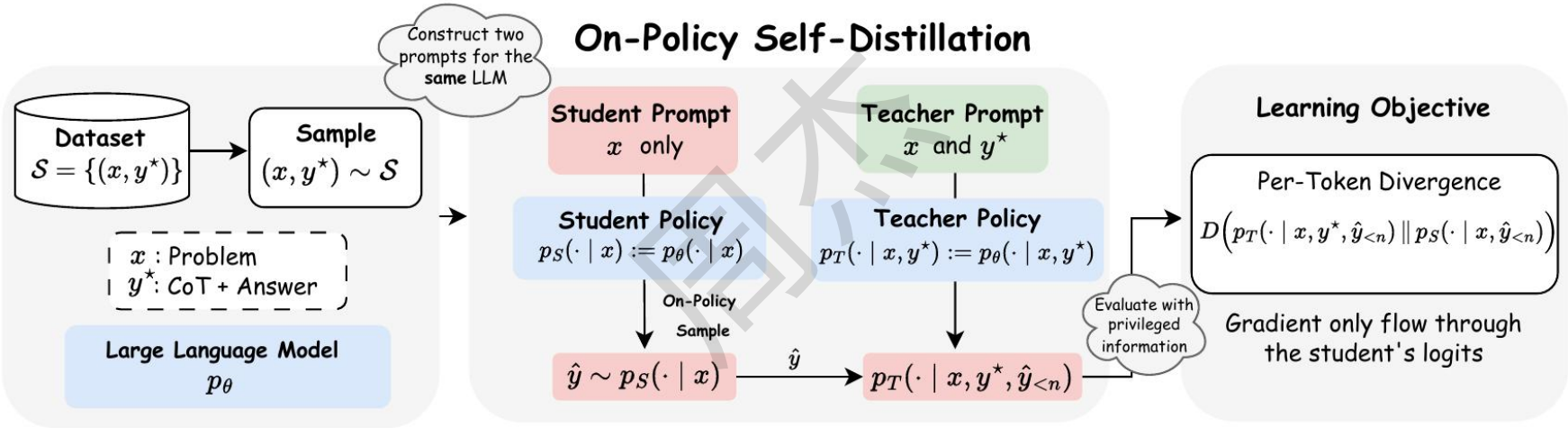
同一个模型先看到正确解法，
便可能为只看到题目的自己提供指导。

同一模型在不同上下文中的能力差，可以转化为学生的细粒度学习信号

Siyan Zhao, Zhihui Xie, et al. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models, 2026.

OPSD：沿学生自己的推理路径，逐词接受指导

为了让训练贴近实际作答，先让学生只看题目生成回答。
教师额外看到参考解法，再评估同一条回答的每一步，将指导写入学生参数。



训练信号：全词表分布的前向 KL
每一步都能获得反馈，答案错误也能学习。

实现：固定初始模型作教师，只更新学生
截断过大的损失项，减少措辞风格的干扰。

学生在自己会走到的路径上学习，参考解法为每一步提供更细的监督

Siyan Zhao, Zhihui Xie, et al. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models, 2026.

OPSD：更细的监督，减少了反复采样的需要

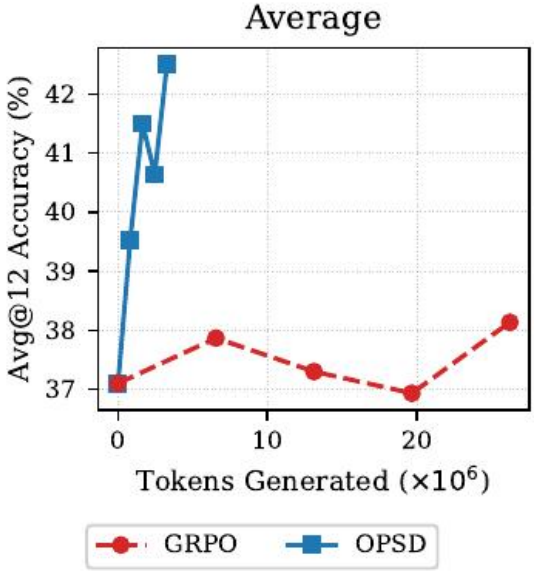
相同数学数据上，Qwen3-1.7B 的三项基准平均成绩从 37.1 提升到 43.4。
每题只采样 1 条、最多 1024 词元，仍优于需要多条长回答的 GRPO。

效果：三项竞赛数学任务均有提升

Method	AIME24	AIME25	HMMT25	Average
Qwen3-1.7B				
Base (Instruct)	51.5	36.7	23.1	37.1
+ SFT	48.4	36.3	22.7	35.8
+ GRPO	51.1	38.3	23.7	37.7
+ OPSD	**57.2**	**43.9**	**29.2**	**43.4**

相对 GRPO，平均成绩高 5.7 个百分点。
在 4B、8B 模型上，平均成绩也优于两种基线。

效率：用更少生成词元达到更高成绩



单次尝试获得更细的监督，可以同时改善推理成绩与生成词元效率

Siyan Zhao, Zhihui Xie, et al. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models, 2026.

OPCD：经验与提示可以内化为独立作答能力

可以内化的不只有参考解法，还包括经验笔记和有用的系统提示。
OPCD 让带上下文的教师指导不带上下文的学生，减少反复携带长提示的负担。

提示变长，不代表经验就能用好

反复携带长提示增加推理开销，
直接堆叠旧对话还可能干扰新问题。

需要把可迁移的经验提炼出来，
再训练出不依赖原提示的学生模型。

原文观察：原始轨迹反而拖累数学表现

Model	Experience Type	Accuracy
Qwen3-8B	w/o Experience	75.1
Qwen3-8B	Raw Trace	70.5
Qwen3-8B	Knowledge	77.4

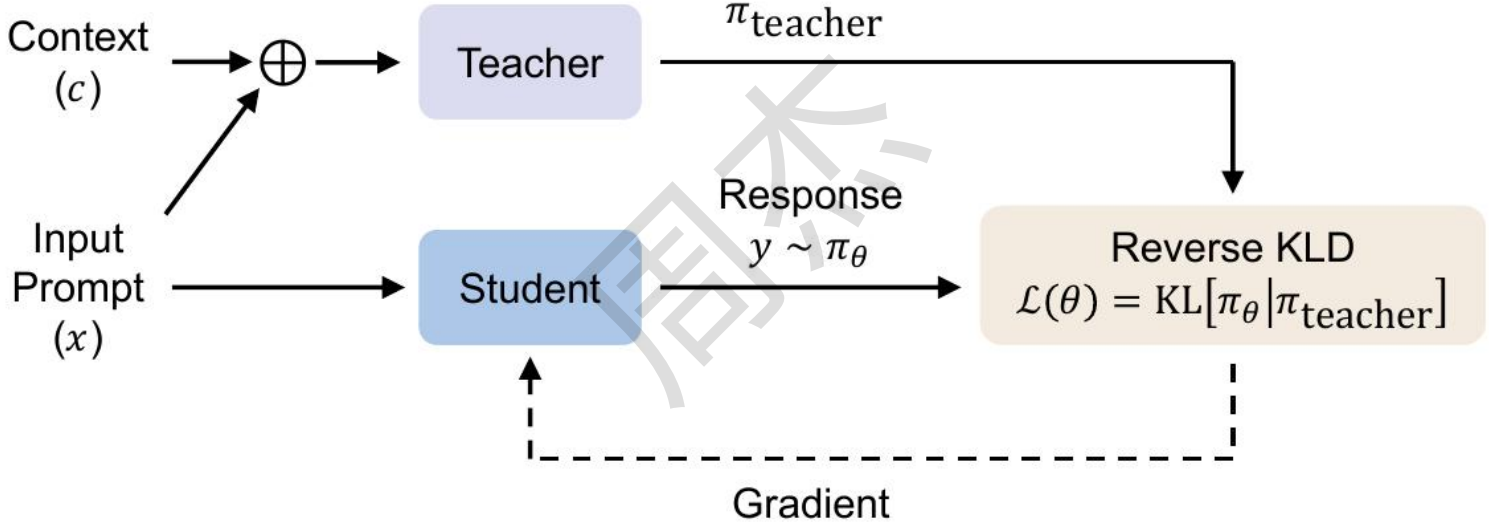
Qwen3-8B，数学验证集
原始轨迹 70.5，提炼后的经验 77.4。

把有用的上下文指导写入参数，让模型脱离原提示后仍能受益

Tianzhu Ye, Li Dong, et al. On-Policy Context Distillation for Language Models, 2026.

OPCD：不带经验先作答，再学带经验的选择

固定教师轨迹可能偏离学生实际生成的内容，因此需要沿学生自身路径训练。
学生不带经验作答，教师带经验评估同一条回答，再用逐词分布差异更新学生。



反向 KL：重点贴近教师更认可的选择
在学生高概率的词元上近似计算分布差异。

经验知识、系统提示都可以作为上下文
默认固定教师，也支持更大模型指导小模型。

让学生在自己的生成路径上学习教师指导，提高上下文知识的吸收效果

Tianzhu Ye, Li Dong, et al. On-Policy Context Distillation for Language Models, 2026.

OPCD：拿走经验和提示，性能提升仍能保留

原文分别内化模型自己提炼的经验，以及外部优化的系统提示。
蒸馏后的学生不再携带这些上下文，仍在对应任务上获得提升。

经验内化：更高准确率，较好保留通用指令能力

Model	Task	Method	Accuracy	IF-Eval (Out-of-Distribution)
Qwen3-8B	Math	Base Model	75.0	81.3
		In-Context	77.6 ± 1.1	—
		Context Distill.	78.5 ± 0.5	81.2 ± 0.2
		OPCD	**79.7** ± 0.5	**81.7** ± 0.4
Qwen3-1.7B	Frozen Lake	Base Model	6.3	67.3
		In-Context	20.2 ± 2.2	—
		Context Distill.	22.9 ± 4.0	65.1 ± 0.5
		OPCD	**26.5** ± 6.4	**67.1** ± 0.5

Frozen Lake 的准确率由 6.3 提升至 26.5。
IF-Eval 为 67.1，接近初始模型的 67.3。

系统提示内化：医疗问答

Model	Method	Accuracy
Llama-3.2-3B-Ins	Base Model	59.4
	In-Context	66.4
	Context Distill.	71.0
	OPCD	**76.3**

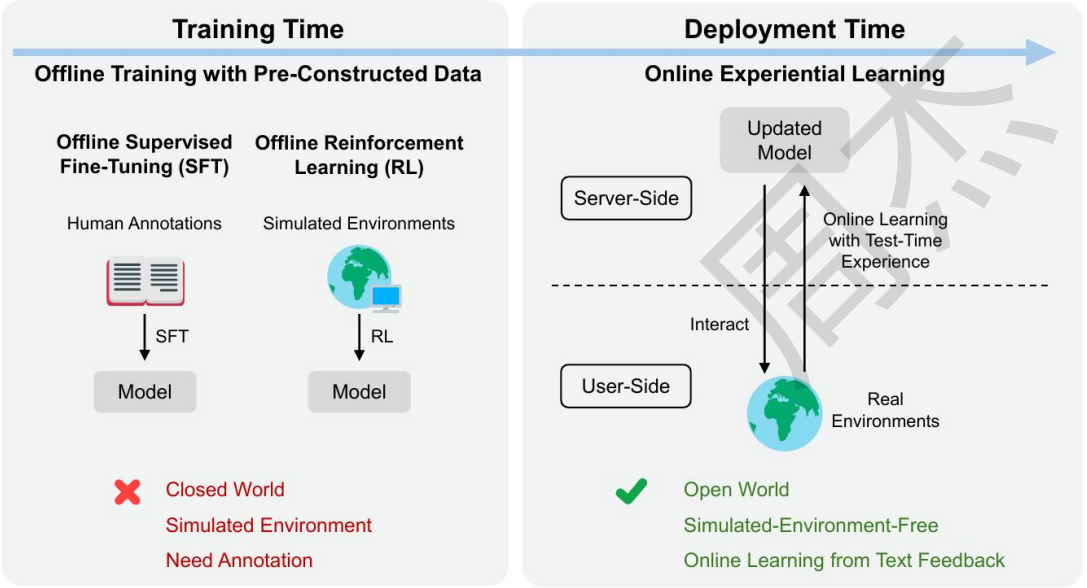
Llama-3.2-3B 的准确率
由 59.4 提升至 76.3。

外部指导可以转化为不依赖原提示的能力，同时较好保留其他任务表现

Tianzhu Ye, Li Dong, et al. On-Policy Context Distillation for Language Models, 2026.

OEL：部署后的交互，也应继续推动模型学习

有了经验内化方法，学习仍不能停在部署之前。
OEL 从用户交互的文字反馈中提炼经验，在训练端无法访问原环境时继续更新模型。



真正的障碍：经验在用户端，训练却发生在服务器端

用户环境难以复制，
反馈往往只是错误信息、
动作结果与状态变化。

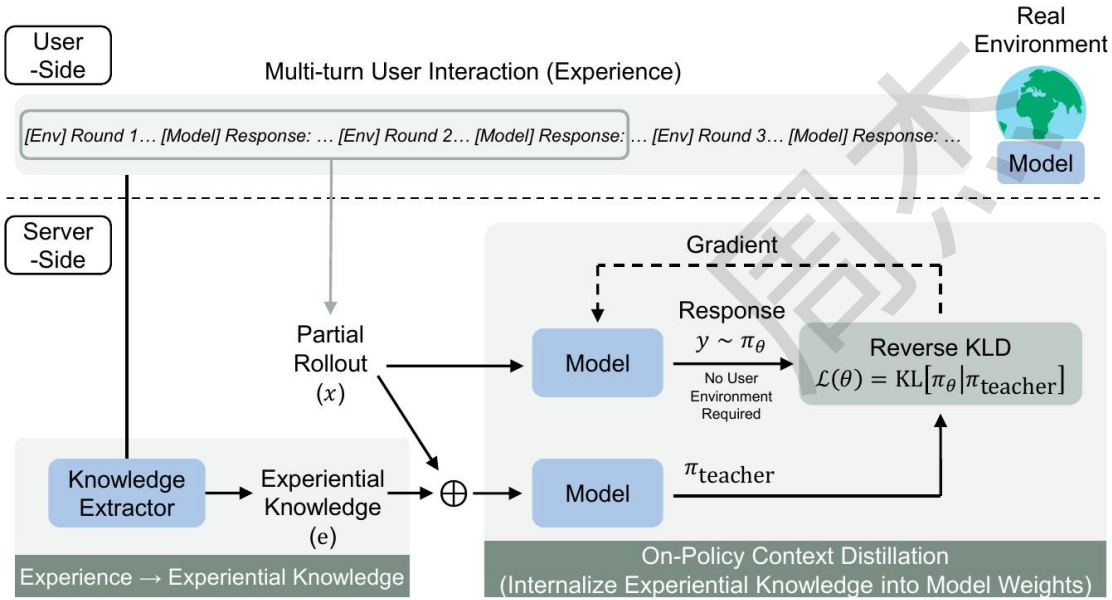
用户端收集交互轨迹，
服务器端从轨迹提取经验，
更新后的模型再进入下一轮。

用户侧收集交互，训练侧提炼与巩固经验，部署后的经历也能成为学习来源

Tianzhu Ye, Li Dong, et al. Online Experiential Learning for Language Models, 2026.

OEL：复用交互前缀，无需重建用户环境

训练端不能重新运行用户环境，就保留某次回复之前的交互历史，让当前模型重新作答。
教师额外看到提炼后的经验，用逐词指导完成巩固，不必续跑完整多轮交互。



先提炼：把交互变成可迁移经验

结合先前经验阅读新轨迹，
逐步积累有用的策略与教训。

再巩固：重新生成一轮回答

学生只看交互前缀，
冻结教师额外看到经验，
通过反向 KL 更新学生参数。

更新后重新部署，
用新策略的经历启动下一轮。

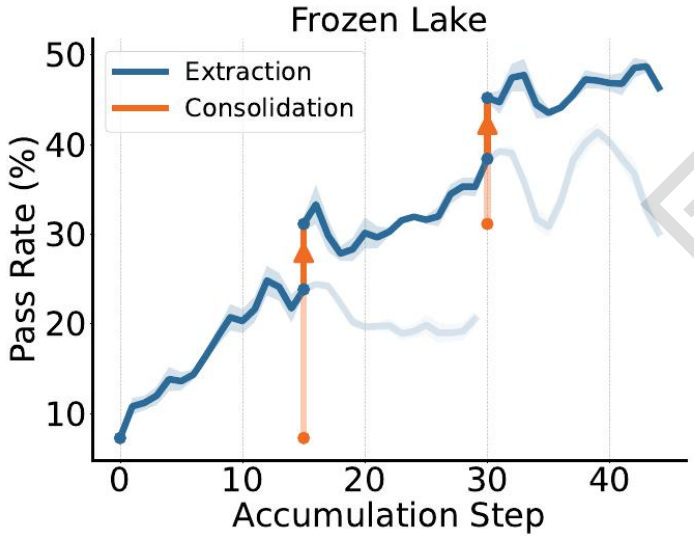
历史前缀提供情境，当前策略生成回答，经验教师为参数更新提供指导

Tianzhu Ye, Li Dong, et al. Online Experiential Learning for Language Models, 2026.

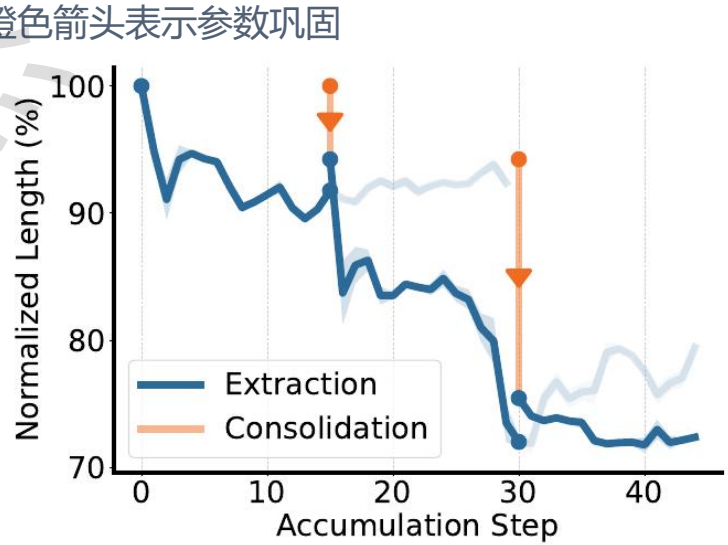
OEL：反复提炼与巩固，解题更准也更简洁

在 Frozen Lake 上，Qwen3-1.7B 交替进行经验提取与参数巩固。
通过率随轮次继续提高，第三轮平均单轮回答长度约为初始的 70%。

只积累上下文会趋于饱和，巩固后继续提升



知识内化后，平均回答长度缩短约 30%



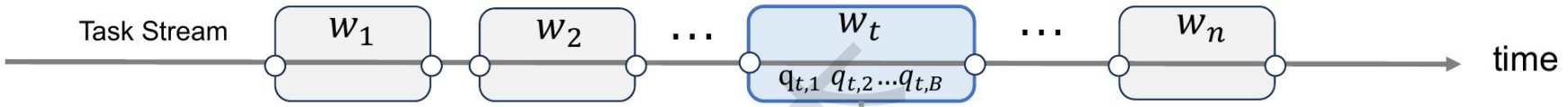
把经验写入参数后继续收集新经历，任务通过率与词元效率可以一起改善

周杰

Tianzhu Ye, Li Dong, et al. Online Experiential Learning for Language Models, 2026.

LifeSkill：提取技能与执行任务可以共同学习

持续内化依赖经验质量，看起来合理的技能，未必能真正帮助执行。
LifeSkill 用执行结果训练技能提取器，再将有效技能带来的成功做法学进策略模型。



完成当前批次的交互与更新，再进入下一批任务

先要知道：什么技能真有用

技能没有标准答案，语言上合理，
不等于实际执行有效。

再要做到：没有提示也能用

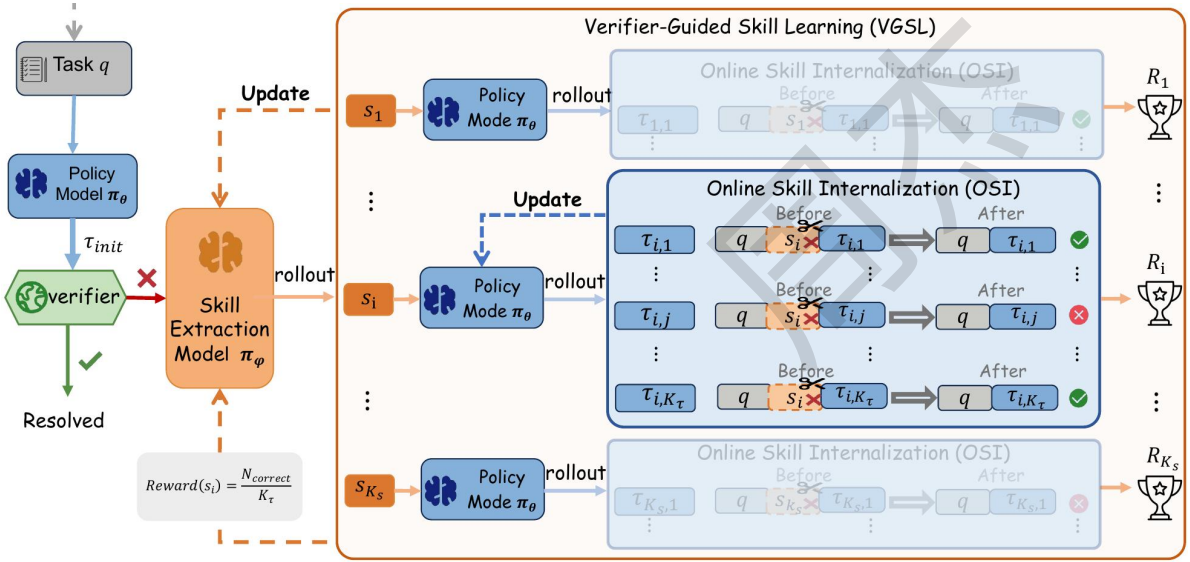
成功经验若停留在外部文本里，
模型参数就没有吸收新的做法。

技能提取得更有效，策略内化得更充分，两种能力可以在交互中共同改进

Bo Mao, Jie Zhou, et al. Learning While Acting: A Skill-Enhanced Test-Time Co-Evolution Framework for Online Lifelong Learning Agents, 2026.

LifeSkill：技能好不好，要看真实执行结果

自由文本技能没有现成的标准答案，语言上的合理性不能替代任务成功。
让同一条技能指导多次执行，用验证通过的比例评价它，再训练技能提取器。



失败后，提出多条候选技能

技能提取器读取任务和失败轨迹，
给出不同的改进做法。

每条技能都经过多次试用

让策略模型分别执行，
以验证通过的比例作为技能奖励。

效果更好的技能，获得更高奖励

比较候选技能的相对效果，
用 DAPO 更新技能提取器。

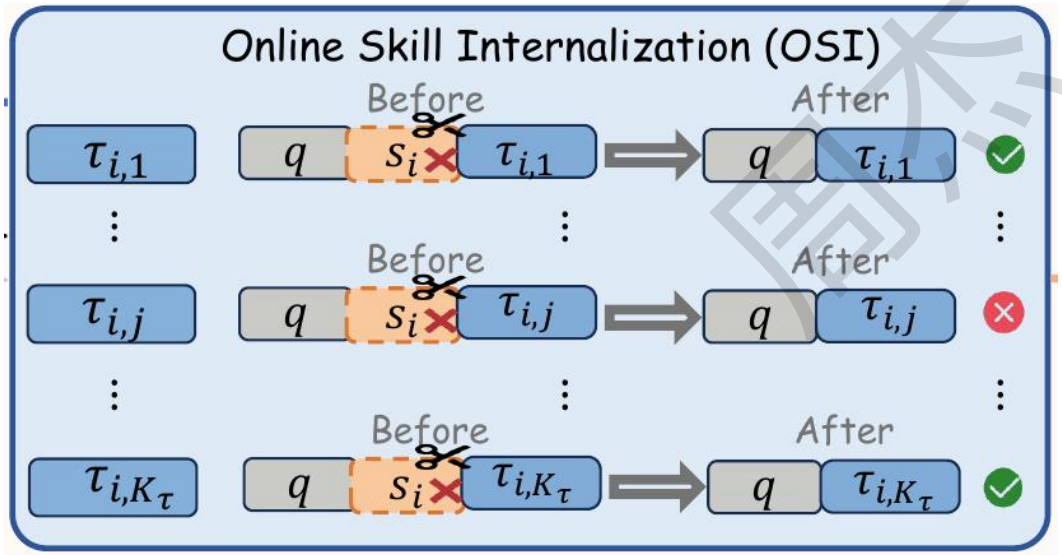
技能提取器学习的是能提高执行成功率的指导，而不是仅仅写出合理解释

Bo Mao, Jie Zhou, et al. Learning While Acting: A Skill-Enhanced Test-Time Co-Evolution Framework for Online Lifelong Learning Agents, 2026.

LifeSkill：去掉训练输入中的技能，内化成功做法

技能提示帮助模型做对任务之后，还需要让策略学会独立完成。
在线内化保留原任务与成功轨迹，移除输入中的技能文本，再据此更新策略参数。

在线技能内化 OSI（原文图2局部）



学习输入要与独立执行时一致

只删除输入中的技能文本。
原任务和完整的成功轨迹保留，
使模型学习直接响应原任务。

成功轨迹直接训练策略模型

在原任务条件下重新计算似然，
用验证通过的轨迹更新参数。

在不带技能的任务条件下学习成功轨迹，让后续任务受益于更新后的策略

Bo Mao, Jie Zhou, et al. Learning While Acting: A Skill-Enhanced Test-Time Co-Evolution Framework for Online Lifelong Learning Agents, 2026.

LifeSkill：验证与内化共同改善在线学习表现

Llama-3.1-8B-Instruct 在 DB、OS、KG 任务上的平均准确率达到 59%。
验证与内化均带来收益，但 KG 仍未超过最强基线，跨任务提升仍不均衡。

平均准确率 59%，超过两类强基线

原文表1节选，准确率

Method	Experience	Accuracy ↑			
		DB	OS	KG	Avg.
Group Self-Consistency (Zheng et al., 2025)	4	0.63	0.49	**0.36**	0.49
RFT (Yuan et al., 2023)	0	0.70	0.51	0.35	0.52
LifeSkill (Ours)	0	**0.82**	**0.64**	0.32	**0.59**

验证与内化，各自带来收益

原文表3，准确率

Configuration	DB	OS
w/o VGSL (LLM-Judge)	0.70	0.62
w/o Skill Extraction	0.64	0.53
w/o OSI	0.79	0.60
Full Model	**0.82**	**0.64**

+7 个百分点

相对最强训练基线 RFT
52% 提升至 59%

+10 个百分点

相对 Group Self-Consistency
49% 提升至 59%

改用 LLM 打分后，DB 降 12 个百分点。
去掉在线内化，DB / OS 也都下降。

技能提取与策略执行能够共同进步，长期跨场景成长仍需要进一步检验

Bo Mao, Jie Zhou, et al. Learning While Acting: A Skill-Enhanced Test-Time Co-Evolution Framework for Online Lifelong Learning Agents, 2026.

经验驱动学习：让经历真正改变后续行动

探索提供原始经历，记忆与技能帮助复用，内化再把有效做法写进参数。
这些环节共同决定：经历过一次之后，下一次能否做得更好。

经验要留下来

保存有用信息，持续修订技能，
减少过时内容与冲突规则对后续执行的干扰。

需要时用得好

把检索、执行与反馈连接起来，
用任务结果判断经验何时有用、为何无效。

有效做法能内化

当参数可以更新，把验证有效的做法学进去，
同时检查旧能力是否受损。

经验可以改进局部能力，下一步把视角扩展到整套学习系统



華東師範大學
EAST CHINA NORMAL UNIVERSITY

循环自我进化

经验在更新，学习方法也能更新

已有基础　记忆、技能与参数吸收经验，一些学习环节已经能自动优化。

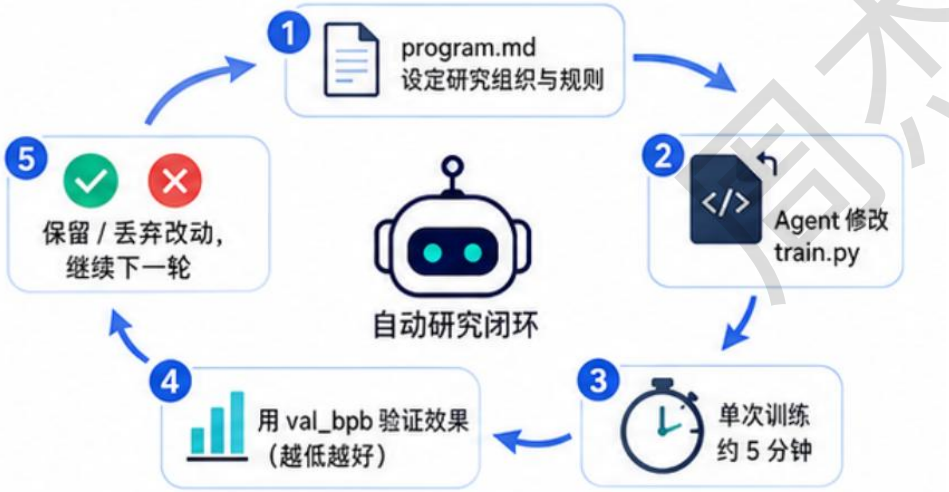
继续追问　怎样让改进机制持续有效，并把它本身纳入更新？

关注点扩展到学习系统，以及产生改进的方法。

循环自我进化：优化对象扩展到学习系统

经验可以更新记忆、技能与参数，也能暴露学习流程中的不足。
自动优化由此延伸到执行框架、改进方法，以及支撑学习的任务与环境。

AutoResearch：一个可执行的研究循环



执行框架（Harness）

更新记忆、工具与上下文流程。

改进机制

更新诊断、搜索与优化器本身。

学习条件

调整任务、环境与训练数据。

循环评测筛选有效改动，改进者自身的更新进一步触及递归自我改进

Andrej Karpathy. AutoResearch, 2026. Anthropic. When AI Builds Itself, 2026.

AutoHarness：规则检查也能由模型自己编写

技能与记忆能提供建议，但一个违规动作仍可能让整项任务失败。
AutoHarness 让模型编写检查程序，再根据环境反馈修正代码。

78%

Gemini-2.5-Flash 的棋赛败局被归因于非法落子

论文引用的 GameArena 棋赛结果：
许多失败发生在策略较量之前。

问题：知道规则，不等于每次都守规则

提示词需要在每一步重新发挥作用，
一个违规动作就可能让整局失败。

办法：让模型给自己编写检查程序

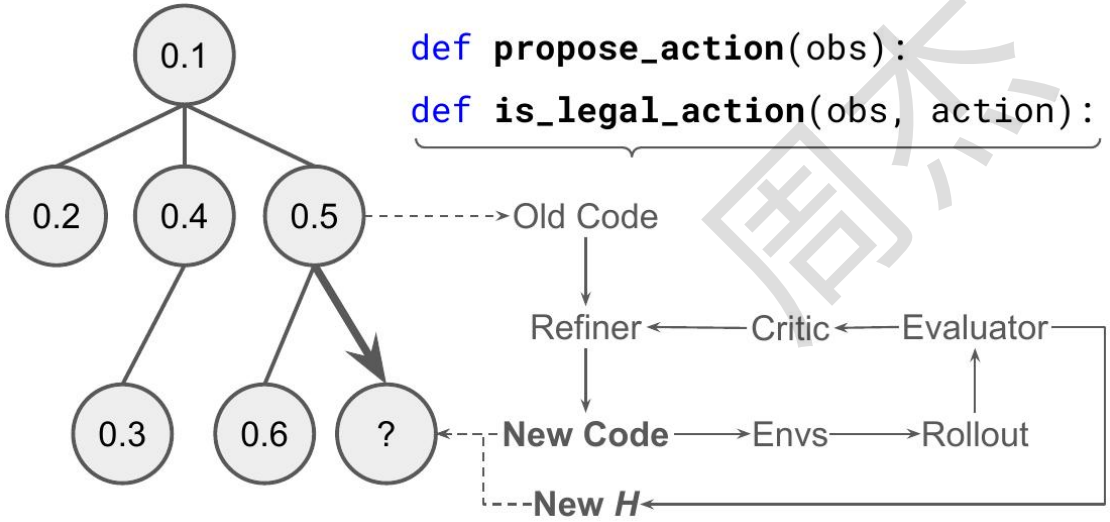
先合成代码，再让环境指出错误，反复修正。
每个环境得到一个专用框架，
模型权重保持不变。

将规则检查固化为程序，减少重复违规，为策略选择留下空间

Xinghua Lou, Miguel Lázaro-Gredilla, et al. AutoHarness: improving LLM agents by automatically synthesizing a code harness, 2026.

AutoHarness：运行中的错误指导代码搜索

一次写出的检查器也会出错，需要反复执行来暴露问题。
系统保留多个代码分支，根据合法性与异常反馈选择并修改候选。



先执行：让错误暴露出来

候选代码生成动作并检查合法性，在环境中试跑，收集违规和异常。

再选择：兼顾探索与继续修正

Thompson sampling 在多个候选间选择下一条值得继续修改的分支。

再改写：错误反馈交给模型

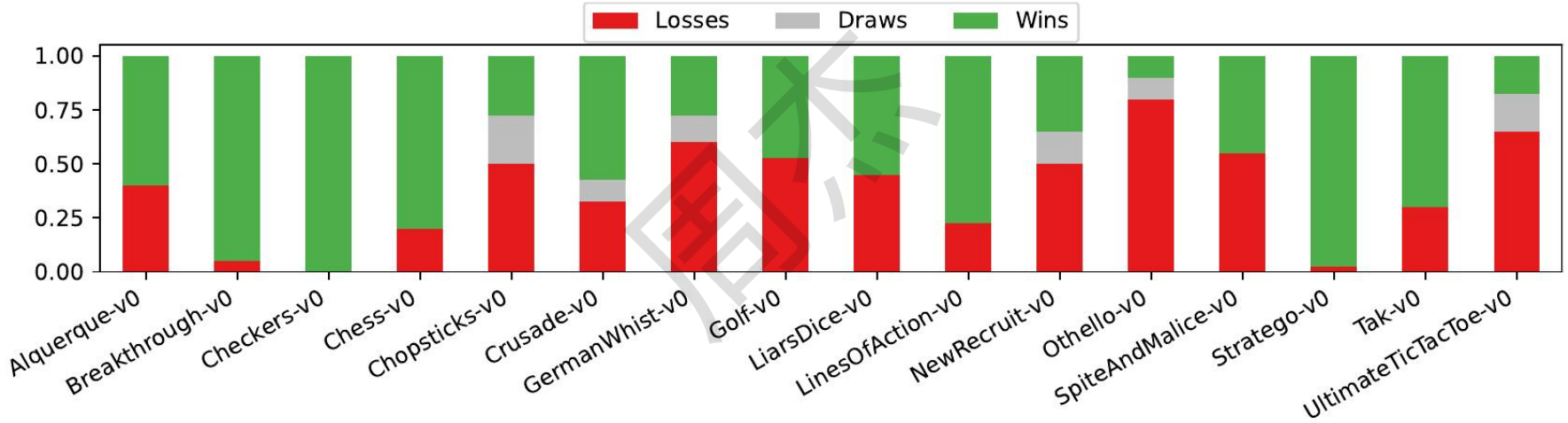
保留分支，用后续执行继续检验。

模型提出动作，程序检查合法性，违规时重试后再提交环境

Xinghua Lou, Miguel Lázaro-Gredilla, et al. AutoHarness: improving LLM agents by automatically synthesizing a code harness, 2026.

AutoHarness：可靠执行让小模型赢得更多

独立轨迹检验了 145 个游戏的动作合法性，全部达到 100%。
在另行评测的 32 个游戏中，自动检查器也改善了胜率或平均奖励。



双人对弈：56.3% 胜，38.2% 负

每游戏 40 局，先后手各占一半。

单人游戏：平均奖励也提高

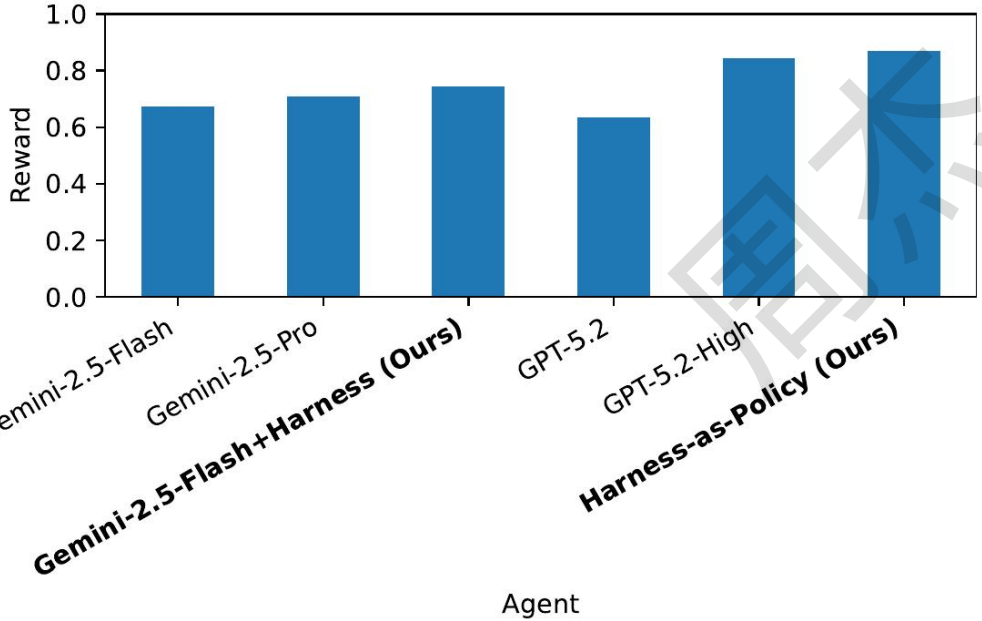
Flash 0.673 → 加框架 0.745；Pro 为 0.707。

在这组游戏中，补好规则检查让 Flash 的总体表现超过 Pro

Xinghua Lou, Miguel Lázaro-Gredilla, et al. AutoHarness: improving LLM agents by automatically synthesizing a code harness, 2026.

AutoHarness：部分游戏可由代码直接决策

当规则明确、策略可编程时，代码还可以直接决定下一步动作。
在 16 个单人游戏中，搜索同时利用合法性与最终奖励合成完整策略。



平均奖励达到 0.870

高于 GPT-5.2-High 的 0.844。
逐游戏比较为 3 胜、5 负、8 平，
优势体现在整体均分。

前期搜索，换取低运行成本

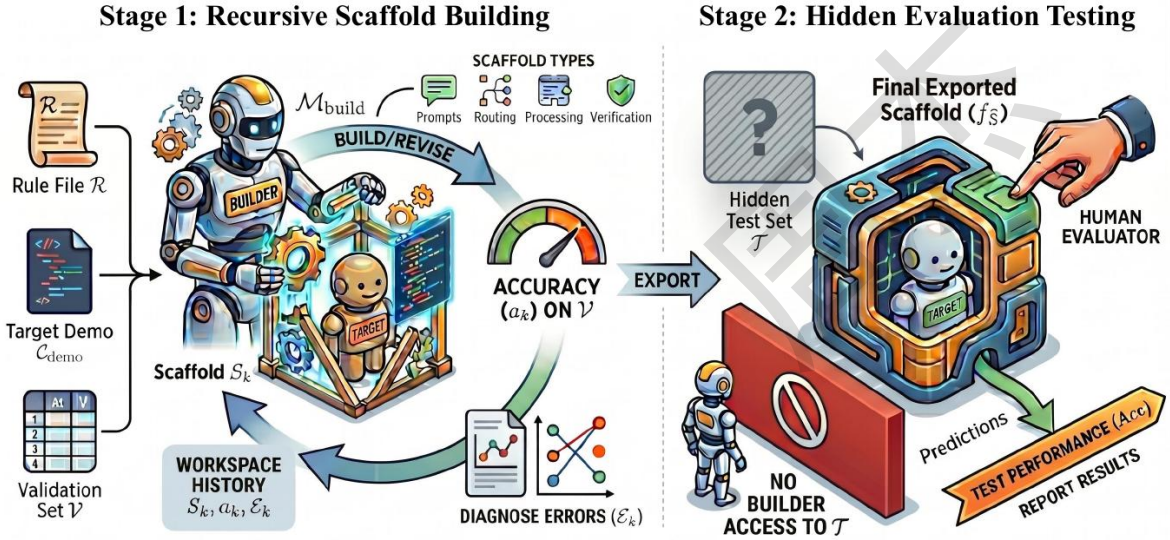
合成平均需要 89.4 轮修改；
完成后由普通 Python 代码决策，
运行时不再调用 LLM。
当前证据来自这组单人游戏。

这组单人游戏的策略可提前合成，运行时无需逐步调用模型

Xinghua Lou, Miguel Lázaro-Gredilla, et al. AutoHarness: improving LLM agents by automatically synthesizing a code harness, 2026.

AI4AI：强模型能为小模型编写推理流程

代码能约束动作，也能接管小模型不稳定的推理步骤。
AI4AI 让强模型设计外部流程，帮助参数固定的小模型完成心智推理。



强模型负责设计与改进

读取任务规则和少量验证样本，编写 Harness，执行后诊断错误，再修改路由、提示或求解程序。

小模型负责运行最终流程

导出的程序接受未见过的题目。测试时不再调用强模型改方案，小模型参数始终固定。

强模型构建的程序可以反复复用，将能力传递给小模型的执行系统

Cheng Qian, Wenting Zhao, et al. AI4AI at Test-Time: Strong-to-Weak Capability Transfer via Harnesses, 2026.

AI4AI：可形式化的推理交给程序执行

单纯要求小模型多想几步，未必能修复状态跟踪与逻辑判断错误。
构建者将可形式化的步骤写成规则，其余问题交给受约束的模型调用。

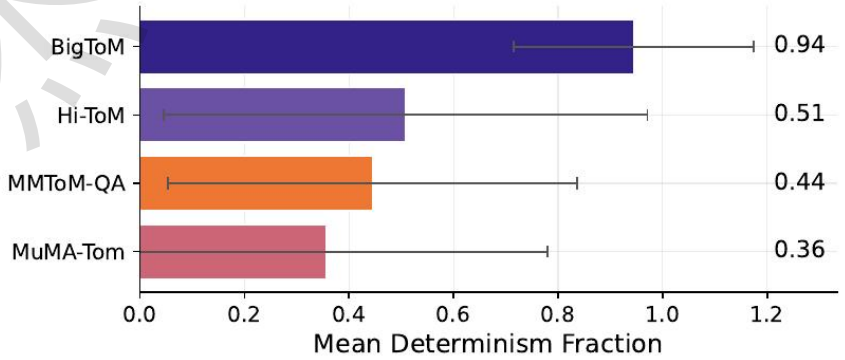
可形式化的步骤直接执行

例如识别人物是否目睹变化，
用规则更新信念状态，处理肯定与否定。
部分题目无需再调用目标模型。

难以形式化的步骤缩小推理范围

按题型选择专门流程，提取相关状态，
约束提示与答案格式，减少无关负担。

原文分析：直接由程序回答的样本比例



BigToM 约 94%，MuMA-ToM 约 36%。
能够转成明确规则的程度，随任务明显变化。

程序接管确定性步骤，模型只需处理剩余的判断

Cheng Qian, Wenting Zhao, et al. AI4AI at Test-Time: Strong-to-Weak Capability Transfer via Harnesses, 2026.

AI4AI：外部流程显著改善心智推理

在四个心智理论基准上，实验固定 GPT-5.4-mini，只改变外部流程。
所有构建配置均超过直接调用，最佳自动流程接近人工参考流程。

Metric	Value
GPT-5.4-mini vanilla baseline	0.488
GPT-5.4 vanilla baseline	0.619
GPT-5.4-mini human-inspired	0.939
Mean over all scaffolded runs	0.763 (+0.275)
Best run (GPT-5.5, GPT Codex)	0.912
– uplift over baseline	+0.423 (86.7%)
Builder beating vanilla baseline	100%

普遍受益：所有运行都超过直接调用

57 次运行的平均准确率为 76.3%，直接调用为 48.8%。

最佳运行达到 91.2%

高于直接使用 GPT-5.4 的 61.9%。
人工设计的 UserHarness 为 93.9%，自动构建仍有改进空间。

在这些结构化任务中，流程设计显著影响同一小模型的最终表现

Cheng Qian, Wenting Zhao, et al. AI4AI at Test-Time: Strong-to-Weak Capability Transfer via Harnesses, 2026.

AI4AI：额外规则也会干扰已有能力

同一构建者分别为两个目标模型设计流程，收益仍有明显差异。
原本不擅长的任务改善更多，已经擅长的任务则可能出现退步。

Benchmark	GPT-5.4-mini			Gemini-3.5-flash		
	Base	Scaf.	Δ	Base	Scaf.	Δ
BigToM	0.50	0.89	+0.39	0.49	0.92	+0.42
Hi-ToM	0.57	0.73	+0.16	0.82	0.77	-0.04
MMToM-QA	0.41	0.70	+0.28	0.75	0.84	+0.09
MuMA-Tom	0.47	0.71	+0.24	0.98	0.96	-0.02

GPT-5.4-mini：四项任务均有提升

四项基准平均提高 26.2 个百分点。

Gemini-3.5-flash：部分任务出现回退

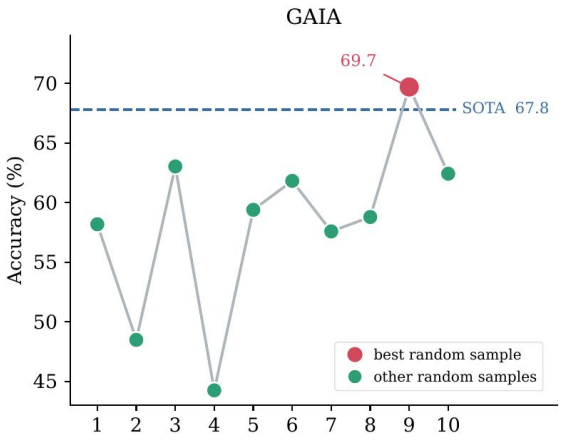
Hi-ToM 平均下降约 4 点，MuMA 下降约 2 点。

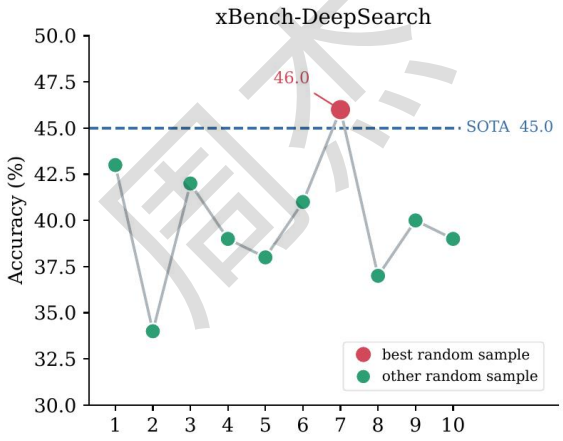
外部流程需要适配模型短板，增加规则并不总能带来收益

Cheng Qian, Wenting Zhao, et al. AI4AI at Test-Time: Strong-to-Weak Capability Transfer via Harnesses, 2026.

AutoMem：记忆架构需要随任务调整

MemPro 改进记忆读写程序，AutoMem 关注记忆模块怎样组合。
它联合搜索编码、存储、检索与维护，避免局部改好却难以配合。





四个相互影响的选择

编码：留下什么经验
存储：用什么形式组织
检索：如何找到有用信息
维护：处理过时或冲突记忆

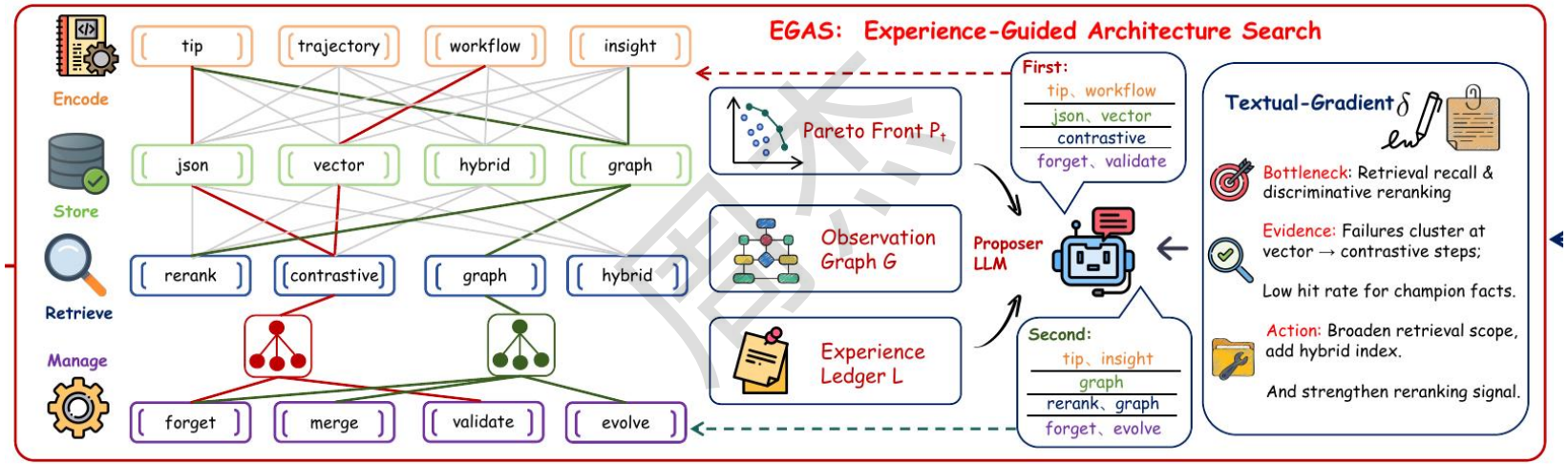
原文图 1：相同基座下，
不同架构的准确率大幅波动。

记忆效果取决于模块配合，适合一个任务的架构未必适合另一个

Lin Du, Jie Zhou, et al. AutoMem: A Text-Gradient Recursive Self-Improvement Framework for Automated Memory Architectures Search, 2026.

AutoMem：搜索经验指导下一轮架构选择

评估一个记忆架构需要完整运行智能体，盲目尝试会反复付出成本。
系统保留历代架构、评测结果与反思，据此提出兼容的模块组合。



组合必须兼容，收益还要经留出任务验证

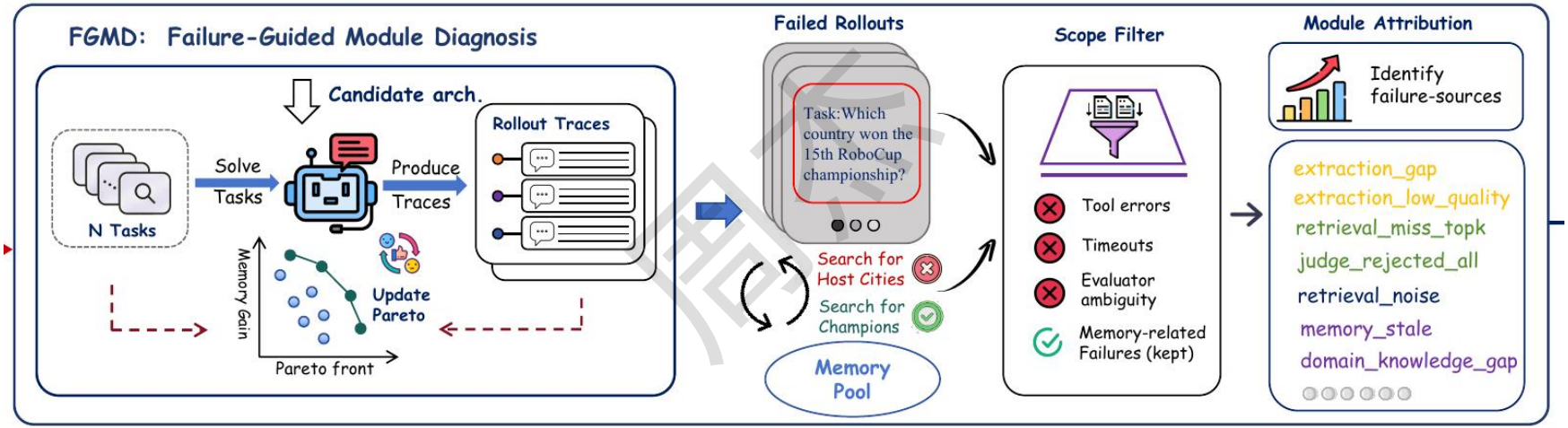
从四类模块中提出候选，按准确率、记忆增益和执行成本共同筛选。

基座参数保持不变，搜索历史持续帮助筛选记忆模块组合

Lin Du, Jie Zhou, et al. AutoMem: A Text-Gradient Recursive Self-Improvement Framework for Automated Memory Architectures Search, 2026.

AutoMem：先定位失败，再提出模块修改

任务失败还不足以说明该改哪个模块，工具错误也可能与记忆无关。
系统结合失败统计和具体轨迹形成模块诊断，再用后续试验检验建议。



排除无关失败

工具报错与超时不归因给记忆。

定位模块瓶颈

结合失败统计和代表性轨迹。

让建议接受验证

有效方向保留，无效方向降级。

文字反馈提出修改方向，执行结果决定哪些诊断值得继续采用

Lin Du, Jie Zhou, et al. AutoMem: A Text-Gradient Recursive Self-Improvement Framework for Automated Memory Architectures Search, 2026.

AutoMem：答得更准，效率收益因任务而异

在两种基座、三个基准上，AutoMem 都超过各自最强的记忆基线。
Qwen 的 GAIA 结果同时改善准确率与执行成本，其他任务仍有权衡。

原文表 2：Qwen3.5-122B-A10B，Perf. 为准确率，Cost 为每任务千 token

Memory Setting	GAIA				xBench				WebWalkerQA			
	Perf.	Cost	Delay	#Steps	Perf.	Cost	Delay	#Steps	Perf.	Cost	Delay	#Steps
Backbone: Qwen3.5-122B-A10B												
No-Memory	65.0	335.5	661	13.1	30.0	412.6	433	14.0	67.6	111.7	309	7.3
Mem0 [4]	67.1	215.7	365	9.7	36.0	434.4	621	14.3	66.5	131.1	383	7.0
MemoryBank [29]	67.8	200.4	302	9.5	45.0	430.2	535	14.9	67.1	111.8	283	6.6
ExpeL [7]	63.6	204.0	329	9.7	44.0	351.7	395	12.9	67.1	114.2	226	7.1
Voyager [8]	66.4	235.7	358	11.0	40.0	383.6	425	13.8	66.9	130.0	260	8.0
Agent-KB [12]	64.3	221.6	422	10.0	40.0	338.8	458	12.4	68.9	119.6	304	6.9
Memp [30]	66.4	199.1	316	9.8	41.0	332.4	345	12.5	66.5	104.5	215	6.7
AUTOMEM (ours)	**71.5**	128.4	270	6.0	**46.0**	395.7	729	15.6	**72.5**	118.7	182	6.4

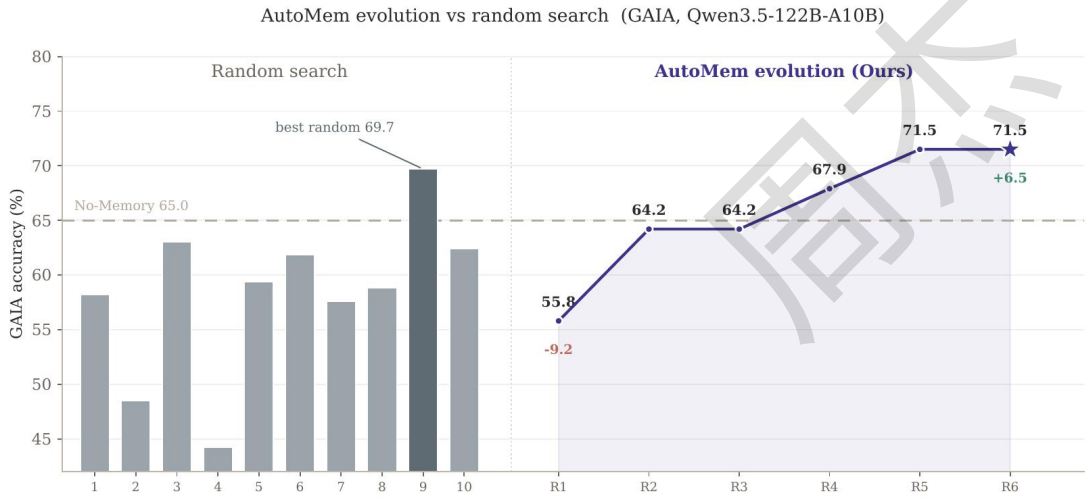
GAIA：71.5% 准确率、128.4k token；xBench 的准确率提高，但延迟与步骤增加。

任务适配在六组设置中提高了准确率，但并非所有执行开销都下降

周杰

AutoMem：有方向的搜索更快找到好架构

在 GAIA 上，第 5 轮已超过 10 个随机架构中的最好结果。
完整搜索的任务执行 token 也更少，失败反馈帮助减少了无效尝试。



搜索阶段的任务执行成本

GAIA 为随机搜索的 0.55 倍
xBench 为随机搜索的 0.47 倍

Benchmark	Search protocol	Tokens (M) ↓	Rel. cost ↓
GAIA	Random search	259.0	1.00×
	AUTOMEM (one evolution run)	**142.7**	**0.55×**
xBench-DS	Random search	339.0	1.00×
	AUTOMEM (one evolution run)	**160.1**	**0.47×**

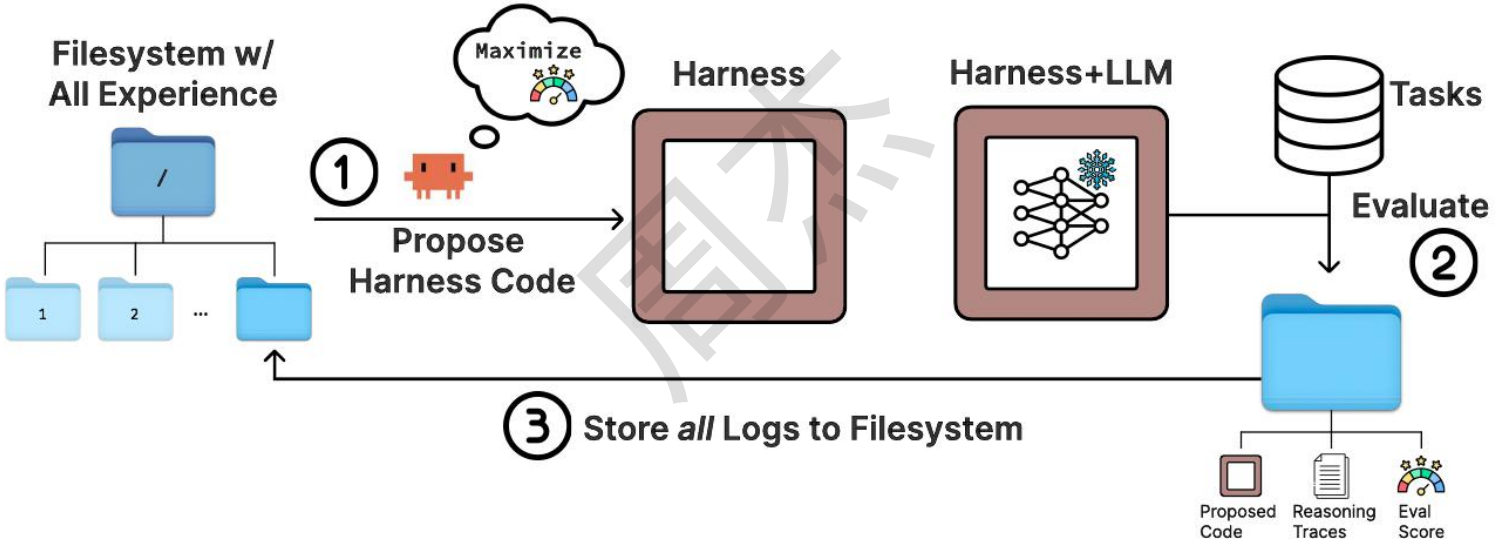
原文图 3 与表 3
成本统计不含架构提议与诊断调用。

在所评估的搜索中，复用试验反馈同时改善架构质量与执行成本

Lin Du, Jie Zhou, et al. AutoMem: A Text-Gradient Recursive Self-Improvement Framework for Automated Memory Architectures Search, 2026.

Meta-Harness：完整历史帮助诊断框架问题

记忆模块可以组合搜索，完整运行框架的错误却往往跨越多个环节。
Meta-Harness 保存历代代码、得分与完整轨迹，让优化者按需追查。



查历史：跨版本定位问题　　**改代码：检索、记忆与调度**　　**跑任务：评分并保存轨迹**

把历史变成可查询的证据，支持从失败过程追到具体程序改动

Yoonho Lee, Roshen Nair, et al. Meta-Harness: End-to-End Optimization of Model Harnesses, 2026.

Meta-Harness：摘要会丢掉关键诊断线索

在在线文本分类中，三种设置都能查看历史代码与得分。
加入原始轨迹后，优化者能追查具体状态变化，找到更好的搜索候选。

Method	Scores	Code	Summ.	Traces	Median ↑	Best Acc ↑	> ZS
Scores Only	✓	✓	×	×	34.6	41.3	26
Scores + Summary	✓	✓	✓	×	34.9	38.7	23
Meta-Harness (full)	✓	✓	-	✓	**50.0**	**56.7**	39

候选的中位准确率：34.9% → 50.0%

完整接口的中位候选，已超过两种简化接口找到的最好候选；这里比较的是搜索集表现。

历史确实被大量使用

终端任务搜索中，每轮读取 82 个文件（中位数），代码与轨迹各约四成。

完整轨迹在该搜索实验中更有效，过早压缩可能删掉有用线索

Yoonho Lee, Roshen Nair, et al. Meta-Harness: End-to-End Optimization of Model Harnesses, 2026.

Meta-Harness：上下文更少，分类反而更准

选出的分类框架在三个数据集的留出测试上超过 ACE。
它按类别组织相关例子，并加入相似但标签不同的对照，减少无效上下文。

Harness	Datasets			Avg.	
	USPTO	S2D	Law	Acc	Ctx ↓
Zero-Shot	12.0	63.2	7.0	27.4	0
Few-Shot (8)	14.0	67.9	21.0	34.3	2.0
Few-Shot (32)	13.0	72.2	21.0	35.4	7.9
Few-Shot (all)	15.0	78.3	29.0	40.8	12.3
MCE [52]†	14.0	83.0	23.0	40.0	28.5
ACE [59]†	**16.0**	77.8	29.0	40.9	50.8
Meta-Harness	14.0	**86.8**	**45.0**	**48.6**	11.4

搜索发现了怎样的策略?

先列出合法类别，避免忽略选项；
每类检索一个与问题相关的例子，
再加入相似、但标签不同的对照例子。

换到 9 个未参与搜索的数据集

平均准确率：ACE 70.2%
→ Meta-Harness 73.1%。
说明这套组织经验的方式可以复用。

挑选和组织经验比堆叠内容更有效，这套分类策略还能迁移到新数据集

Yoonho Lee, Roshen Nair, et al. Meta-Harness: End-to-End Optimization of Model Harnesses, 2026.

Meta-Harness：检索策略能跨模型复用

同样使用 BM25，搜索程序会按题型调整去重、重排与示例数量。
选定框架用于 200 道未见数学题，在五个模型上都超过无检索设置。

Method	GPT-5.4n	GPT-5.4m	Gem-3.1FL	Gem-3F	GPT-20B	Avg.
No Retriever	23.0	28.8	28.6	42.6	47.6	34.1
Dense Retrieval (k=1)	27.1 (+4.1)	24.5 (-4.3)	31.3 (+2.7)	42.3 (-0.3)	46.9 (-0.7)	34.4 (+0.3)
Dense Retrieval (k=5)	31.1 (+8.1)	28.3 (-0.5)	37.1 (+8.5)	47.2 (+4.6)	46.7 (-0.9)	38.1 (+4.0)
Random Few-shot	23.1 (+0.1)	24.5 (-4.3)	31.0 (+2.4)	40.4 (-2.2)	41.8 (-5.8)	32.2 (-1.9)
BM25 Retrieval	30.2 (+7.2)	29.2 (+0.4)	32.8 (+4.2)	46.6 (+4.0)	48.9 (+1.3)	37.5 (+3.4)
Meta-Harness	31.7 (+8.7)	30.4 (+1.6)	34.9 (+6.3)	46.3 (+3.7)	50.6 (+3.0)	**38.8** (+4.7)

相对无检索：五个模型均提升

平均 34.1% → 38.8%，提升 4.7 个百分点。

普通检索也可能适得其反

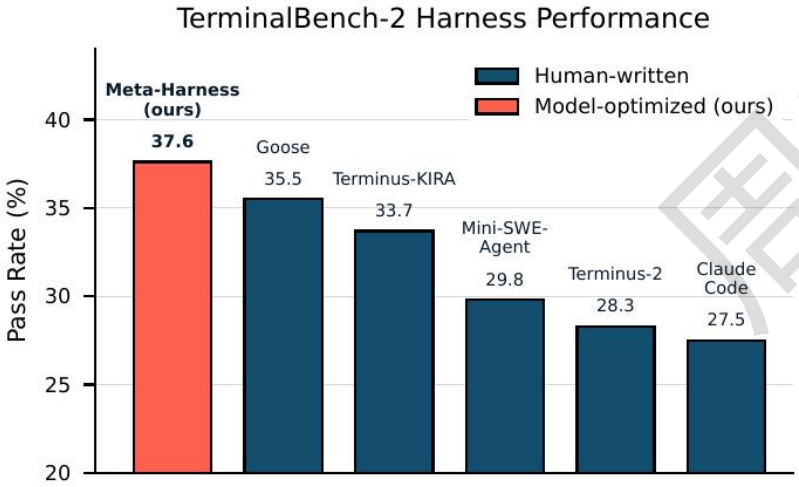
密集检索和随机示例在部分模型上出现退步。

更合适的检索与组织程序可供多个模型共享，普通检索未必都有益

Yoonho Lee, Roshen Nair, et al. Meta-Harness: End-to-End Optimization of Model Harnesses, 2026.

Meta-Harness：历史失败能纠正修改方向

终端任务的搜索日志显示，优化者会对照失败版本，撤回有害修改。
该实验在同一组 89 个任务上搜索和评测，展示了持续诊断的过程。



先尝试：修复结构，也重写提示词

多个版本退步后，发现共同问题：
新增清理指令让智能体删掉了必要状态。

再隔离：撤回提示词，单独测试修复

退步收窄，但直接调整完成流程仍不稳。

最后转向：先补充环境信息

任务开始前提供环境快照，产生更好候选。

历史失败帮助纠正改法，该终端框架的迁移效果仍需另行验证

Yoonho Lee, Roshen Nair, et al. Meta-Harness: End-to-End Optimization of Model Harnesses, 2026.

AHE：改好框架需要看清证据与改动

保留历史以后，还需要把冗长记录变成能定位改动的证据。
AHE 暴露可编辑组件、分层整理轨迹，并要求每次修改写出预期影响。

不知道该改哪里

提示词、工具、记忆相互影响，
把所有问题都塞进提示词，
很难定位具体责任。

看不完执行历史

大量日志记录了发生的事情，
真正值得修复的根因
却埋在细节里。

说不清改动有没有用

总分上涨不代表每题都变好，
一次“修复”也可能
破坏原来能完成的任务。

把组件变成独立文件

暴露 7 类可编辑组件，
支持逐文件检查与回退。

把轨迹整理成证据

先看任务诊断与整体总结，
需要时再深入原始轨迹。

把修改写成可检验的预测

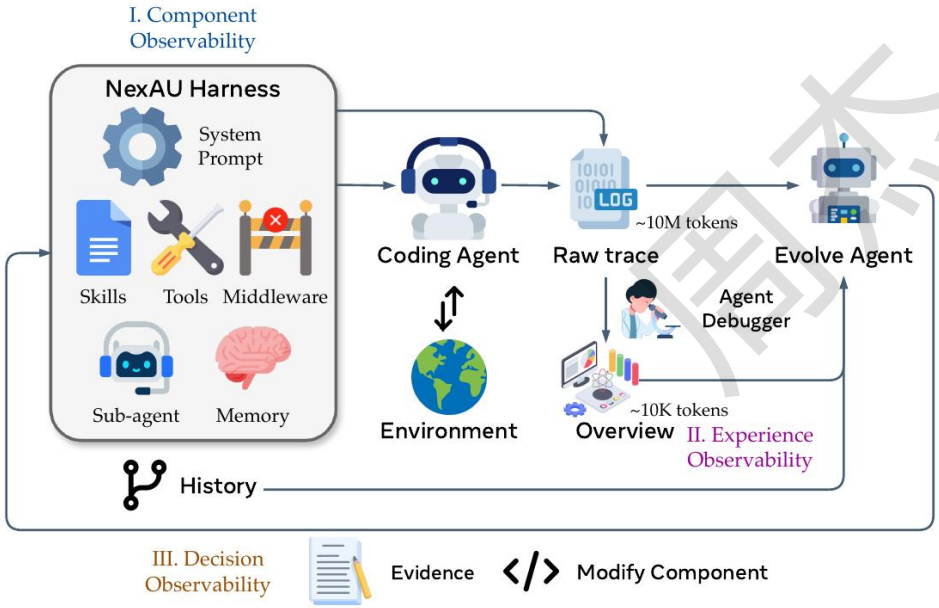
记录预期修复与退步的任务，
下一轮逐项对照。

明确哪里能改、依据是什么、预期会怎样，才能逐轮检查修改效果

Jiahang Lin, Shichun Liu, et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses, 2026.

AHE：修改预测要由下一轮执行核对

听起来合理的反思未必能够改善执行，需要先提出可检验的预测。
系统记录改动依据与预计修复或退步的任务，再对照下一轮实际结果。



执行 → 诊断 → 修改 → 验证

Debugger 将轨迹整理为
逐任务根因报告与整体总结；
Evolve Agent 据此修改文件。

新一轮先核对上一轮预测

把逐任务结果与改动清单对照，
保留有效修改，回退无效修改。
原始轨迹保留，支持继续追查。

把建议变成可核对的修改记录，才能保留有效改动并回退无效改动

Jiahang Lin, Shichun Liu, et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses, 2026.

AHE：迁移后的主要收益是减少开销

框架在同一组终端任务上迭代后，冻结并迁移到 500 个软件修复任务。
新任务上的总体成功率略升，平均 token 降低，减少了重复执行负担。

		Success rate ↑				Tokens k ↓			
Repo	N	ACE	TF-GRPO	NexAU_0	**AHE**	ACE	TF-GRPO	NexAU_0	**AHE**
All	500	74.6%	74.2%	75.2%	**75.6%**	679	582	526	**461**
django	231	79.2%	78.8%	79.2%	**81.0%**	707	583	527	**484**
sympy	75	69.3%	68.0%	**70.7%**	**70.7%**	602	572	494	**479**
sphinx-doc	44	61.4%	65.9%	68.2%	**70.5%**	990	848	731	**656**
matplotlib	34	70.6%	70.6%	**73.5%**	**73.5%**	622	530	486	**391**
scikit-learn	32	**93.8%**	**93.8%**	**93.8%**	87.5%	451	378	307	**257**
pydata	22	**77.3%**	**77.3%**	**77.3%**	72.7%	563	516	386	**338**
astropy	22	**59.1%**	**59.1%**	54.5%	50.0%	546	470	667	**277**

成功率：75.2% → 75.6%　　**平均 token：526k → 461k（−12%）**

这次跨环境迁移主要提高了执行效率，成功率提升幅度较小

Jiahang Lin, Shichun Liu, et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses, 2026.

AHE：有效组件组合起来未必更强

单独替换进化后的记忆、工具或中间件，都能提高原框架表现。
组合效果却不等于收益相加，重复验证还可能干扰长任务的继续执行。

Variant	All 89 tasks	Easy 4 tasks	Medium 55 tasks	Hard 30 tasks
NexAU_0	69.7%	87.5%	78.2%	51.7%
+ memory only	75.3%	50.0%	83.6%	**63.3%**
+ tool only	73.0%	75.0%	87.3%	46.7%
+ middleware only	71.9%	**100.0%**	81.8%	50.0%
+ system_prompt only	67.4%	75.0%	78.2%	46.7%
AHE full	**77.0%**	**100.0%**	**88.2%**	53.3%

系统提示词单独替换反而退步

总体：69.7% → 67.4%
收益主要来自可执行组件。

难题上，记忆单独使用更好

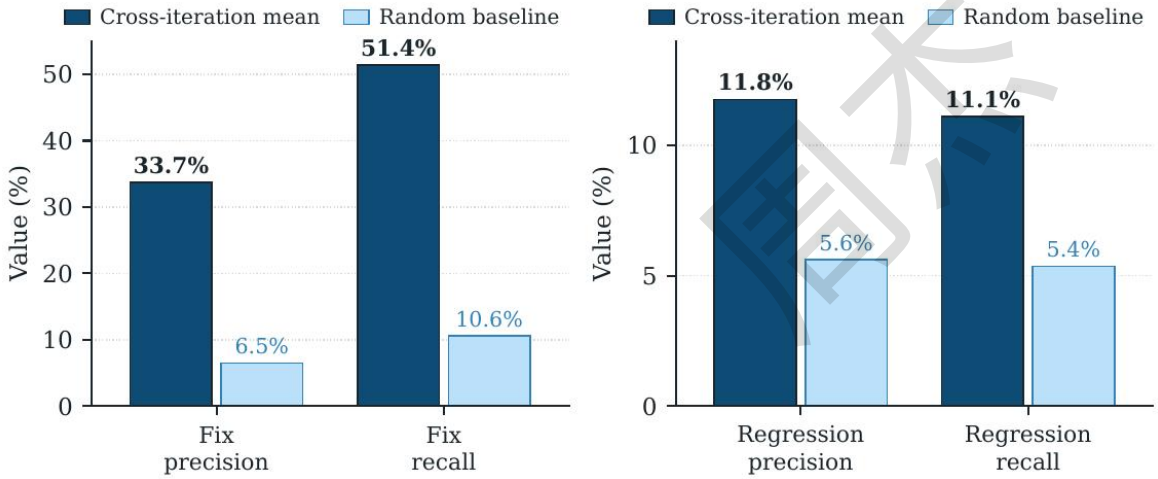
仅记忆：63.3%
完整 AHE：53.3%
工具与反复检查可能过早收束。

组件之间会相互影响，整套框架必须重新验证

Jiahang Lin, Shichun Liu, et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses, 2026.

AHE：能预见修复，仍难预见退步

修改预测能识别部分可修复任务，明显优于随机猜测。
但多数退步仍未提前发现，模型自己的解释无法替代真实回归测试。



修复方向有信号

修复召回率 51.4%，
明显高于随机基线 10.6%。
但修复预测精确率仅 33.7%。

大部分退步仍未预见

退步召回率仅 11.1%，
多数被破坏的任务
未被提前标记出来。

持续改进还要检验已成功的任务，防止修复带来未预见的退步

Jiahang Lin, Shichun Liu, et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses, 2026.

可靠改进：修好当前问题，也经得起后续任务

运行日志能帮助提出修改，难点在于判断哪些修改值得长期保留。
需要同时检查修复效果、后续收益与新增错误，再学习更可靠的修改方式。

跨任务验证

AutoSaddler 检查局部修复能否保留到后续任务，
避免修好一处，却在别处引入新错误。

学会提出改动

RHO 从历史轨迹获得无标签的比较反馈，
Harness-R1 进一步用执行收益训练框架编辑者。

控制反复试题

RRSI 约束提案与版本选择，
减少对固定演化集的迎合与无效复杂度。

每次修改都要接受执行检验，值得保留的更新才能成为下一轮的起点

AutoSaddler：一次修复要经得起后续任务

知道哪里能改之后，还要确保修复能帮助后续任务。
AutoSaddler 从长轨迹追查系统缺陷，将局部失败转为可持续复用的修改。

案例：为什么漏掉了另一个出行候选？

表面上：没有识别歧义

智能体看到一条符合条件的记录，
就直接预订车辆。

仅补“遇到歧义请提问”，
仍无法让它发现漏掉的候选。

根源是：只读了第一页记录

59 条记录只返回了前 8 条，
另一个候选在第 19 号位置。

修改：增加按日期、起终点搜索
完整记录的工具，让歧义可被发现。

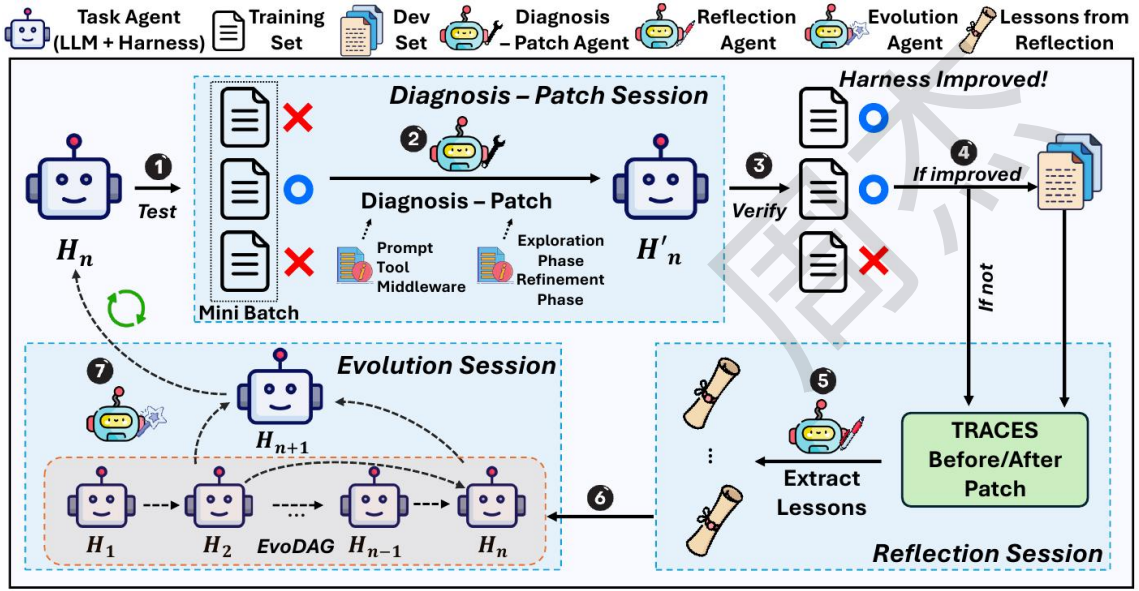
先追查失败根源，才能判断应该补工具、改执行逻辑，还是补充指令

周木

Sungho Park, Wonjoong Kim, et al. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces, 2026.

AutoSaddler：局部修复还要经过独立筛选

诊断可能有误，修复也可能波及其他任务，需要分层检查修改效果。
系统先验证小批量任务，再用开发集筛选，并把修复与退步写入历史。



诊断与修改共享上下文

按需读取长轨迹和源码，
定位根因，修改提示、工具
或中间件。

两层评测，逐步积累经验

先验证当前小批量，再评开发集；
记录修复与退化，写入 EvoDAG，
跨历史版本重组下一轮候选。

每轮修改都接受任务检验，历史同时保留有效经验与失败教训

Sungho Park, Wonjoong Kim, et al. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces, 2026.

AutoSaddler：收益保留到了不同任务组

训练、开发与测试来自不同任务组，软件修复还划分了不同代码仓库。
固定 Opus 4.6 后，优化框架在三个基准的测试成功率均高于原始框架。

Harness (Type)	GAIA2 Universe (Pass@1)			Avg.
	21 (107)	22 (112)	27 (81)	
Default Agent (Manual)	54.8 ± 4.8	51.5 ± 4.9	52.7 ± 4.3	53.0 ± 1.5
GEPA (Auto)	60.1 ± 3.9	47.9 ± 3.4	56.4 ± 0.7	54.6 ± 2.5
Meta-Harness (Auto)	53.0 ± 1.1	51.5 ± 5.2	56.0 ± 0.7	53.2 ± 2.2
AutoSaddler (Auto)	**61.4 ± 2.4**	**60.7 ± 2.4**	**64.6 ± 3.1**	**62.0 ± 1.2**

SWE-Bench Pro (Pass@1)

Harness (Type)	Ansible (96)	Flipt (85)	Element-web (56)	Avg.
SWE-agent (Manual)	40.6 ± 1.9	31.0 ± 3.5	41.1 ± 1.7	37.3 ± 4.8
GEPA (Auto)	50.0 ± 1.3	32.2 ± 1.4	**45.2 ± 0.6**	42.5 ± 1.2
Meta-Harness (Auto)	36.9 ± 2.9	31.3 ± 1.7	38.7 ± 0.8	35.3 ± 2.0
AutoSaddler (Auto)	**58.0 ± 1.8**	**36.5 ± 1.8**	43.5 ± 1.6	**46.9 ± 1.8**

Terminal-Bench 2.0 (Pass@1)

Harness (Type)	Test-Split (40)
Terminus 2 (Manual)	40.0 ± 0.0
Terminus KIRA (Manual)	47.5 ± 2.5
GEPA (Auto)	42.5 ± 2.5
Meta-Harness (Auto)	43.3 ± 5.8
AutoSaddler (Iter2) (Auto)	45.0 ± 0.0
AutoSaddler (Iter34) (Auto)	**50.0 ± 0.0**

GAIA2：53.0 → 62.0

跨不同用户场景，
提升 9.0 个百分点。

软件修复与终端操作

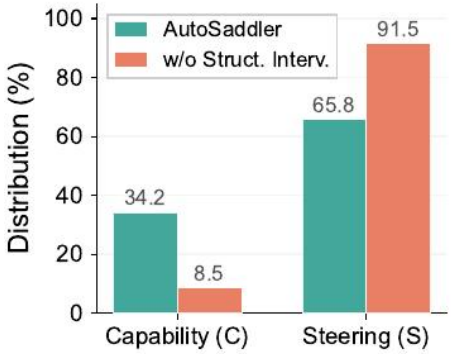
SWE-Bench Pro
37.3 → 46.9（+9.6）
Terminal-Bench 2.0
40.0 → 50.0（+10.0）

未参与修改的任务仍有收益，才说明修复超出了原来的失败案例

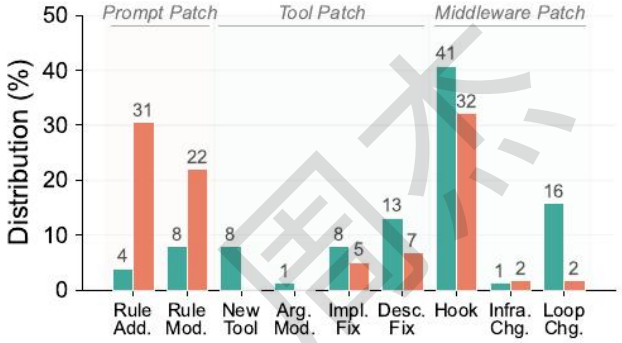
Sungho Park, Wonjoong Kim, et al. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces, 2026.

AutoSaddler：自由编辑容易退回提示修改

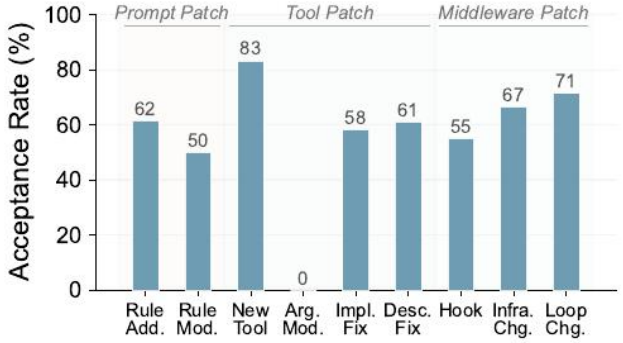
无限制编辑看似灵活，搜索却容易偏向最容易提出的文字修改。
系统显式区分能力补丁与行为引导，先探索工具和执行逻辑，再细化指令。



(a) Capability vs. Steering.



(b) Patch subtype distribution.



(c) Acceptance rate by subtype.

无结构约束：91.5% 的补丁是文字引导；
加入结构后，能力补丁占比由 8.5% 升至 34.2%。

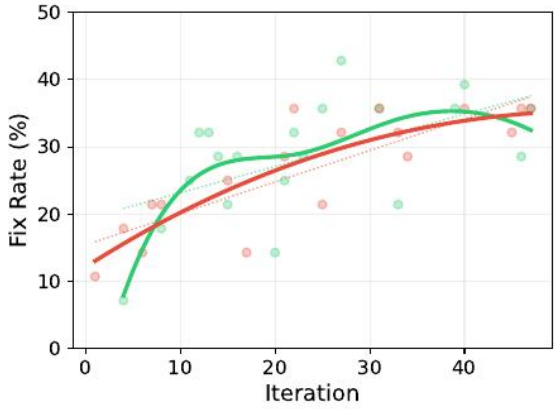
移除结构化修改机制，
GAIA2 成功率 62.0 → 56.9。

组织修改空间，能让搜索更主动地尝试工具与执行逻辑的改进

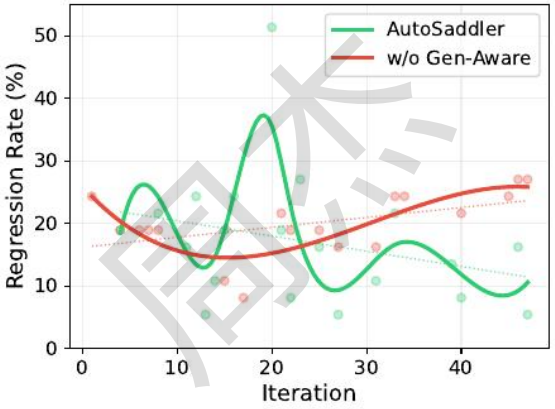
Sungho Park, Wonjoong Kim, et al. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces, 2026.

AutoSaddler：少出新错，才是持久改进

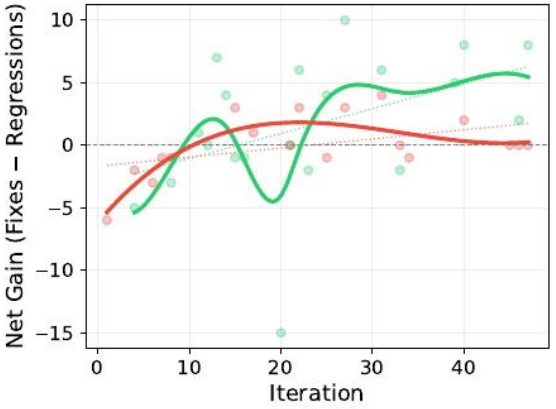
去掉复盘和开发集筛选，旧失败的修复率接近，已成功任务的退步却更多。
GAIA2 测试表现甚至低于原始框架，局部修好并不足以保证整体变强。



(a) Fix Rate (Base-Failed).



(b) Regression Rate (Base-Passed).



(c) Dev-Set Net Gain.

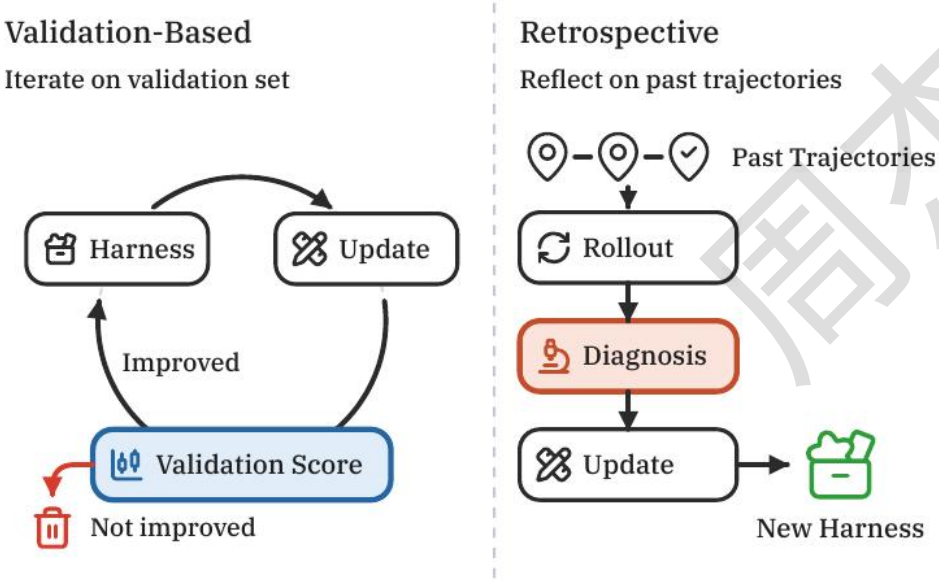
一个具体教训：对高频工具施加过宽的强制改道，会波及许多无关任务。
复盘比较修改前后的轨迹，帮助识别这种超出原问题范围的副作用。

修复与退步必须一起评估，独立筛选帮助保住原本能完成的任务

Sungho Park, Wonjoong Kim, et al. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces, 2026.

RHO：没有答案标签，也能回顾执行经验

独立验证有助于筛选修改，但真实部署中很难持续获得标准答案。
RHO 重做历史任务，利用轨迹中的错误与分歧，为框架改进提供线索。



现实中缺少的是评判信号

过去完成了什么容易记录，
新框架是否更可靠却难以直接打分。

用重新尝试和相互比较找线索

同一任务多做几次，检查内部错误
与结果分歧，再比较新旧框架。
优化阶段不调用标准答案评分。

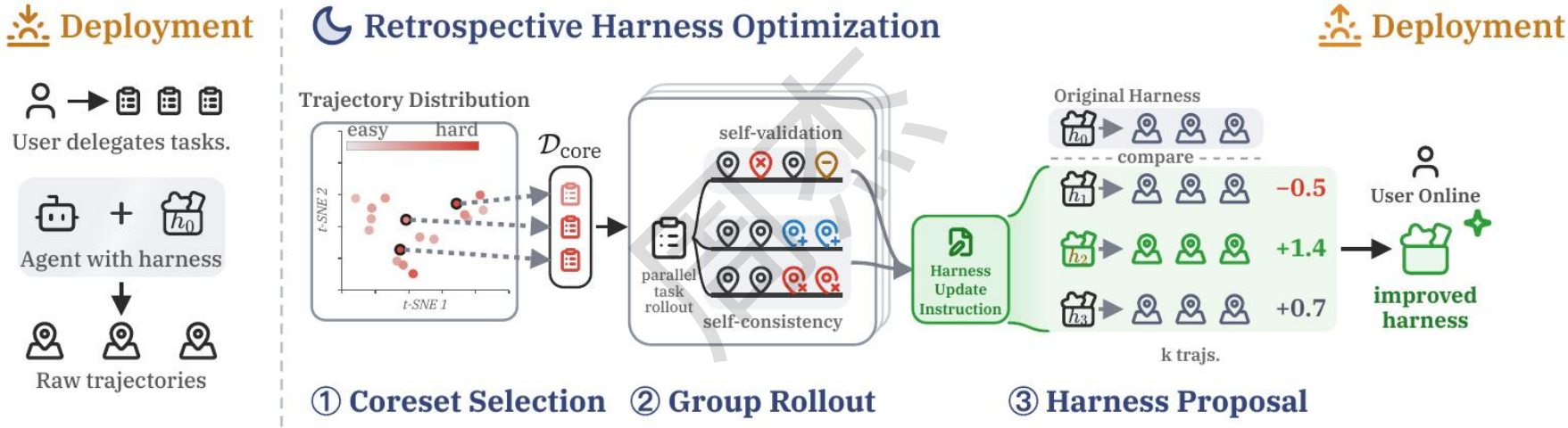
未标注的执行历史也可提供改进信号，关键是如何比较多次尝试

周杰

Wenbo Pan, Shujie Liu, et al. Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference, 2026.

RHO：成组重做让历史问题更清楚

逐条优化全部历史很昂贵，单次执行又难以暴露不稳定行为。
系统选出兼顾难度与多样性的旧题，成组重做，再用自身偏好筛选候选。



选 10 个有代表性的任务

兼顾难度与类型覆盖。

每题并行重做 3 次

检查轨迹内错误与跨轨迹分歧。

提出 3 个框架候选

新旧重跑比较，偏好为正才更新。

代表性任务与多次对照共同提供诊断，自身偏好承担初步筛选

Wenbo Pan, Shujie Liu, et al. Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference, 2026.

RHO：无标签优化改善了后续任务

优化阶段只使用历史轨迹与模型判断，最终评分采用另一批留出任务。
固定目标模型与基础框架后，三个基准均改善，软件修复收益最明显。

Method	Harness Architecture	SWE-Bench Pro Pass	SWE-Bench Pro Δ	Terminal-Bench 2 Pass	Terminal-Bench 2 Δ	GAIA-2 Pass	GAIA-2 Δ
Vanilla Codex	None	0.59	n/a	0.71	n/a	0.29	n/a
Dynamic Cheatsheet (Suzgun et al., 2025)	Skills	0.62	+0.03	0.73	+0.02	0.30	+0.01
ReasoningBank (Ouyang et al., 2025)	Memory	0.61	+0.02	0.73	+0.02	0.28	−0.01
Sleep-time Compute (Lin et al., 2025)	Memory	0.64	+0.05	0.73	+0.02	0.32	+0.03
RHO	Skills+Tools	**0.78**	**+0.19**	**0.76**	**+0.05**	**0.37**	**+0.08**

改动能够落实到执行行为

软件修复中的验证动作增加 61%，
长程任务上的成功率改善更明显。

不同环境需要不同改法

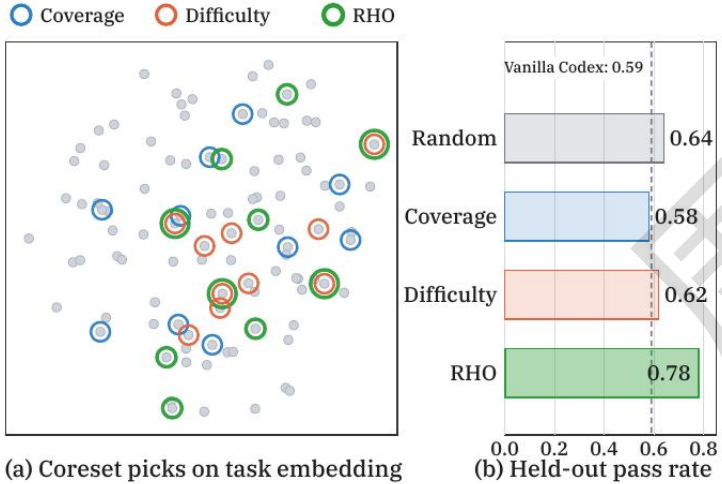
终端任务里生成的脚本未被执行，
收益主要来自写入指令的操作流程。

一次回顾就能改善这组留出任务，优化时无需调用标准答案评分

Wenbo Pan, Shujie Liu, et al. Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference, 2026.

RHO：自身偏好能筛选，却并非准确排名

经验选择需兼顾挑战与覆盖，自身偏好也未必选中测试表现最好的候选。
软件修复中，一个较少被偏好的版本得分更高，说明筛选信号仍有局限。



(a) Coreset picks on task embedding　(b) Held-out pass rate

Dataset	Mean	Chosen	Std	Lowest
SWE-Bench Pro	0.79	0.78	0.06	0.73
Terminal-Bench 2	0.74	**0.76**	0.03	0.71
GAIA-2	0.34	**0.37**	0.03	0.32

偏好最高，不一定测试最好

SWE-Bench Pro 选中候选为 78%，
另一个较少被偏好的候选达到 85%。

偏好可以作筛选信号，
还需要后续任务检验真实效果。

兼顾难度与多样性

自我偏好提供了可用线索，真实收益仍需用后续任务检验

周杰

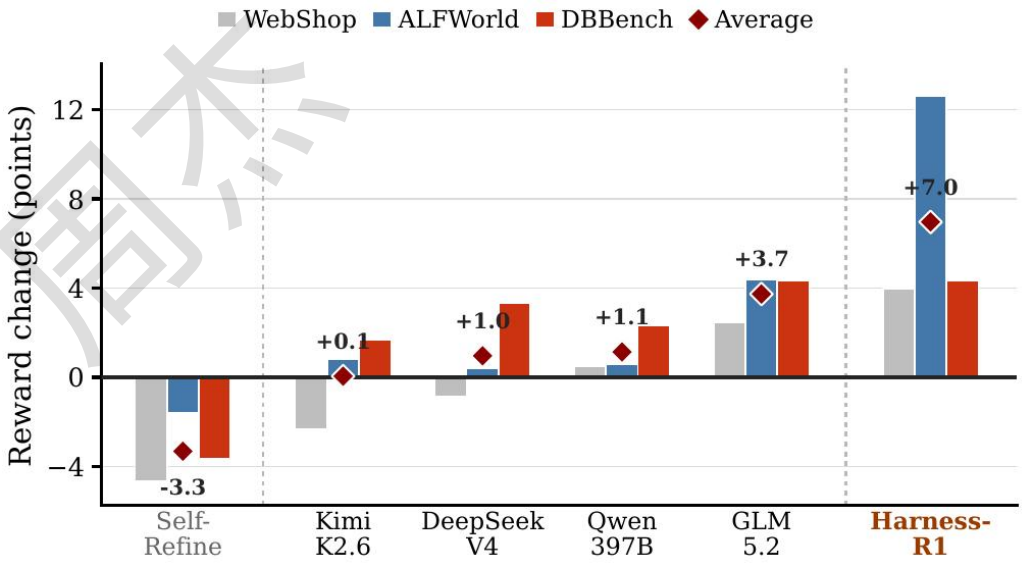
Wenbo Pan, Shujie Liu, et al. Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference, 2026.

Harness-R1：修改框架也能成为学到的能力

筛选修改依赖反馈，提出修改的工程师本身也可以接受训练。
Harness-R1 用目标智能体失败后的实际收益，学习如何改写运行框架。

Harness 决定模型如何行动

组织上下文与状态
校验并转交工具调用
在执行偏离后恢复任务



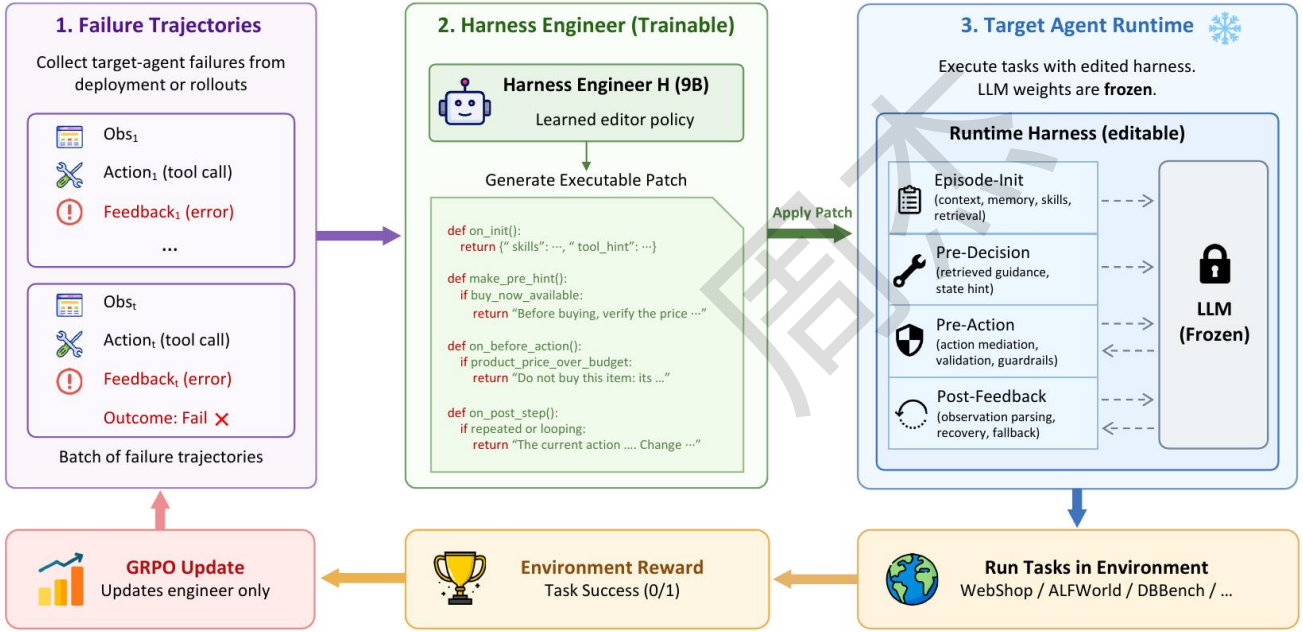
直接让强模型修改，收益并不稳定。
原文图 1 展示环境奖励的变化。

训练对象是框架工程师，补丁实施后的执行收益决定学习方向

Shuai Shao, Kangning Zhang, et al. Harness-R1: Learning to Edit Executable Runtime Harnesses from Agent Failure Trajectories, 2026.

Harness-R1：补丁实效用于训练工程师

自由修改容易破坏运行接口，文字评价又无法判断真实收益。
工程师在四个受约束接口内生成补丁，先经监督训练，再用重跑奖励学习。



四个可编辑位置

初始化时准备上下文
决策前补充状态提示
执行前改写或拦截动作
反馈后识别停滞并恢复

整批任务都重跑，
包括原本成功的任务，
收益同时反映修复与退步。

目标智能体保持冻结，同批任务重跑的收益只更新工程师模型

Shuai Shao, Kangning Zhang, et al. Harness-R1: Learning to Edit Executable Runtime Harnesses from Agent Failure Trajectories, 2026.

Harness-R1：框架学习与模型训练互补

同批任务重跑下，工程师提高了冻结 Qwen3.5-9B 的平均成功率。
目标模型先微调后，再训练与其适配的工程师，仍能获得额外收益。

Method	ALFWorld							WebShop		DBBench	Avg.
	Pick	Look	Clean	Heat	Cool	Pick2	All	Score	Succ.	Succ.	
Qwen3.5-9B (with default harness)	75.8	66.7	16.9	28.8	9.2	47.5	40.6	66.0	31.2	61.0	44.3
GLM-5.2	77.8	78.6	18.2	27.4	16.1	54.9	45.0	68.4	36.0	65.3	48.8
Ours											
Supervised-only engineer	80.8	66.7	22.1	24.7	11.5	36.1	39.4	67.8	38.6	61.3	46.4
Harness-R1	77.8	81.0	58.4	43.8	34.5	39.3	53.2	69.9	42.2	65.3	53.6
Agent SFT	91.9	**100.0**	46.8	56.2	48.3	85.2	71.2	**71.5**	42.6	63.7	59.2
Agent SFT + Harness-R1	**93.9**	**100.0**	**81.8**	**74.0**	**72.4**	**86.1**	**84.0**	68.7	**43.0**	**65.7**	**64.2**

44.3% 提高到 53.6%

冻结目标，优于仅监督训练的工程师 7.1 个百分点。

59.2% 提高到 64.2%

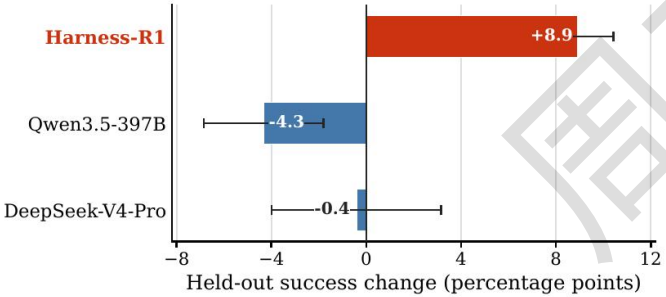
目标先微调，再训练与其适配的框架工程师。

目标模型变强后仍有框架可改，但工程师也需要适配新的目标模型

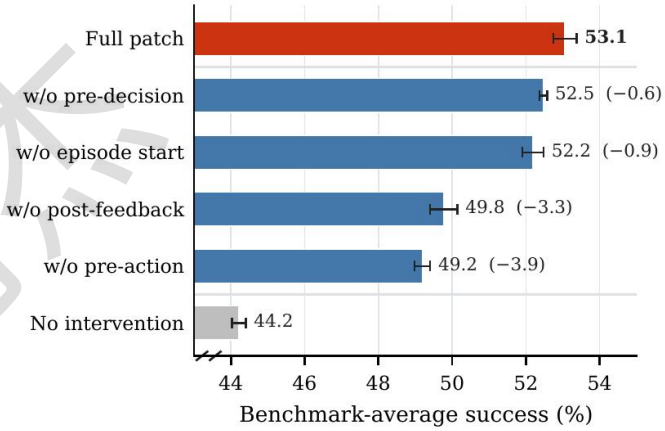
Shuai Shao, Kangning Zhang, et al. Harness-R1: Learning to Edit Executable Runtime Harnesses from Agent Failure Trajectories, 2026.

Harness-R1：少量失败能帮助未见任务

每个基准只提供 10 条失败，生成的补丁用于其余任务，检验迁移能力。
另一组消融逐项关闭补丁接口，动作校验与出错恢复的影响更明显。



(a) Held-out-task generalization from sparse failure evidence.



(b) Fixed-patch lifecycle-position ablation.

未见任务成功率提高 8.9 个百分点

覆盖 1,270 个留出任务，三个随机种子均为正收益。

关闭动作校验与恢复，降幅更大

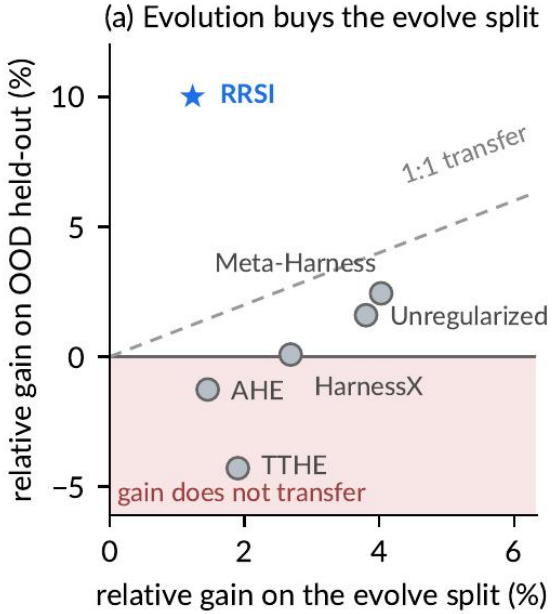
分别下降 3.9 与 3.3 点，具体贡献随环境而变化。

在这些环境中，干预动作与恢复执行是框架改进的重要来源

Shuai Shao, Kangning Zhang, et al. Harness-R1: Learning to Edit Executable Runtime Harnesses from Agent Failure Trajectories, 2026.

RRSI：反复实测也可能过拟合旧题

即使补丁经过实测，反复围绕同一批任务优化仍可能过拟合。
RRSI 约束改动的提出与选择，减少题目记忆、评测噪声和无效复杂度。



三种行为让提升难以迁移

记住任务细节
把特定名称、数值或答案写进程序。

追逐偶然高分
把一次随机成功当作稳定改进。

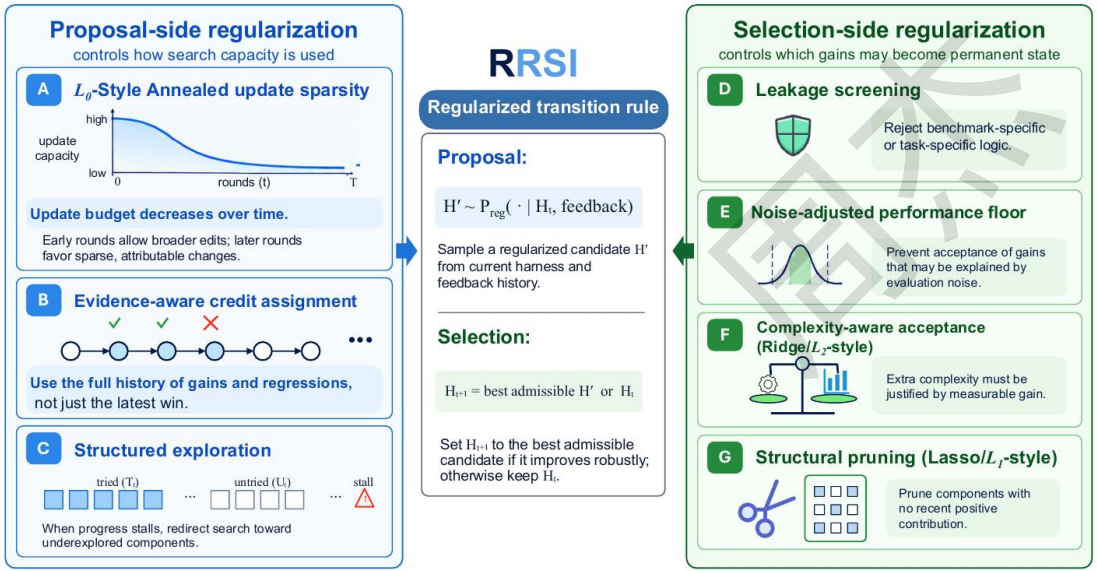
复杂度持续累积
增加调用和上下文，却没有带来通用能力。

演化集得分用于指导搜索，换到新任务仍然有效才是目标

Peng Xia, Rujun Han, et al. RRSI: Regularized Recursive Self-Improvement of Agent Harnesses, 2026.

RRSI：提案与选择都需要约束

一次改太多难以判断原因，只选最高分又容易留下偶然胜出的复杂方案。
系统逐步缩小改动范围，过滤题目专属逻辑，并结合噪声与成本筛选。



提出改动：更少、更有依据

逐轮减少可同时改动的机制；
读取完整历史，避开已失败假设；
停滞时探索尚未尝试的组件。

选择改动：过滤、控成本、剪枝

拒绝题目专属逻辑；
结合评测噪声设置性能门槛；
限制成本增长，删除无效组件。

有限且带噪声的反馈，需要配合可控的改动范围与可靠的保留标准

Peng Xia, Rujun Han, et al. RRSI: Regularized Recursive Self-Improvement of Agent Harnesses, 2026.

RRSI：旧题涨分较少，新任务收益更大

相同模型、初始框架与候选预算下，工作空间实验出现了排序反转。
RRSI 在演化集上提升较小，却在三个未见基准上取得最高分。

Method	In-Distribution		Out-of-Distribution		
	Harvey LAB (Evolve)	Harvey LAB (ID Held-out)	JobBench	GDPval	APEX-Agents
H_0 (no evolution)	89.4	86.9	36.0	48.8	34.2
Meta-Harness (Lee et al., 2026b)	**93.0**	89.2	37.1	49.1	35.7
AHE (Lin et al., 2026a)	90.7	88.7	37.2	47.2	33.1
TTHE (Nie et al., 2026)	91.1	88.5	35.2	47.0	31.7
HarnessX (Chen et al., 2026)	91.8	89.1	36.3	48.5	34.3
RRSI (ours)	90.5	**89.2**	**40.7**	**52.3**	**37.9**

Meta-Harness：演化集 93.0，三个 OOD 基准平均约 40.6。
RRSI：演化集 90.5，OOD 平均 43.6；初始框架 OOD 为 39.7。

这组对照表明，演化集的领先幅度不能代表框架的迁移能力

Peng Xia, Rujun Han, et al. RRSI: Regularized Recursive Self-Improvement of Agent Harnesses, 2026.

RRSI：约束搜索也减少了无效开销

移除提案端或选择端约束，未见任务的平均表现都会下降。
完全去掉正则化后，旧题得分更高，却带来更大开销和更小的迁移收益。

Variant	Harvey LAB (Evolve)	Harvey LAB (ID Held-out)	OOD Avg.	Tokens/trial (m) ↓
H_0 (no evolution)	89.4	86.9	39.7	1.56
Unregularized evolution	92.8	88.9	40.3	3.80
w/o proposal regularizers	90.7	88.8	41.9	2.69
w/o acceptance regularizers	91.5	88.7	41.0	3.59
RRSI	90.5	89.2	43.6	2.42

泛化收益更大

OOD 均分
40.3 → 43.6

运行开销更低

每次评测策略 tokens
380 万 → 242 万

仍有额外计算成本

初始框架仅 156 万；
更强能力仍需付出成本。

这些约束提高了迁移表现并减少开销，但运行成本仍高于初始框架

Peng Xia, Rujun Han, et al. RRSI: Regularized Recursive Self-Improvement of Agent Harnesses, 2026.

高效改进：有限预算怎样支持持续探索

候选增多后，评测与长期运行的开销也在增长。
复用方案、减少执行浪费，并从新论文补充改法，让有限预算支持持续探索。

评测分配

MESH-Harness 复用已有模块，
把运行机会留给更有潜力的组合。

版本比较

RHI 用相邻版本的比较反馈，
以少量迭代改进协作流程。

运行开销

SoL-Pi 优化上下文与工具执行，
减少长期运行中的重复消耗。

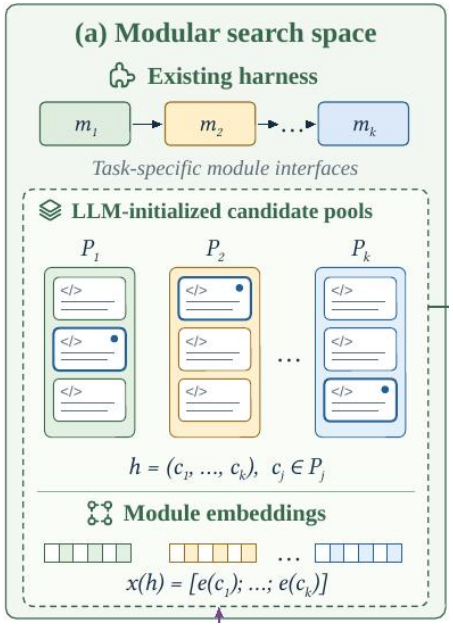
设计来源

ScholarEvolve 从新论文中寻找机制，
让可尝试的改法持续得到补充。

有限预算内，既要找到值得尝试的改法，也要减少比较和执行的代价

MESH-Harness：把评测机会留给好组合

修改要经得起验证，但完整运行一个候选框架，往往比写出它更贵。
MESH-Harness 复用历轮留下的局部模块，让有限评测探索更有希望的组合。



整体重写，难以积累局部设计

一次改动可能只改善检索或错误恢复，
好方案应当保留下来，与其他轮次的模块组合。

固定接口，让不同实现可以替换

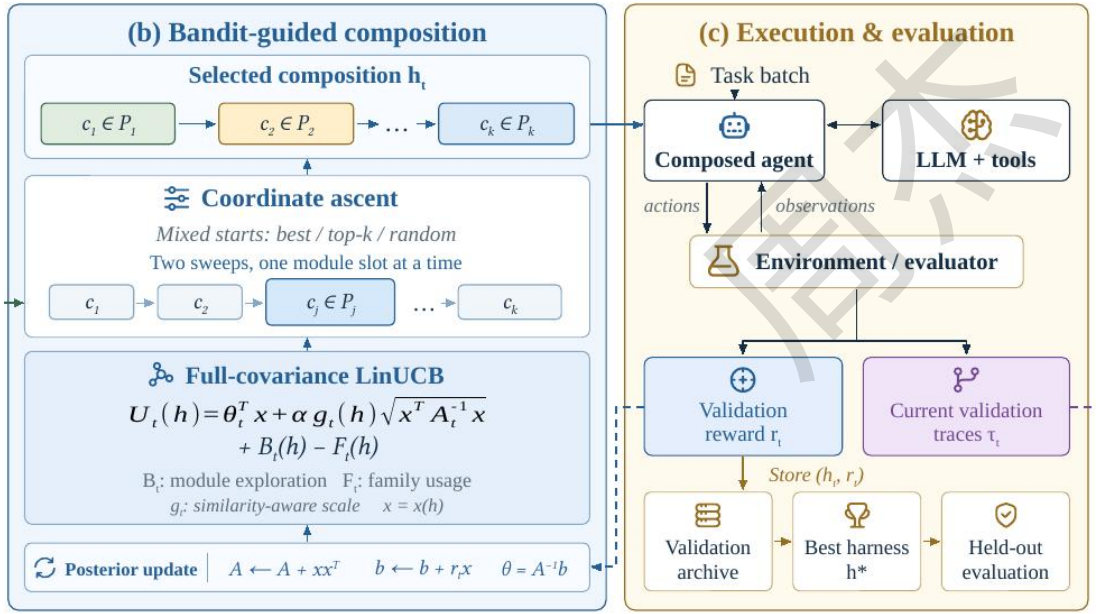
每个功能位置保留一组候选实现，
从每组取一个模块，组成完整执行框架。
模型权重固定，优化对象是外围执行程序。

把有效的局部设计保留下来，下一轮改进就不必每次从头重写

Zhiwei Shang, Yu Huo, et al. MESH-Harness: Self-Improving Agent Harnesses via Bandit-Guided Compositional Evolution, 2026.

MESH-Harness：先挑组合，再花钱验证

模块组合太多，逐个运行会耗尽预算；只挑眼前高分，又容易错过新方向。
方法用历史分数平衡预期收益与探索，再依据真实失败轨迹更新模块。



用 LinUCB 平衡收益与探索

复用模块表示和历史分数，
每次只替换一个位置来搜索，
选出完整组合后再执行评测。

让候选池继续演化

根据失败轨迹修改局部代码，
通过接口检查后加入候选池，
供后续组合选择。

分数决定下一轮评测谁，轨迹决定下一轮改哪里

Zhiwei Shang, Yu Huo, et al. MESH-Harness: Self-Improving Agent Harnesses via Bandit-Guided Compositional Evolution, 2026.

MESH-Harness：同样评测次数，结果更好

同样评测 40 个候选，最终框架冻结后再接受留出测试。
四项汇总指标都超过 Meta-Harness，搜索与测试的合计开销也更低。

文本、数学、代码与交互任务

Method	Text	Math	LiveCodeBench	ScienceWorld
SkillOpt	53.27	46.40	57.00	53.00
DGM-H	42.78	53.00	72.00	52.20
ADIAS	42.50	52.60	69.00	43.10
AHE	47.39	50.60	69.00	49.00
Meta-Harness	50.63	46.20	75.00	53.10
MESH-Harness	**56.33**	**53.21**	**77.00**	**58.10**
Δ vs. Meta-Harness	+5.70	+7.01	+2.00	+5.00

相对 Meta-Harness 分别提高
5.70、7.01、2.00、5.00 分。

总开销降低 14.6%

计入搜索与测试费用，
四项任务合计支出更低。

测试开销降低 44.2%

改进后的框架执行任务时，
也不需要更昂贵的调用。

在这四类任务上，复用模块与组合搜索同时改善了测试结果和总开销

Zhiwei Shang, Yu Huo, et al. MESH-Harness: Self-Improving Agent Harnesses via Bandit-Guided Compositional Evolution, 2026.

MESH-Harness：既要会挑，也要继续改

文本任务中，取消组合引导或停止更新模块，测试分数都会下降。
随机拼装难以选好候选，固定候选池也无法发现尚未出现的新实现。

组合选择：探索哪些候选很重要

Selection	Score	Δ
No UCB bonus ($\alpha = 0$)	51.70	−4.63
$\alpha = 0.2$	53.17	−3.16
$\alpha = 0.4$	54.87	−1.46
Full ($\alpha = 0.8$)	**56.33**	0.00
$\alpha = 1.6$	55.10	−1.23
Random composition	48.80	−7.53
Meta-Harness	50.63	−5.70

随机组合：48.80，完整方法：56.33。

模块演化：新实现与历史轨迹都有用

Variant	Score	Δ
MESH-Harness	**56.33**	0.00
No pool refresh	51.20	−5.13
Refresh without historical traces	53.80	−2.53
Meta-Harness	50.63	−5.70

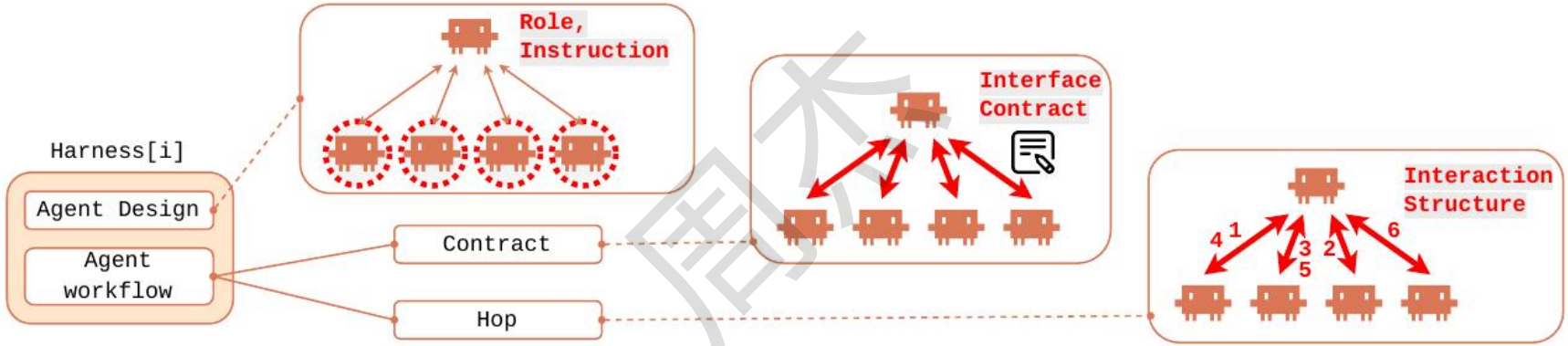
不更新候选池：51.20。
更新但不读历史轨迹：53.80。

有针对性地分配评测，再用轨迹扩展候选，二者共同推动收益

Zhiwei Shang, Yu Huo, et al. MESH-Harness: Self-Improving Agent Harnesses via Bandit-Guided Compositional Evolution, 2026.

RHI：协作流程也能用少量迭代改好

降低搜索开销还有另一条路：围绕当前任务，直接改进智能体的协作流程。
RHI 将角色、信息交接与工作顺序写成文本，用少量迭代改善研究产出。



谁负责什么

角色与指令：
明确子智能体的任务。

向下一步交付什么

Contract：
约定传递的信息与证据。

按什么顺序协作

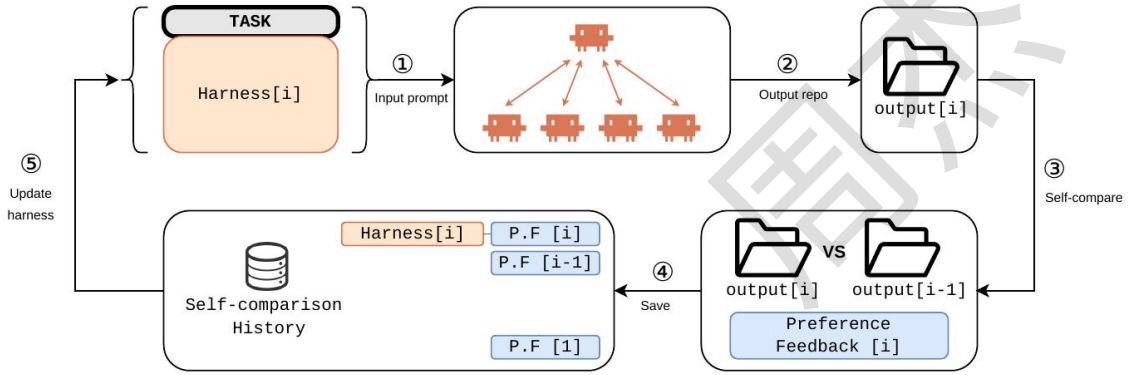
Hop：
规定调用、回退和终止。

模型不变，也能通过更合适的分工与交接提高研究产出质量

Hyunin Lee, Jinglue Xu, et al. Recursive Harness Self-Improvement, 2026.

RHI：每轮只和上一版比较

开放研究难用单一分数评判，大量运行候选框架又太昂贵。
RHI 缓存上一版产出，每轮只执行一个新版本，用成对偏好指导后续修改。



一次新执行，一次新旧比较

缓存上一版结果，
围绕同一个任务比较产出。

积累反馈，再改协作规则

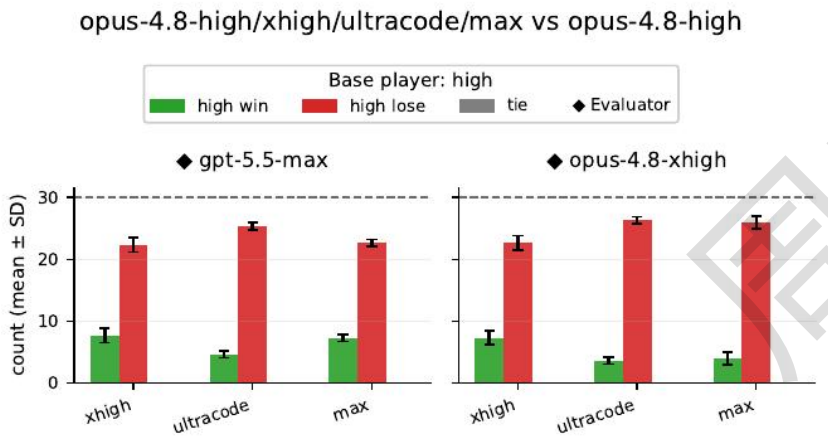
优化器读取比较历史，
重点调整信息交接与流程；
评审细则不直接提供给优化器。

复用旧产出、积累比较历史，以较少的新执行改进任务流程

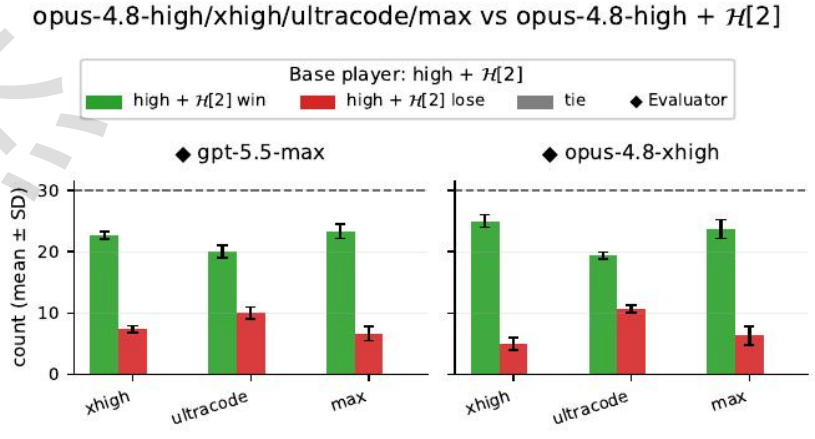
Hyunin Lee, Jinglue Xu, et al. Recursive Harness Self-Improvement, 2026.

RHI：改两轮流程，胜过提高推理档位

30 个合成研究任务要求生成完整仓库，覆盖量化、机器人与药物研究。
在两种模型评审和三个种子下，high 加两轮 RHI 胜过所比较的更高档位。



(a) opus-4.8-high



(d) opus-4.8-high + $\mathcal{H}[2]$

原始 high：大部分比较落后

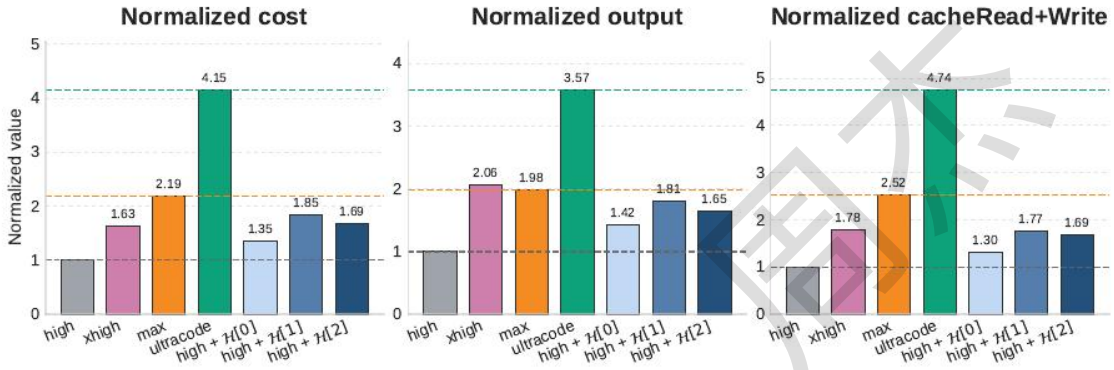
high + 两轮 RHI：胜出更多

在这组任务和模型评审下，定制协作流程比单纯提高推理档位更有效

Hyunin Lee, Jinglue Xu, et al. Recursive Harness Self-Improvement, 2026.

RHI：交接更清楚，执行也更省

改进后的 Opus 4.8 产出更优，任务执行成本也低于 max 和 ultracode。
输出长度不足以解释全部收益，论文将变化与更有效的信息交接联系起来。



(e) Normalized resource usage.

两轮后的执行成本

比 max 低约 23%
比 ultracode 低约 60%

缓存读写量相应减少
约 32% 与 64%。

这里比较各版本的任务执行成本；
多轮搜索、比较和改写另有开销。

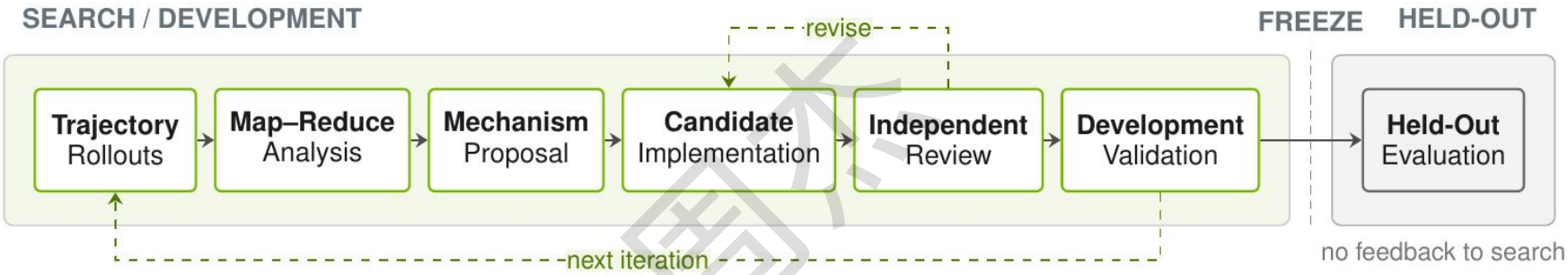
组件语义分析显示，信息交接
规则呈现更明显的任务差异。

更有针对性的信息流值得优化，但其因果作用还需要进一步验证

Hyunin Lee, Jinglue Xu, et al. Recursive Harness Self-Improvement, 2026.

SoL-Pi：长期运行，先减少每步浪费

协作流程可以更省，单个智能体反复读历史、看长日志的开销也值得改。
SoL-Pi 从执行轨迹中寻找浪费，在预设能力约束下搜索更高效的框架。



先广泛提出方向，再逐项打磨

152 个方向、535 个执行环境，
独立实现、审查与开发评测，
最终保留四种可组合机制。

先满足能力要求，再比较效率

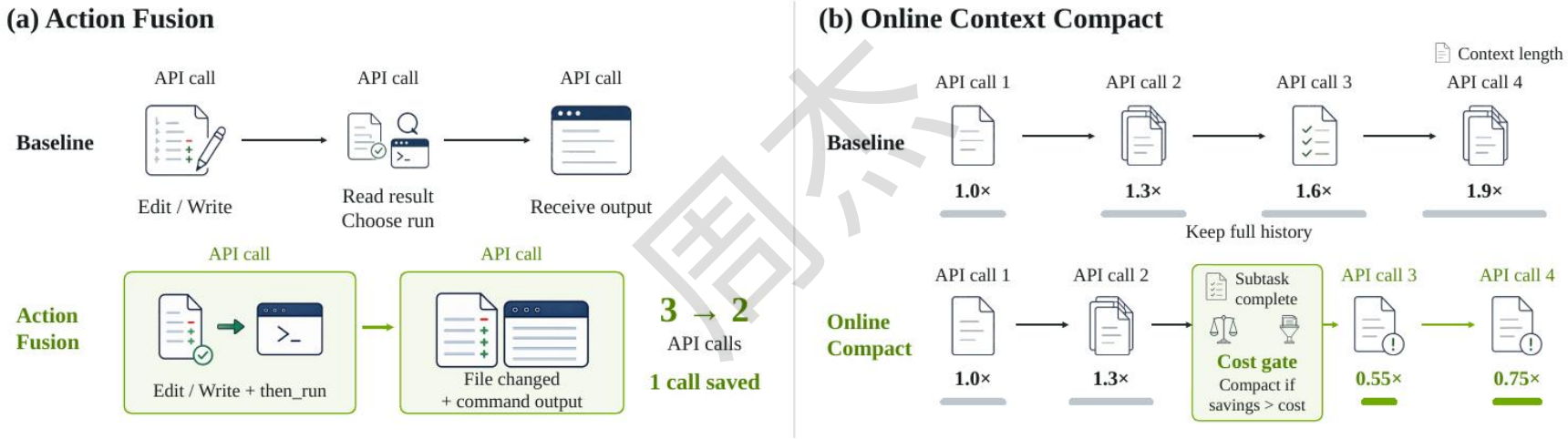
能力容差与效率指标预先固定；
候选冻结后才做留出评测，
其结果不回流到代码修改中。

要扩大自主研究规模，先减少每次交互中不断累积的重复开销

Haozhe Liu, Tian Ye, et al. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness, 2026.

SoL-Pi：少一次往返，少一段重放

修改文件后常要接着运行检查，已完成的子任务也不必反复完整重读。
两种机制分别合并连续动作、适时压缩历史，保留继续执行所需的信息。



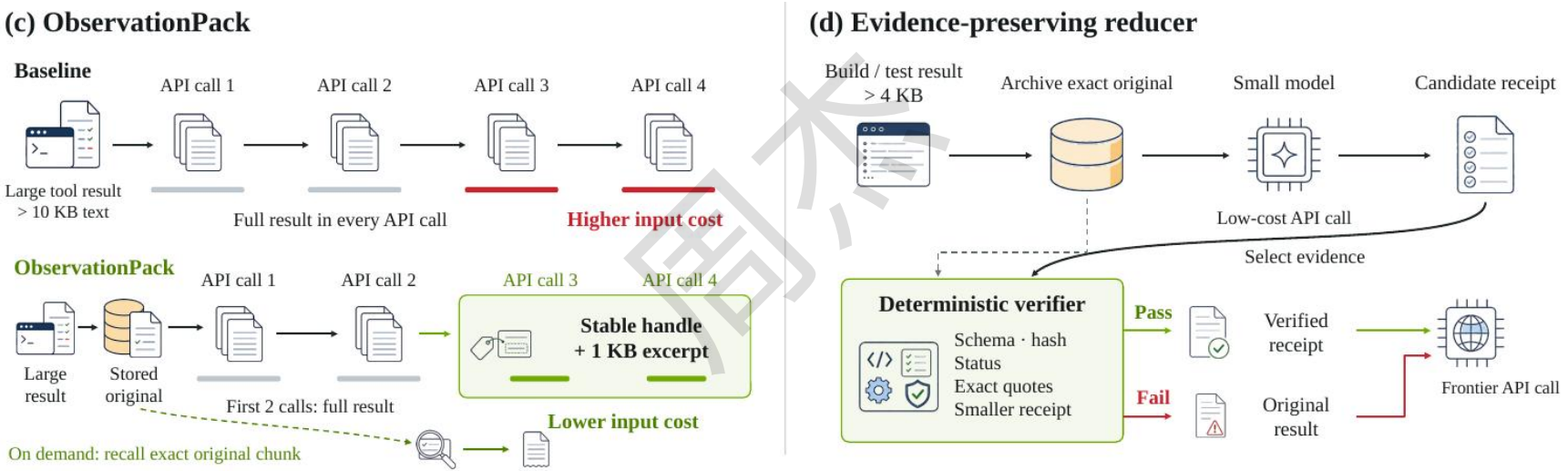
Action Fusion：编辑与后续命令一次提交，减少中间模型往返。
Online Context Compact：在子任务完成时，比较预计输入节省与缓存重写成本。

把可预知的后续动作合并，在合适时机整理已完成的工作

Haozhe Liu, Tian Ye, et al. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness, 2026.

SoL-Pi：少读长日志，仍能找回证据

长输出中只有少数内容影响下一步，直接删掉全文又可能丢失关键证据。
SoL-Pi 保存原文，按需发送节选或提取结果，检查失败时回退到全文。



ObservationPack：前两次完整发送大输出，之后换成句柄与节选，可按需取回原文。
Reducer：小模型提取日志证据，逐条核对原文；检查失败时仍返回完整日志。

减少每次阅读的内容，同时保留原文、精确回读与失败回退

Haozhe Liu, Tian Ye, et al. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness, 2026.

SoL-Pi：流量近乎减半，质量仍有取舍

51 个 EdgeBench 任务中，完整配置在两种模型上的平均分均略低于 Pi。
与此同时，token 流量减少近一半、费用降低约三分之一，体现效率取舍。

Harness	Recorded Token Traffic (B)					Token Cost ($)↓	Avg. Score↑	Token Eff. ($/score)↓
	Input↓	Cache R.↓	Cache W.↓	Output↓	Total↓			
GPT-5.6 Sol (Search Backend)								
Codex [30]	0.0053	3.0287	0.0145	**0.0052**	3.0537	1,787	34.738	1.0086
Pi [35]	0.0011	2.1326	**0.0141**	0.0059	2.1538	1,339	44.833	0.5855
SoL-Pi [Efficiency]	0.0009	**1.0605**	0.0316	0.0061	**1.0990**	**894**	42.003	**0.4174**
SoL-Pi [Performance]	**0.0002**	2.0005	0.0160	0.0056	2.0224	1,271	**47.208**	0.5280
Opus 5 (Held-Out Backend)								
Claude Code [36]	0.0002	1.8116	0.1701	0.0226	2.0045	2,535	43.689	1.1377
Pi [35]	**0.0000**	2.3203	**0.0348**	0.0145	2.3697	1,741	44.756	0.7625
SoL-Pi [Efficiency]	**0.0000**	**1.2626**	0.0352	**0.0123**	**1.3101**	**1,158**	42.224	**0.5376**
SoL-Pi [Performance]	**0.0000**	2.0520	0.0352	0.0144	2.1016	1,605	**50.482**	0.6235

相对 Pi 的完整配置

Token 流量
减少 44.7%–49.0%

API 费用
降低约三分之一

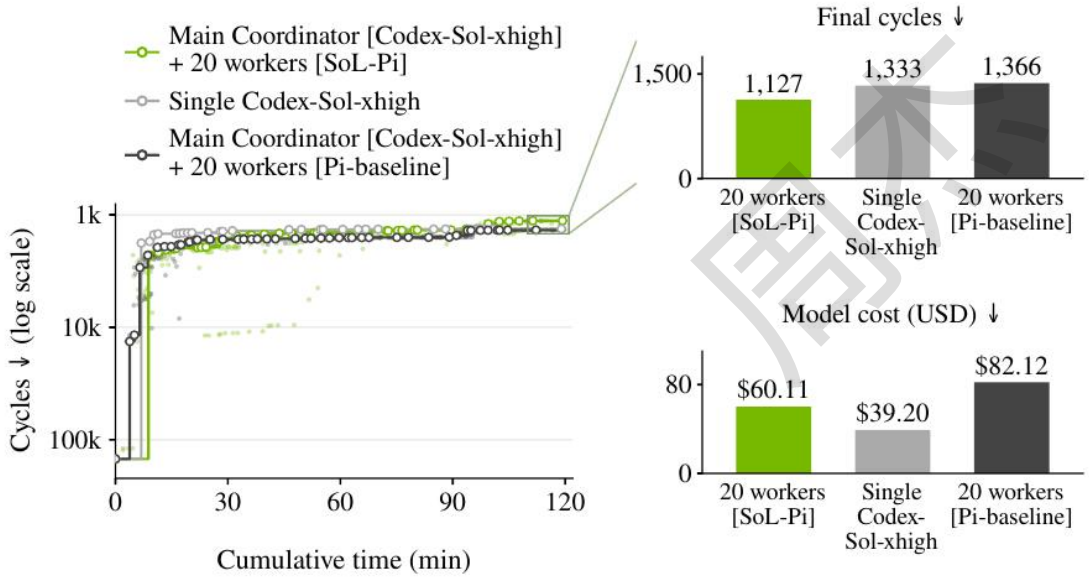
保留约 94% 的 Pi 平均分：44.8 → 42.0 / 42.2。
完整配置偏向效率；表中的 Performance 是按各模型单独选择的单机制配置。

框架效率要与任务质量一起衡量，减少 token 并不等于无损提效

Haozhe Liu, Tian Ye, et al. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness, 2026.

SoL-Pi：单体更省，多体探索也受益

两小时算子优化中，协调者分别带领 20 个 Pi 或 SoL-Pi 智能体搜索。
SoL-Pi 组得到更低的执行周期和费用；各配置仅运行一次，证据仍属初步。



同样规模，更低开销

执行周期：1,366 → 1,127
API 费用：$82.12 → $60.11

单智能体仍然最省钱

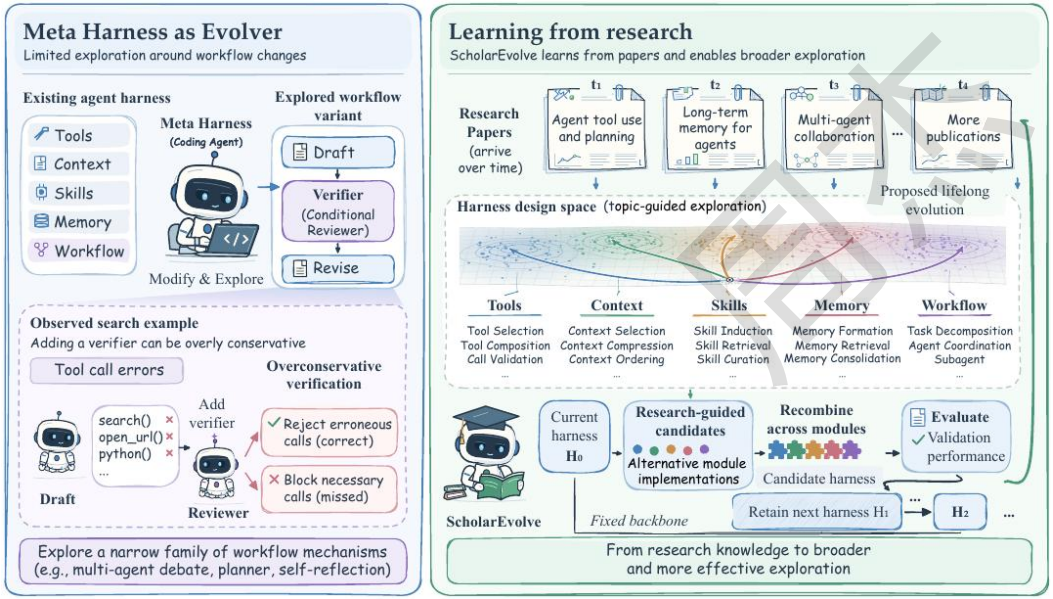
单体花费 $39.20，结果为 1,333 周期。各配置仅运行一次，当前结果属于初步证据。

减少单体的重复开销，也有助于改善多智能体探索的结果与成本

Haozhe Liu, Tian Ye, et al. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness, 2026.

ScholarEvolve：新改法也可以从论文里学

搜索更高效之后，仍可能困在熟悉的改法里：失败能指出问题，却未必给出方案。
ScholarEvolve 从研究文献引入新机制，扩展固定模型可尝试的设计。



突破已有经验的局限

工具、上下文、技能、记忆、
工作流分别寻找候选机制，
扩展可尝试的设计范围。

让新研究触发新一轮改进

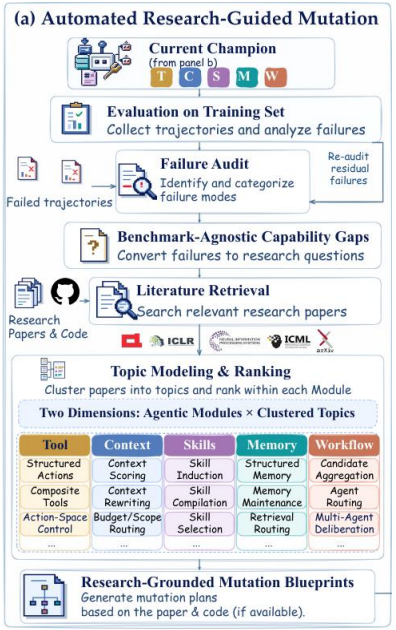
持续引入论文、实现并评测，
把有效设计保留在执行框架中。

执行经验指出哪里需要改，研究文献补充尚未尝试的改进思路

Jingbo Yang, Kwei-Herng Lai, et al. Learning from Research: Toward Lifelong Agent Harness Evolution, 2026.

ScholarEvolve：读懂机制，还要验证组合

找到相关论文不等于改好框架，单个有效模块也未必能与其他模块配合。
方法先按机制组织候选、按接口实现修改，再通过完整执行选择组合。



1 先覆盖不同机制，减少重复探索

将失败抽象成能力缺口，检索论文并合并相近主题，
轮流从不同机制类别中选择文献。

2 把研究方法写成可执行的修改方案

明确接口、状态和运行步骤，只改一个模块；
通过导入、接口与动作检查后进入任务评测。

3 单项评测后，再检验完整组合

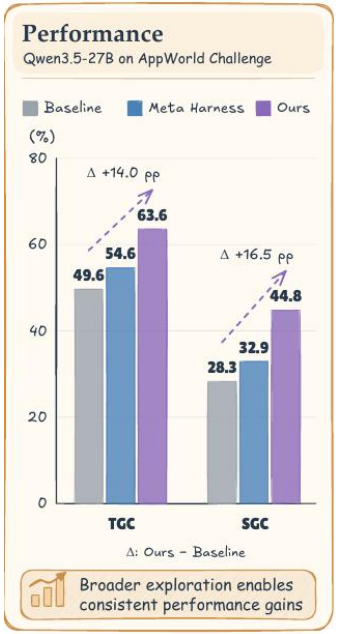
先用单项收益筛组合，再实测交互影响；
确认验证集收益后，才替换当前保留的框架。

论文提出候选方向，接口限定实现方式，真实执行决定哪些组合有效

Jingbo Yang, Kwei-Herng Lai, et al. Learning from Research: Toward Lifelong Agent Harness Evolution, 2026.

ScholarEvolve：新设计能用于未见应用

AppWorld Challenge 包含演化与验证阶段未出现的 Amazon、Gmail 应用。
固定执行模型后，文献引导的框架仍提高完成率，在更高 API 需求下也有收益。



Qwen3.5-27B：未见应用

任务完成率 49.6% → 63.6%
比 Meta Harness 高 9.0 个百分点。

Telecom 服务任务
GPT-5.4-mini 的 pass[1]
72.7% → 81.9%

更高 API 需求仍有收益

Backbone	Reference APIs		
	≤ 9 (n=168)	10–12 (n=147)	> 12 (n=102)
Qwen3.5-27B	13.1	18.4	9.5
GPT-5.4-mini	5.4	15.9	7.2

超过训练中的 12 个 API 上限，
两种模型仍提高 9.5 / 7.2 分。

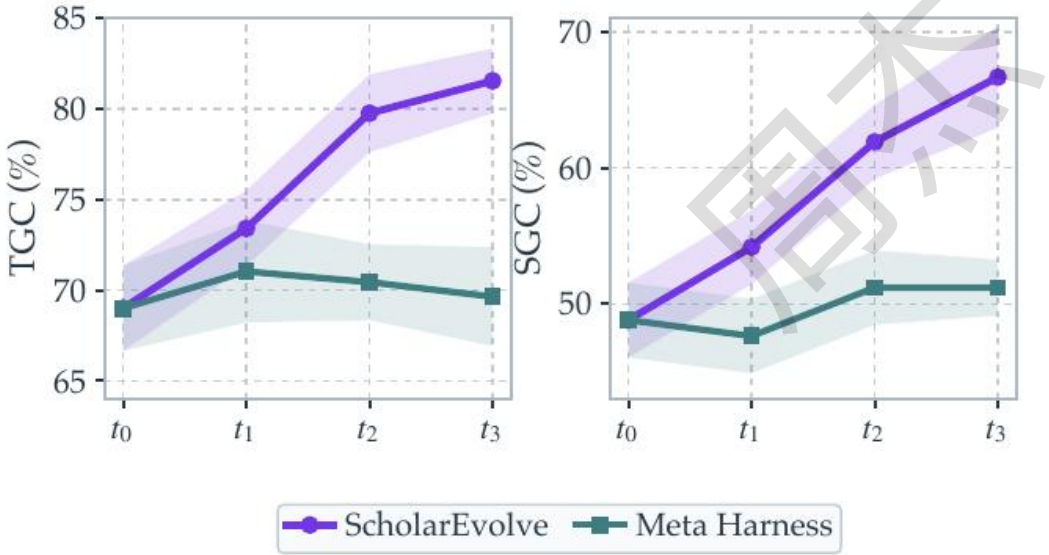
文献带来的设计能超出已见失败场景，泛化收益仍要由留出任务检验

周杰

Jingbo Yang, Kwei-Herng Lai, et al. Learning from Research: Toward Lifelong Agent Harness Evolution, 2026.

ScholarEvolve：论文持续更新，框架继续改进

三批新文献陆续到来时，保留已有模型、演化轨迹与产物，继续修改框架。
AppWorld Normal 完成率逐轮提高，但单项更强的模块未必组成更强的整体。



单项更强，不一定组合更强

Qwen 技能在开发集的得分
从 75.4% 换成 73.7%，
完整组合反而从
80.7% 升至 84.8%。

先筛选，再共同执行验证

单项收益用于缩小搜索范围；
选择最终框架要看整体表现。

持续引入新想法，还要检验模块配合，才能把外部知识变成有效改进

Jingbo Yang, Kwei-Herng Lai, et al. Learning from Research: Toward Lifelong Agent Harness Evolution, 2026.

递归改进：让提出改法的机制也能更新

改好当前结果，与改好产生结果的方法，是两层不同的目标。
研究状态可以反复修正，研究程序、搜索策略与优化器也可以成为更新对象。

更新优化器

Hyperagents 开放两层程序的修改，
VERSE 用实际执行验证优化器的自我更新。

研究结果与方法

AREX 在研究循环中完善状态与答案，
AIDE² 则进一步改写产生研究结果的程序。

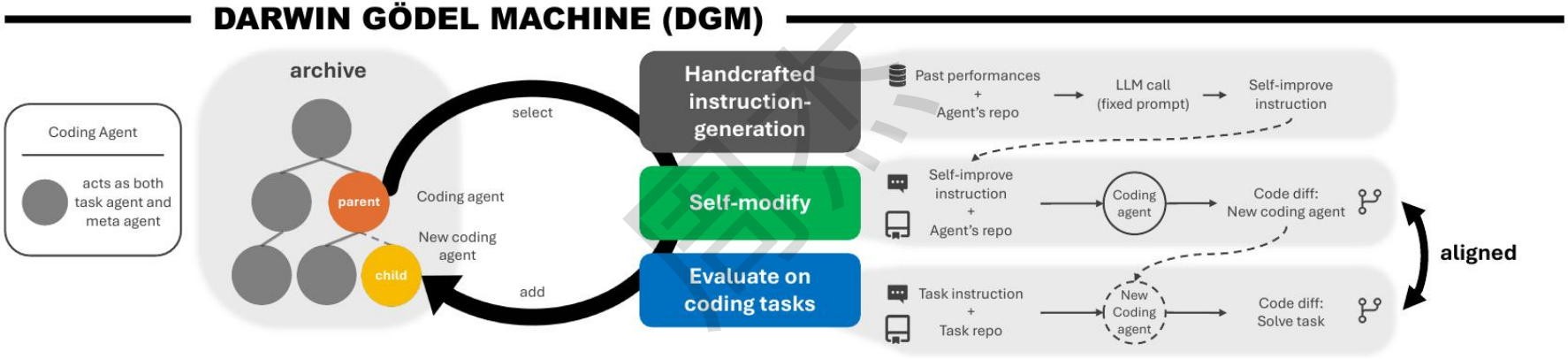
更新改进能力

Dream-RSI 在历史搜索中试验探索策略，
Frontis-MA1 将有效的改进操作训练进模型。

递归进步需要检验：已有更新能否让系统更有效地产生后续改进

Hyperagents：改进者本身，也需要改进

新知识能扩展改进思路，但负责诊断和修改的流程，也可能成为进步的瓶颈。
会评审论文、设计机器人奖励，并不自动意味着更会生成下一次有效修改。



原有机制：改什么，仍由固定流程指导

人工编写的指令生成步骤，不随任务智能体进化。

Hyperagents 想多迈一步

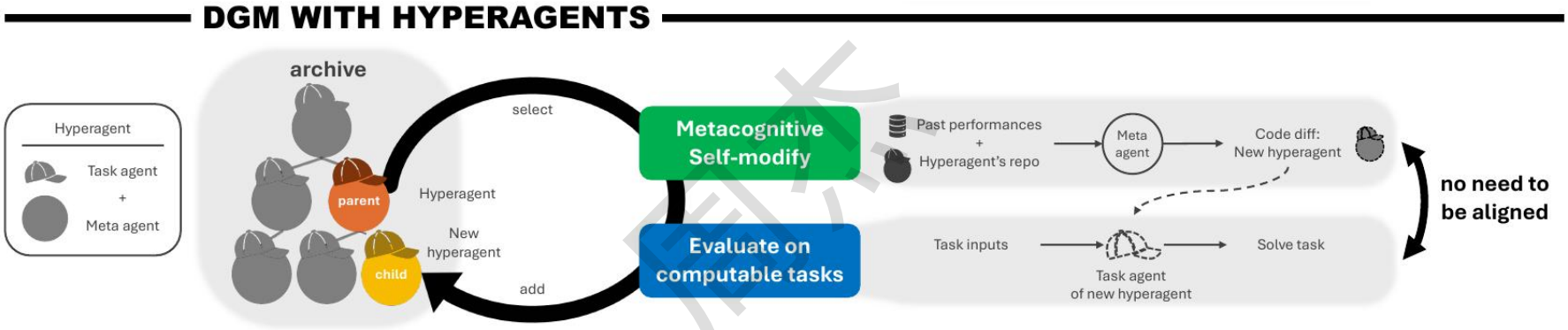
让“如何产生下一次改进”本身也能被修改。

要持续产生好改动，还需要让“如何改进”本身成为可优化的对象

Jenny Zhang, Bingchen Zhao, et al. Hyperagents, 2026.

Hyperagents：把两层能力一起写进程序

固定的修改流程会限制后续探索，因此把任务智能体与元智能体放进同一程序。
元智能体可以改写两者；历代版本保留为探索起点，积累任务经验与改进经验。



任务层：怎样把事做好

例如分阶段评审、
建立奖励设计知识库。

改进层：怎样改得更好

自动加入性能跟踪与持久记忆，
记录有效改动和失败原因。

实验中固定的部分

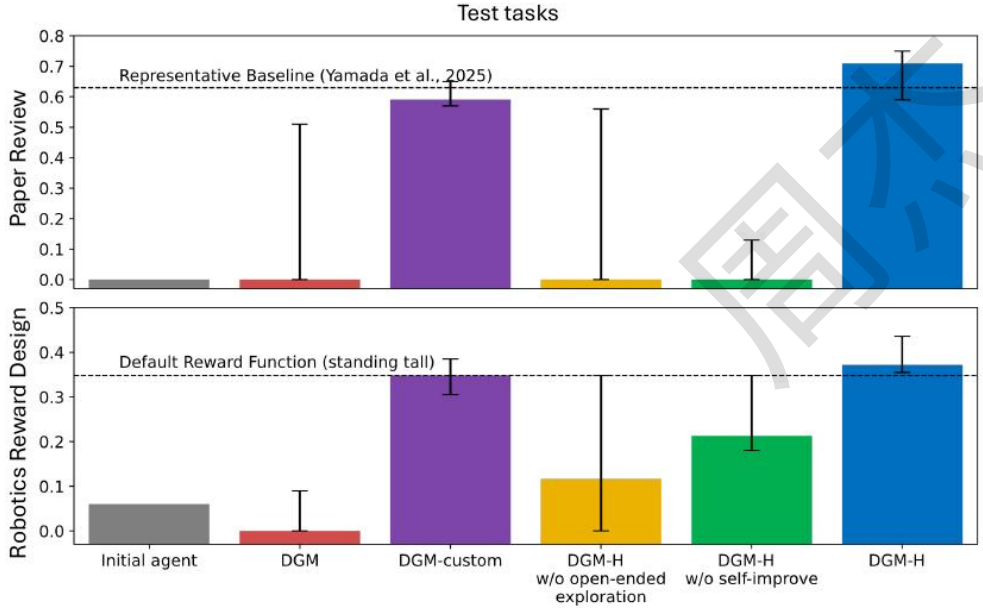
基础模型权重，以及外层
父代选择和评估机制。

任务层学会把事做好，改进层学会生成更有效的下一次修改

Jenny Zhang, Bingchen Zhao, et al. Hyperagents, 2026.

Hyperagents：改进机制可以走出编程任务

在评审与机器人奖励设计中联合进化 100 轮，再评估留出任务。
方法优于不自改等消融；相对人工定制版的中位数更高，但差异未达显著。



论文录用预测：71.0%

人工定制 DGM：59.0%；
静态评审基线：63.0%。

奖励设计：从走路迁移到增高

测试时更常设计出诱导跳跃的奖励，
避开仅仅“站高一点”的局部最优。

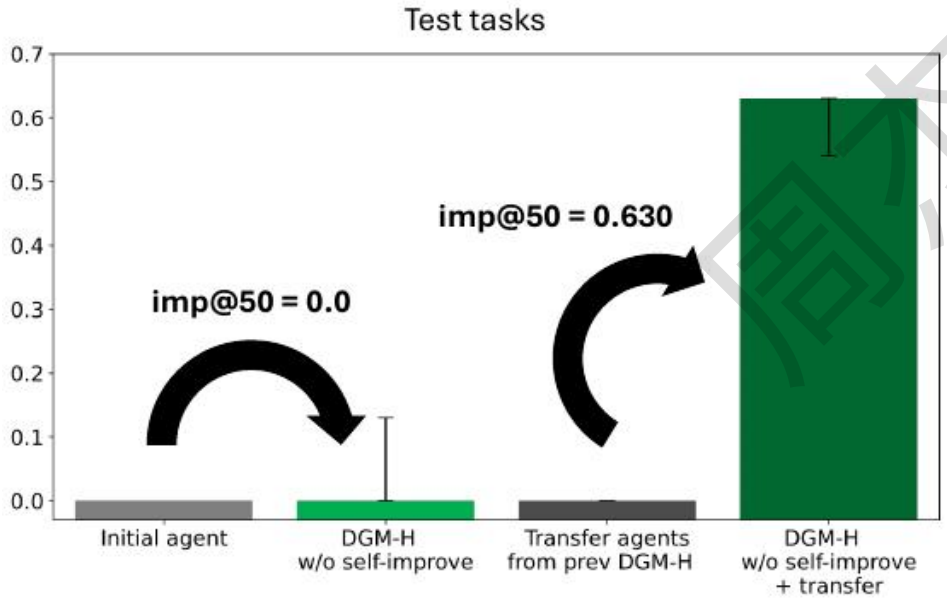
比人工定制版的中位数更高，
但两领域差异均未达统计显著。

让元智能体进化、保留探索分支，有助于把自我改进拓展到编程之外

Jenny Zhang, Bingchen Zhao, et al. Hyperagents, 2026.

Hyperagents：换个领域，仍然更会改进

将已进化的 Hyperagent 转入数学评分，再固定元智能体开展 50 轮修改。
迁移起点的任务得分仍接近零，却能更有效地生成高质量的任务程序。



相同的 50 轮改进预算

初始元智能体：改进量中位数 0
迁移来的元智能体：改进量 0.63
最终任务程序在留出测试上达到 63%。

这个结果测的是“更会改进”

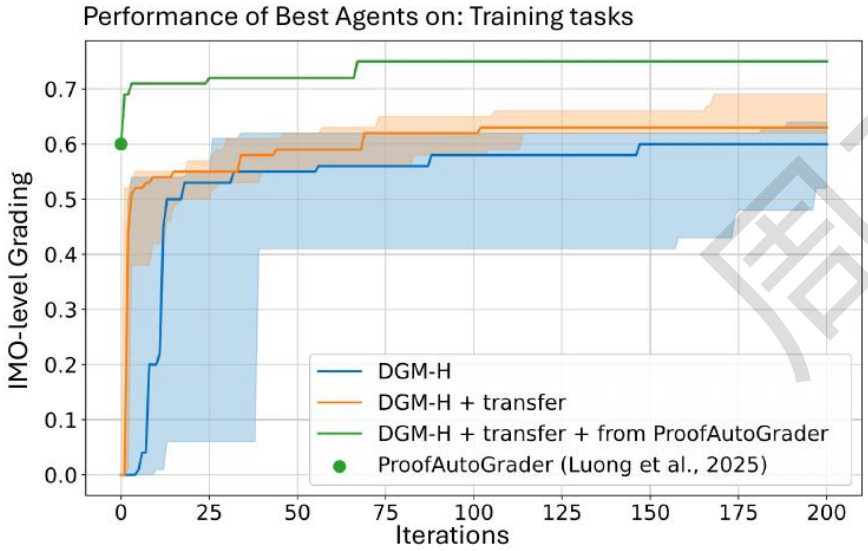
迁移起点本身的评分表现仍接近零；
图中 63% 是继续生成任务程序后得到的。
进化出的记忆与性能跟踪支持后续诊断。

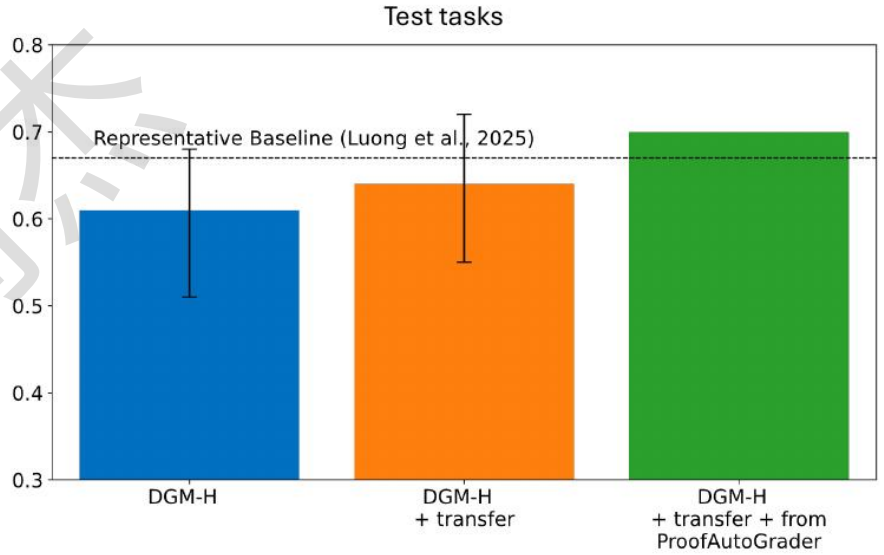
跨领域复用的可以是改进能力，包括记录表现、总结经验与安排修改

Jenny Zhang, Bingchen Zhao, et al. Hyperagents, 2026.

Hyperagents：改进经验能成为下轮起点

迁移后继续进化，数学评分更早取得进展；接上成熟的评分系统也能继续提升。
结果支持改进经验的复用，而持续自我加速仍需要更长、更广的验证。





从头 61% → 迁移后 64%（差异未显著）

强基线 67% → 继续进化 70%

改进经验能够复用并与成熟系统结合，但不能据此认定持续自我加速

Jenny Zhang, Bingchen Zhao, et al. Hyperagents, 2026.

VERSE：改进者自改，先确认改对没有

允许优化器改写自己，还不够：未经验证的反思可能把错误经验带入下一轮。
VERSE 让优化器先用真实执行核验诊断与补丁，再更新自己的工作方式。

仅加自我演化：验证集仍选择初始框架，后续版本没有超越起点。

Optimizer	Self-evolving	Verified	$r^\star$	Val-selected	Last
Blank (initial harness h_0)	–	–	–	$33.95_{\pm 0.82}$	$33.95_{\pm 0.82}$
Meta-Harness (Lee et al., 2026b)	✗	✗	4	$39.20_{\pm 1.11}$	$35.80_{\pm 0.82}$
Self-evolving optimizer (w/o verification)	✓	✗	0	$31.79_{\pm 1.35}$	$31.79_{\pm 3.13}$
Simple verification only	✗	✓	5	$39.81_{\pm 1.41}$	$37.65_{\pm 2.74}$
Self-evolving optimizer + simple verification	✓	✓	6	$\mathbf{41.05}_{\pm 0.82}$	$\mathbf{41.05}_{\pm 0.82}$

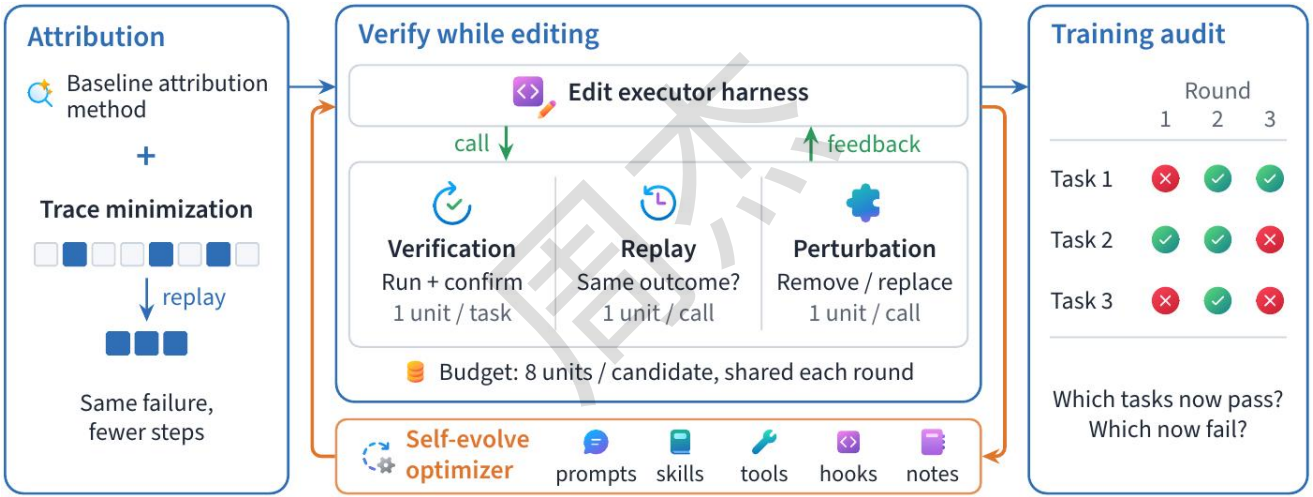
允许先试运行草稿后，自我演化达到 41.05%，高于仅验证的 39.81%。
这里优化的是两套外围程序，优化器与执行模型的权重均保持固定。

在论文的对照中，自我演化只有结合执行验证，才带来进一步收益

Zekai Wang, Yingqiang Ge, et al. VERSE: Verified Self-Evolving Optimizer for Agent Harnesses, 2026.

VERSE：先复现和验证，再积累经验

长轨迹容易造成错误诊断，单次成功也可能是偶然，需要更直接的执行证据。
VERSE 重放和扰动可疑步骤、试跑补丁，再据此改进优化器的提示、工具与技能。



缩短失败轨迹 → 重放并扰动可疑步骤 → 修复连续通过两次才确认。
逐轮记录修好与退化的任务，更新优化器的提示、技能、工具和笔记。

内层改执行框架，外层改优化方法，两层都由可检验的反馈驱动

Zekai Wang, Yingqiang Ge, et al. VERSE: Verified Self-Evolving Optimizer for Agent Harnesses, 2026.

VERSE：改进优化流程，收益迁移到新任务

训练、验证与测试任务分开，验证集选出的框架再接受未见任务测试。
四种优化器加入 VERSE 后均超过各自基线，收益也出现在更新的五语言任务上。

		In-distribution		Out-of-distribution	
Method	$r^\star$	Val-selected	Last	Val-selected	Last
Blank (initial harness h_0)	–	$33.95_{\pm 0.82}$	$33.95_{\pm 0.82}$	$27.41_{\pm 0.82}$	$27.41_{\pm 0.82}$
Meta-Harness (Lee et al., 2026b)	4	$39.20_{\pm 1.11}$	$35.80_{\pm 0.82}$	$28.66_{\pm 0.82}$	$30.84_{\pm 1.95}$
+ VERSE (w/o self-evolve)	6	$34.57_{\pm 0.31}$	$34.57_{\pm 0.31}$	$28.35_{\pm 1.36}$	$28.35_{\pm 1.36}$
+ VERSE (w/ self-evolve)	2	$\mathbf{42.28}_{\pm 1.11}$	$\mathbf{41.36}_{\pm 2.16}$	$\mathbf{37.69}_{\pm 1.56}$	$\mathbf{38.01}_{\pm 1.56}$
AHE (Lin et al., 2026)	6	$33.95_{\pm 1.23}$	$33.95_{\pm 1.23}$	$26.17_{\pm 0.54}$	$26.17_{\pm 0.54}$
+ VERSE (w/o self-evolve)	2	$\mathbf{37.96}_{\pm 2.14}$	$32.10_{\pm 0.31}$	$\mathbf{35.51}_{\pm 1.08}$	$\mathbf{34.58}_{\pm 1.95}$
+ VERSE (w/ self-evolve)	5	$\mathbf{35.19}_{\pm 2.14}$	$\mathbf{35.80}_{\pm 1.11}$	$\mathbf{30.22}_{\pm 0.62}$	$\mathbf{30.22}_{\pm 1.89}$
Self-Harness (Zhang et al., 2026a)	6	$29.63_{\pm 0.53}$	$29.63_{\pm 0.53}$	$29.28_{\pm 2.72}$	$29.28_{\pm 2.72}$
+ VERSE (w/o self-evolve)	5	$\mathbf{40.12}_{\pm 1.35}$	$\mathbf{40.12}_{\pm 1.63}$	$\mathbf{31.78}_{\pm 2.47}$	$\mathbf{35.20}_{\pm 0.31}$
+ VERSE (w/ self-evolve)	1	$\mathbf{34.88}_{\pm 1.35}$	$\mathbf{33.33}_{\pm 1.93}$	$\mathbf{30.22}_{\pm 0.82}$	$\mathbf{31.78}_{\pm 1.95}$
HarnessX (Chen et al., 2026c)	0	$32.72_{\pm 1.11}$	$34.26_{\pm 2.45}$	$27.41_{\pm 1.25}$	$32.09_{\pm 1.65}$
+ VERSE (w/o self-evolve)	1	$\mathbf{38.89}_{\pm 2.45}$	$32.10_{\pm 1.35}$	$27.10_{\pm 1.43}$	$29.28_{\pm 0.31}$
+ VERSE (w/ self-evolve)	3	$\mathbf{35.80}_{\pm 0.82}$	$\mathbf{36.11}_{\pm 1.60}$	$\mathbf{37.69}_{\pm 2.43}$	$\mathbf{32.71}_{\pm 0.93}$

最优框架的测试准确率

同分布：42.28%
强基线：39.20%

新任务：37.69%
强基线：29.28%

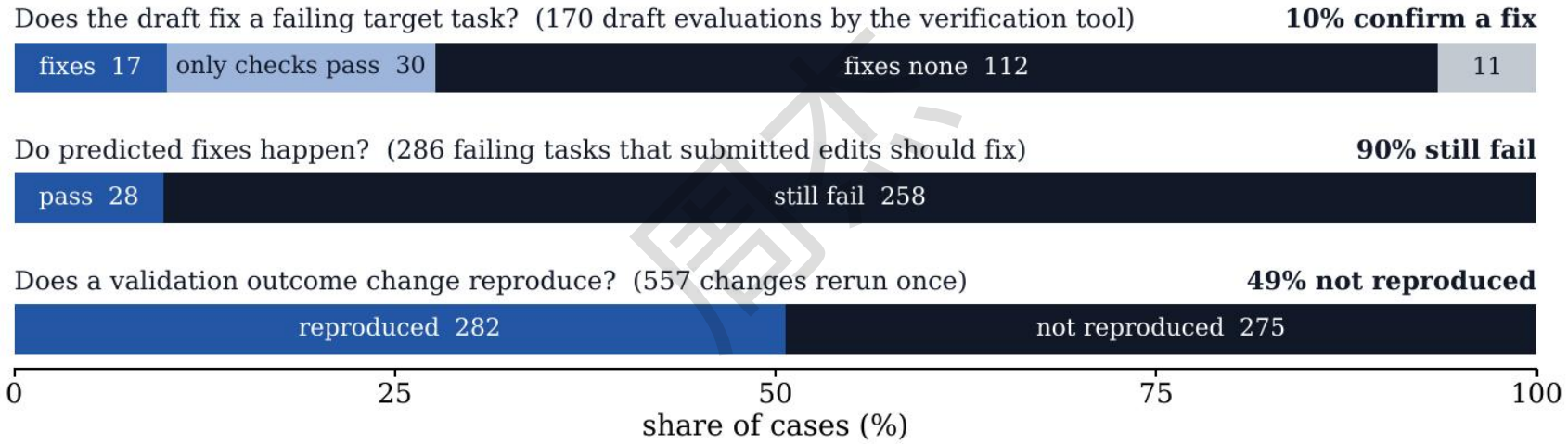
自我演化的增益因宿主而异，
并非每项都高于仅加验证工具。

诊断与验证流程的改进可以泛化，自我演化的额外收益仍因宿主而异

Zekai Wang, Yingqiang Ge, et al. VERSE: Verified Self-Evolving Optimizer for Agent Harnesses, 2026.

VERSE：预计修好，运行后却常常失败

18 次演化运行显示，优化器常高估修复效果，任务结果也存在执行噪声。
验证不仅筛查草稿，还能发现退化，阻止错误经验继续指导后续修改。



自我演化后，81 次草稿评测中有 34 次检查已通过任务，主动防止“修好一个、弄坏另一个”。

值得积累的不只是补丁，还有验证修复、发现退化和处理不确定性的方法

Zekai Wang, Yingqiang Ge, et al. VERSE: Verified Self-Evolving Optimizer for Agent Harnesses, 2026.

AREX：让验证决定下一轮研究查什么

研究循环还有一类基础更新：修正已有发现，明确下一步该查什么。
AREX 逐项核验条件，保留已证实的发现，把证据缺口变成下一轮研究目标。

发现答案：同时满足所有条件

候选空间很大，证据分散在多个来源。
时间、关系或数值等条件彼此关联，
看似合理的候选也可能只满足其中一部分。

核验候选：逐项找出证据缺口

已有候选后，每个条件可单独检查。
保留有证据支持的结论，
将冲突、缺失和待确认条件转成新目标。

继续搜索，却没有明确进展

复核结果，为下一轮明确研究目标

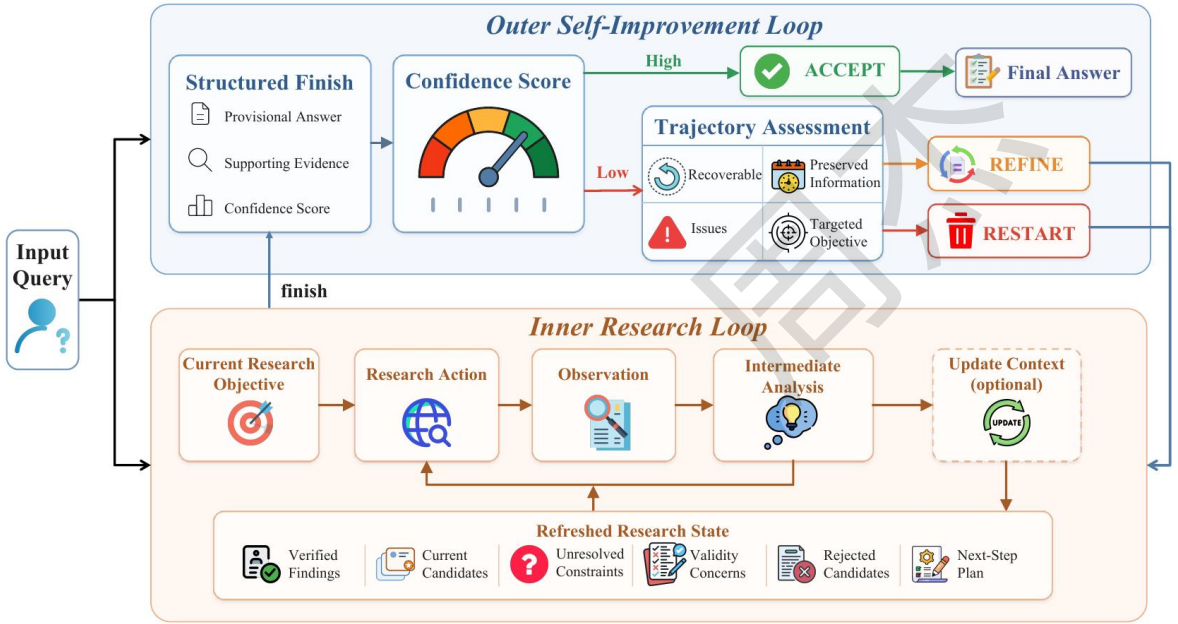
这里循环改进的是研究状态与答案：保留已知进展，继续解决不确定性

周杰

Shuqi Lu, Chaofan Li, et al. AREX: Towards a Recursively Self-Improving Agent for Deep Research, 2026.

AREX：内层找证据，外层定方向

长研究既要保留可靠发现，也要及时放弃无效路线，单纯延长轨迹难以做到。
内层形成候选答案，外层结合证据与置信度，决定接受、定向完善或重新开始。



内层：形成可核验的答案

搜索、浏览并整合证据，
输出答案、来源与置信度。

外层：处理仍未解决的问题

保留有价值的发现继续完善，
无有效进展时重新研究。

模型自主更新上下文

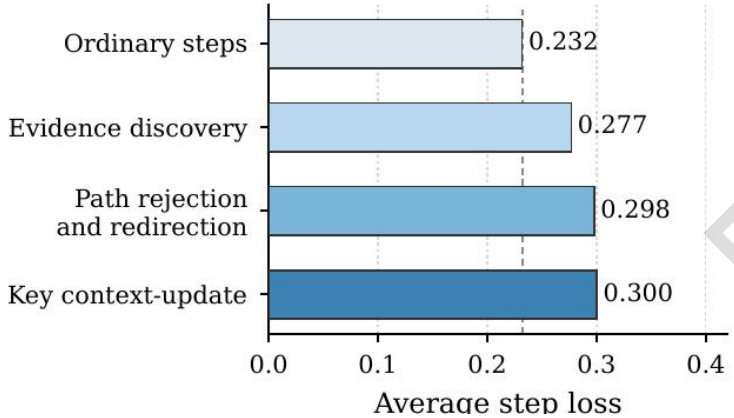
留下证据、被排除的候选与后续计划。

验证决定下一轮目标，自主上下文更新让关键进展在长轨迹中得以保留

Shuqi Lu, Chaofan Li, et al. AREX: Towards a Recursively Self-Improving Agent for Deep Research, 2026.

AREX：关键几步，值得更强的训练信号

找到决定性证据、否定错误方向等关键动作，仍比常规操作更难学会。
AREX 分阶段学习研究能力，强化关键步骤监督，并在强化学习中加入相应奖励。



Training setting	Accuracy
Replace multi-round capability training by mixed training	77.5
Replace key-step focused supervision by random-step replay	74.1
Replace step-aware RL by standard GRPO	79.4
Full AREX	82.5

关键步骤仍有更高损失

整段轨迹监督更容易学会常规操作。

改用随机步骤重放：82.5 降至 74.1

BrowseComp 准确率（%），相同训练预算。

相同训练预算下，集中学习关键决策，比随机重放步骤更有效

周杰

Shuqi Lu, Chaofan Li, et al. AREX: Towards a Recursively Self-Improving Agent for Deep Research, 2026.

AREX：较小模型，也能做好深度研究

Base 使用 122B 总参数、10B 激活参数，Turbo 使用 4B 稠密模型。
两者在多种研究评测上取得较强结果，体现研究流程与训练方式的共同价值。

Model	BrowseComp	GAIA	xbench-2510	DeepSearchQA	WideSearch-en	HLE (tool)
Qwen3.5-35B	61.0	80.0	50.3	68.5	57.1	47.4
Qwen3.5-122B	63.8	81.6	–	–	60.5	47.5
Qwen3.5-397B	78.6	83.5	61.0	82.1	74.0	48.3
Ours						
AREX-Turbo	70.7	81.6	57.0	78.5	68.5	40.6
AREX-Base	82.5	85.4	71.0	89.9	82.0	52.4

Base：广覆盖搜索达到 82.0

WideSearch-en 为原文比较中的最高分。

Turbo：深度搜索仍有竞争力

BrowseComp 70.7，DeepSearchQA 78.5。

Base 六项超过 Qwen3.5-397B；Turbo 五项超过 Qwen3.5-35B

Shuqi Lu, Chaofan Li, et al. AREX: Towards a Recursively Self-Improving Agent for Deep Research, 2026.

AREX：整理状态与外层复核，各有贡献

相同单轮交互预算下，上下文更新使 BrowseComp 从 59.6 提高到 71.4。
再加入外层复核达到 82.5，说明保留研究进展与纠正未决问题能够相互补充。

两种机制的独立贡献

Method	Setting	Acc.
AREX w/o ACU	w/o outer loop	59.6
	w/ outer loop	69.8
AREX w/ ACU	w/o outer loop	71.4
	w/ outer loop	82.5

ACU：自主上下文更新；准确率单位为 %。

多数更新远早于窗口耗尽

Update usage	
Metric	**Value**
Cases w/ update	80.3%
Upper bound	128,000
Min. context tokens	3,788
Mean context tokens	25,721
Median context tokens	25,386
Max. context tokens	128,591

平均约 2.57 万 token，窗口上限为 12.8 万。

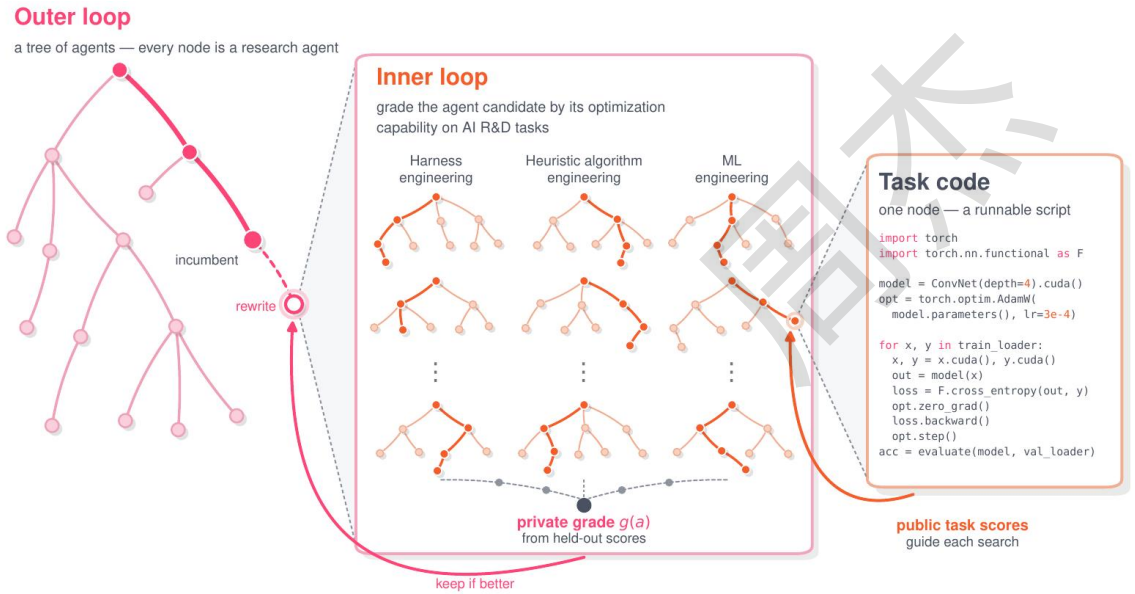
上下文更新保留研究进展，外层复核继续纠正第一轮留下的问题

周杰

Shuqi Lu, Chaofan Li, et al. AREX: Towards a Recursively Self-Improving Agent for Deep Research, 2026.

AIDE²：进一步改写产生研究结果的程序

研究循环能改善当前答案；再进一步，产生这些答案的研究程序也可以修改。
AIDE² 重写研究智能体自身，用相同预算下的产出筛选更有效的搜索与实验方式。



内层：执行具体研究任务

围绕公开反馈迭代任务代码，结束后提交一个候选解。

外层：改写研究智能体

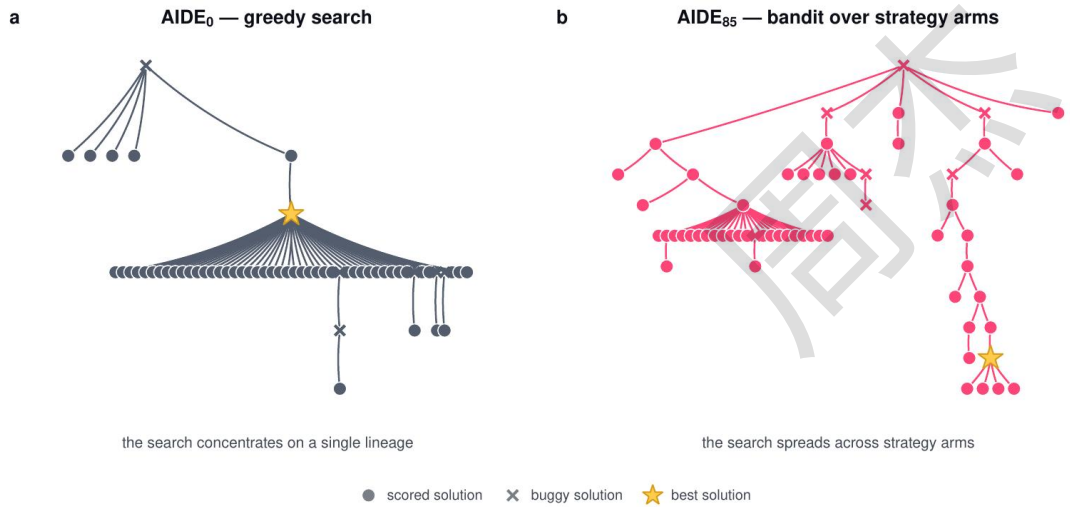
以隐藏数据上的综合表现筛选新版本，再继续改写。每项任务的评测预算固定。

将研究智能体作为优化对象，让研究效率接受固定预算与隐藏数据的检验

Dhruv Srikanth, Bingchen Zhao, et al. Recursive self-improvement of AI research agents, 2026.

AIDE²：更会分配探索，也更会管理历史

八天内提出 99 次改写，其中七次成为最优版本，选择评分从 0.703 升至 0.778。
主要变化落在搜索与上下文管理：少沿单一路线重复尝试，也少反复阅读历史。



搜索：在不同策略间分配机会

用 bandit 选择不同生成策略；
周期性从当前最优解分叉，
换一种思路继续探索。

上下文：摘要与按需失败记忆

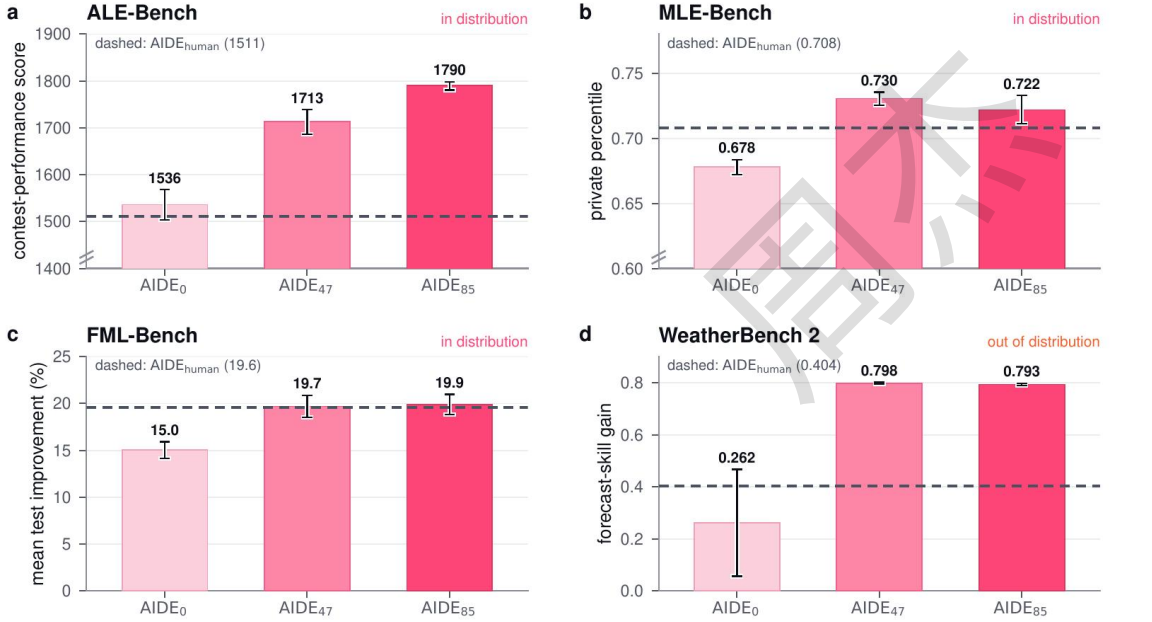
只保留角色所需的紧凑历史；
错误频繁时加入重复报错，
让每次调用的输入保持有界。

同样的资源，探索哪些方向、每步读取多少历史，都会改变研究效率

Dhruv Srikanth, Bingchen Zhao, et al. Recursive self-improvement of AI research agents, 2026.

AIDE²：效率改进，能迁移到未见研究任务

冻结不同阶段的研究智能体，在未参与版本选择的四个外部基准上测试。
最终版本均匹配或超过人工开发的 AIDEhuman，但并非每项任务都随版本单调提高。



陌生领域也能受益

气象数值模型优化没有进入自我改进的选择任务。
演化版本仍获得更高收益。

性能并非逐版本单调上升

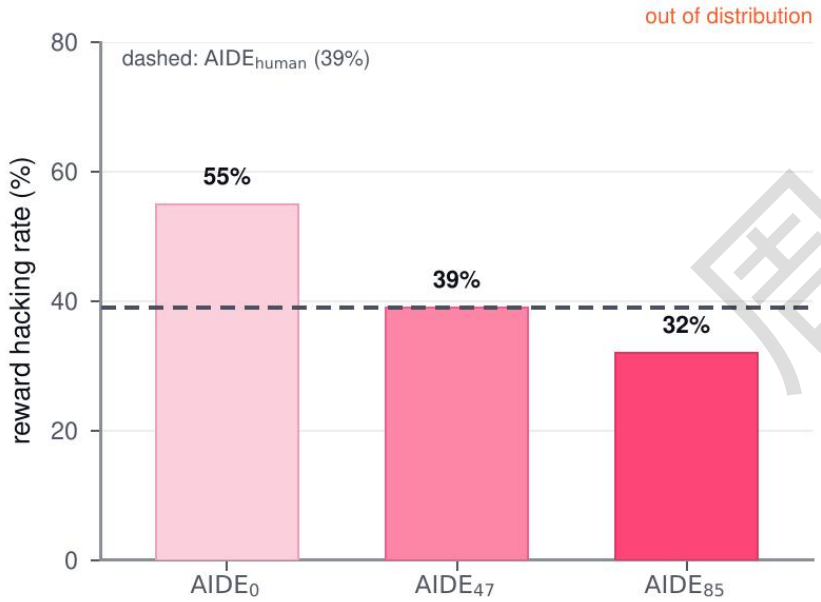
AIDE$_{47}$ 在部分任务略优于最终 AIDE$_{85}$，说明综合选优不能保证每项能力同时提高。

研究效率的收益超出了选择任务，不同任务的最佳版本却未必相同

Dhruv Srikanth, Bingchen Zhao, et al. Recursive self-improvement of AI research agents, 2026.

AIDE²：分数更高，虚假收益反而更少

在未参与选择的算子任务中，先优化独立加速比，再检查真实训练流程的收益。
沿演化版本观察到奖励投机减少，但实验未分离哪一项改写造成了这种变化。



奖励投机率：55% → 32%

最终版本低于人工开发基线的 39%。
结果来自 38 组算子与训练场景组合，
显示改进行为发生了可迁移的变化。

行为变化清楚，成因尚未被分离

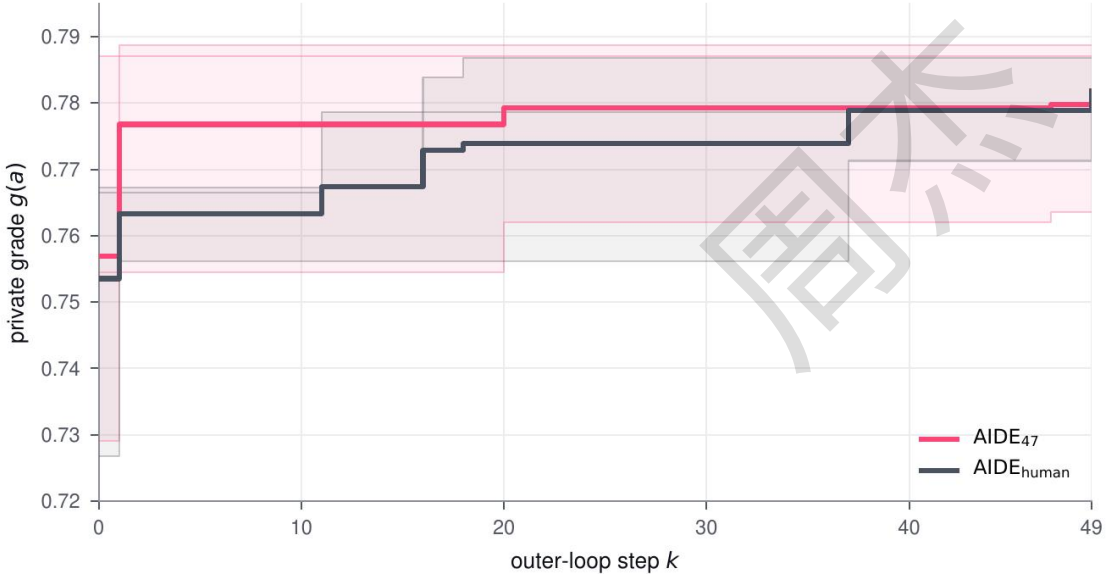
实验没有确定是哪一项改写造成下降，
因此不能把它归因于某个单独机制。

真实下游收益能检验改进质量，这组结果显示奖励投机减少，成因仍未确定

Dhruv Srikanth, Bingchen Zhao, et al. Recursive self-improvement of AI research agents, 2026.

AIDE²：研究更强，是否也更会自我改进

把演化出的 $AIDE_{4\ 7}$ 放到外层，与人工开发版从同一起点推动下一轮改进。
两组各做三次 50 步运行：都能找到有效新版本，但尚不能区分改进效率。



可以继续推动改进

演化版本作为优化者时，
仍能找到被接受的新版本，
没有出现明显能力退化。

尚不能证明改进速度更快

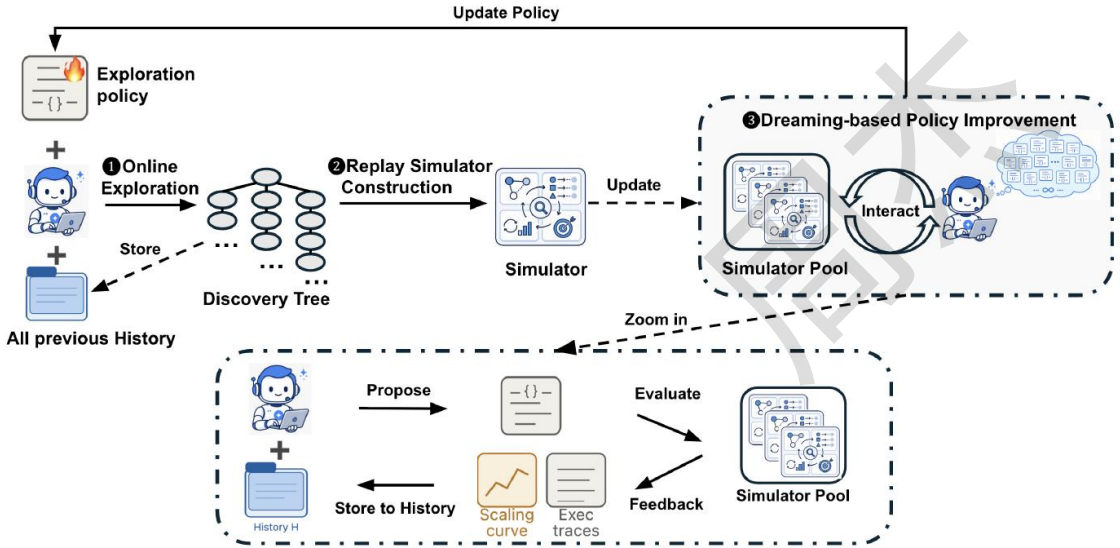
平均终点接近，种子间有波动；
论文认为当前结果不足以
区分两者的自我改进效率。

持续产生有效版本已有证据，能否进一步形成自我加速仍待检验

Dhruv Srikanth, Bingchen Zhao, et al. Recursive self-improvement of AI research agents, 2026.

Dream-RSI：探索策略也应越用越高效

改进者可以修改自身方法，但怎样比较不同的探索策略，仍然是个昂贵的问题。
每次都重跑整段搜索，反馈既慢又贵；Dream-RSI 把已有搜索历史变成回放实验场。



把探索过程独立出来优化

基础智能体与评测器固定；
可改写的调度代码决定
分支、并行与停止。

真实探索与历史回放交替

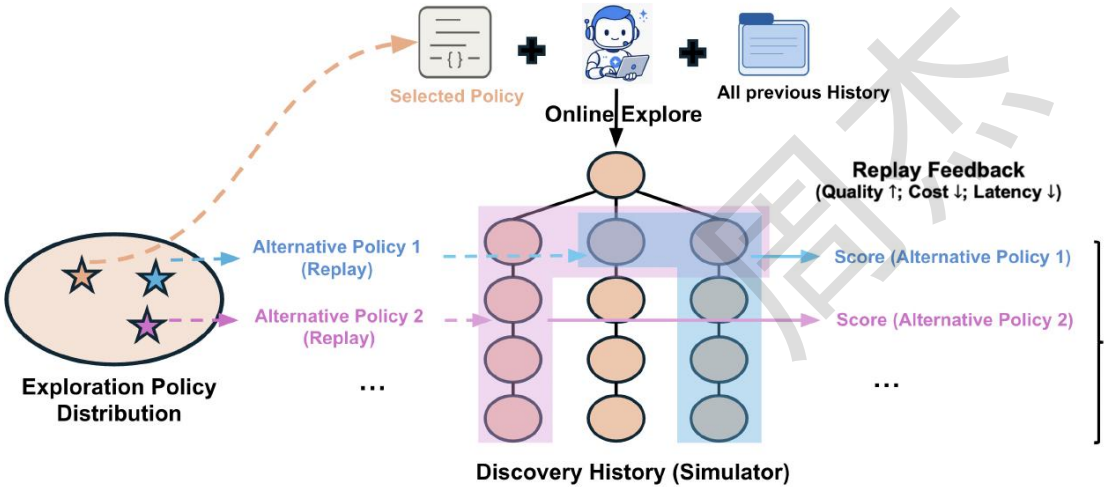
线上产生新历史，离线筛选
新策略，再部署到下一轮。

一次真实搜索留下的历史，可以反复用于筛选更高效的探索策略

Tong Zheng, Xidong Wu, et al. Dream-RSI: Recursive Self-Improvement through Evolving Worlds, 2026.

Dream-RSI：先在历史里试走，再决定如何探索

搜索树已记录每次尝试的结果，比较探索策略时，不必重新生成和执行所有候选。
系统逐步揭示已访问节点，让策略调整分支、并行与停止，再用真实探索扩充历史。



按探索顺序逐步揭示结果

策略只能看到已访问节点，
不能提前读取后续结果；
在累积历史上综合评分。

回放也有边界

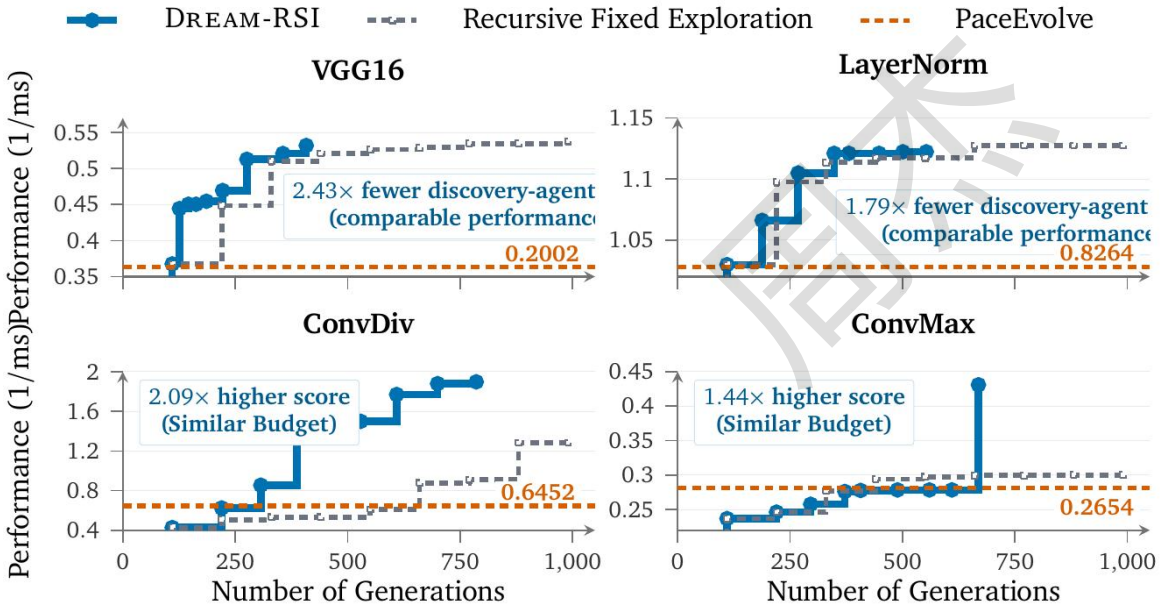
只能重用记录过的结果，
无法预测历史中从未出现
的新分支。

回放降低策略比较成本，真实探索继续覆盖历史中没有的新方向

Tong Zheng, Xidong Wu, et al. Dream-RSI: Recursive Self-Improvement through Evolving Worlds, 2026.

Dream-RSI：探索预算分得好，发现也更好

四项 GPU 算子优化使用相同基础智能体，对比自适应探索与固定策略。
前者能用更少调用达到相近性能，也能在相近调用预算下找到更快的算子。



更早达到相近性能

VGG16：仅需 1/2.43 的调用
LayerNorm：仅需 1/1.79

相近预算，找到更快算子

ConvDiv：性能为 2.09 倍
ConvMax：性能为 1.44 倍
性能指标为运行时的倒数。

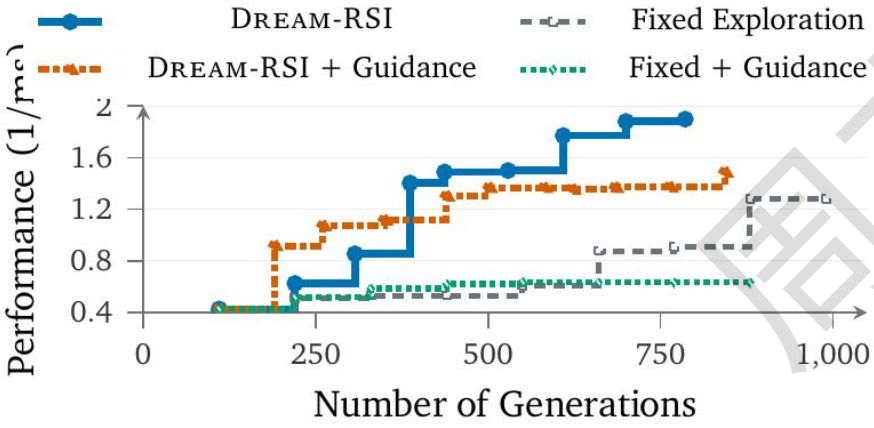
更合理地分配探索预算，可以同时改善发现效率与最终方案质量

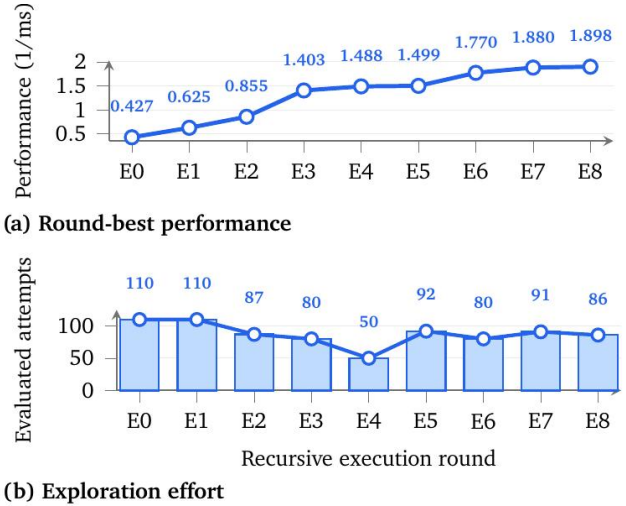
周志

Tong Zheng, Xidong Wu, et al. Dream-RSI: Recursive Self-Improvement through Evolving Worlds, 2026.

Dream-RSI：历史可以用来试策略，不只写提示

在 ConvDiv 上，把历史压缩成方向性提示，反而降低了两种搜索方法的最终性能。
历史回放则支持动态调整探索力度：先节省尝试，进入平台期后再扩大探索。





方向性提示可能约束探索

这一现象来自 ConvDiv 的对比实验。

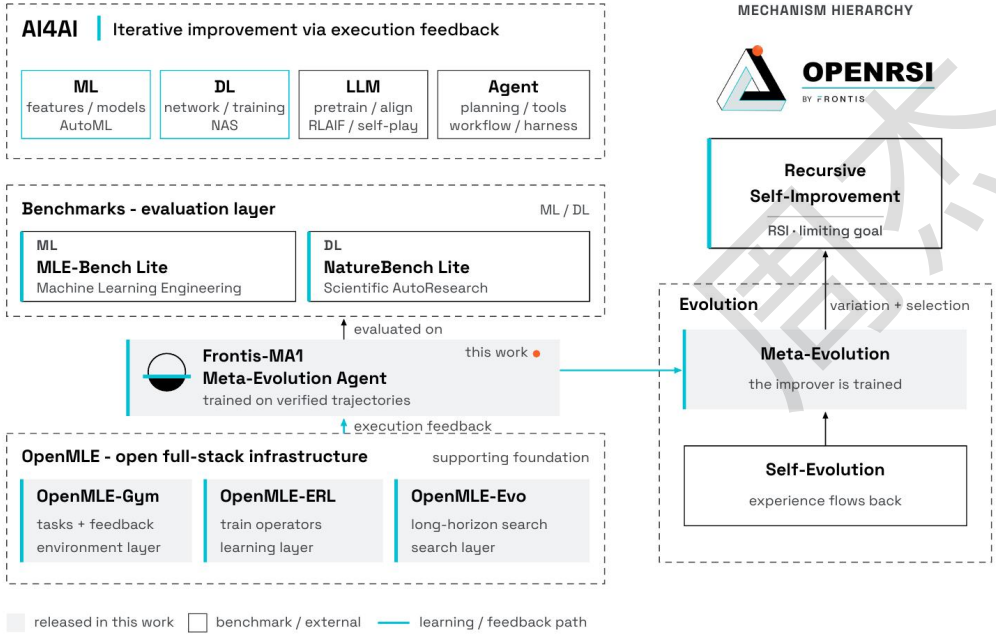
每轮尝试数先从 110 降到 50，
随后回升至 92，并伴随性能改善。

历史的价值还在于比较不同选择，过早限定方向可能束缚后续探索

Tong Zheng, Xidong Wu, et al. Dream-RSI: Recursive Self-Improvement through Evolving Worlds, 2026.

Frontis-MA1：把改进本领练进模型

搜索策略可以变得更聪明，负责提出修改的模型也需要更会改。
Frontis-MA1 用真实执行结果训练局部修改能力，再将这些能力组合成长程搜索。



实验结果必须能可靠验证

OpenMLE-Gym 提供 5,758 个任务，隔离运行程序，返回分数与错误信息，为训练和搜索提供统一反馈。

训练与搜索使用相同操作

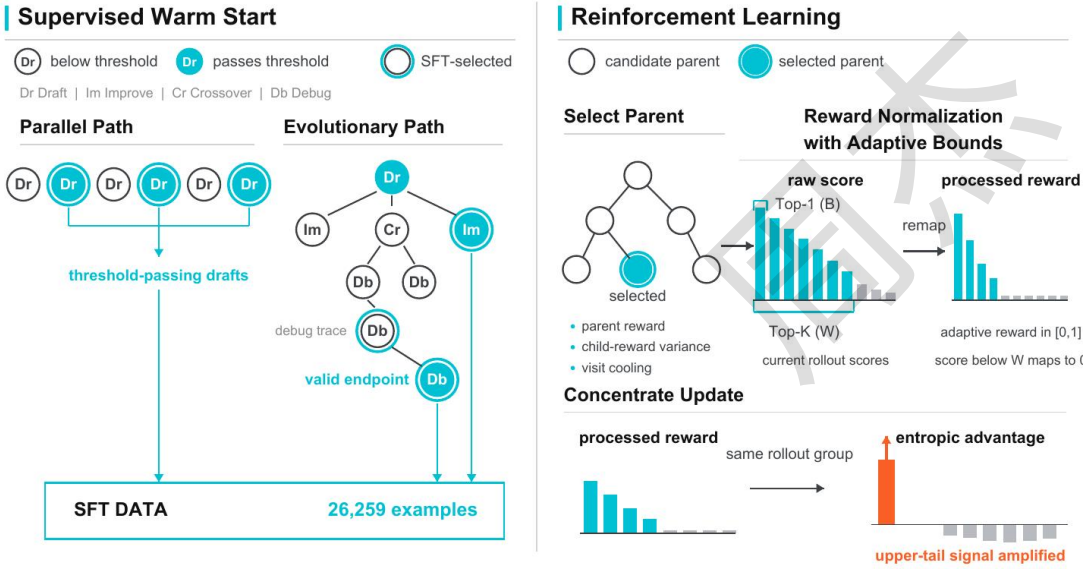
模型学习如何起草、修复和改进程序，搜索再决定何时调用这些能力。
这是迈向递归自改进的可验证一步。

实验经验既能帮助搜索选下一步，也能训练出更会提出修改的模型

Junlin Yang, Che Jiang, et al. Frontis-MA1: Training an AI4AI Model towards Recursive Self-Improvement in Machine Learning Engineering, 2026.

Frontis-MA1：先练好局部修改，再接成长程搜索

完整搜索轨迹难收集，也难在不同策略间复用，训练因此聚焦可重复使用的局部操作。
先学起草、修复、改进与重组，再用执行反馈区分“能运行”和“确实改得更好”。



四种共享操作

Draft 起草方案，Debug 修复错误，
Improve 改进方案，
Crossover 融合两个方案的长处。

SFT 学会做，RL 学会做得更好

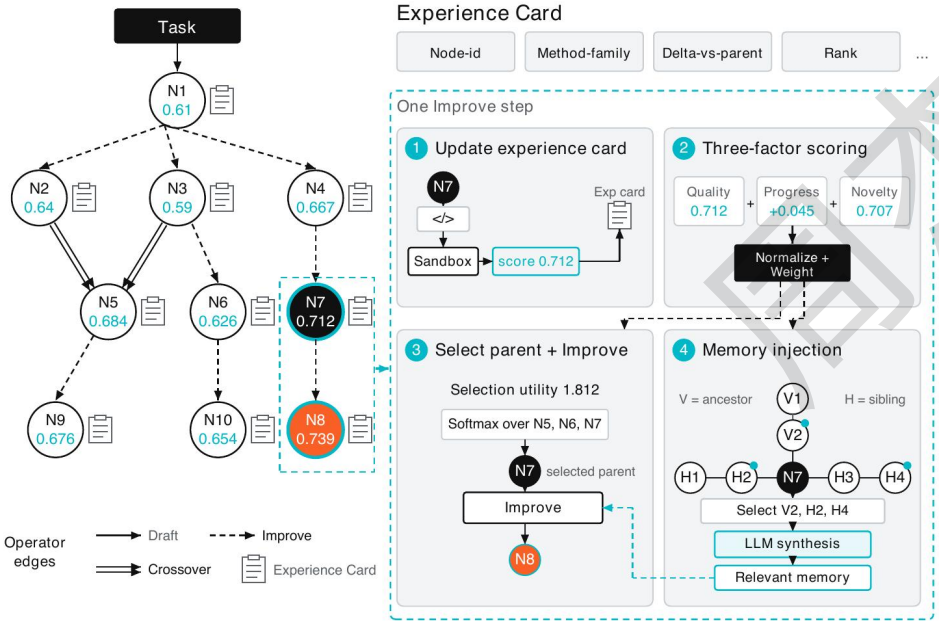
SFT 保留可执行、高质量的修改。
RL 动态调整奖励尺度，放大优质候选之间的差异，更强调组内好方案。
异步执行减少等待慢实验的时间。

训练与搜索共用四种操作，让学到的修改能力直接服务下一轮探索

Junlin Yang, Che Jiang, et al. Frontis-MA1: Training an AI4AI Model towards Recursive Self-Improvement in Machine Learning Engineering, 2026.

Frontis-MA1：既选有潜力的分支，也取相关经验

只追最高分容易丢掉有潜力的方向，把全部历史塞进上下文又会掩盖关键信息。
搜索同时考虑质量、进步与新颖性，并按当前操作取用相关的成功和失败经验。



选父方案：兼顾质量、进步和新颖性

记录分数、相对父方案的增益和方法类别，
给正在进步或探索新方向的分支保留机会。

取经验：随操作选择相关历史

Improve 对照祖先与兄弟方案的得失。
Crossover 寻找两个方案的互补之处。
Debug 检索出现过同类错误的尝试。
需要时才生成摘要，并限制上下文长度。

保留值得探索的分支，再提供与当前修改相关的证据，搜索才能持续推进

Junlin Yang, Che Jiang, et al. Frontis-MA1: Training an AI4AI Model towards Recursive Self-Improvement in Machine Learning Engineering, 2026.

Frontis-MA1：模型训练与搜索改进能够叠加

22 项 MLE-Bench Lite 任务分别固定搜索框架和模型，检验两类改进的贡献。
每任务 12 小时、单张 RTX 4090 限 12 GB 显存；增强搜索保持沙箱总算力预算。

Model / system	Framework	Valid Rate ↑	Medal Average ↑	Human Rank ↑
A. Controlled comparisons				
Frontis-MA1-35B				
Qwen3.6-35B-A3B	OpenMLE-Evo	19.67/22	39.39%	0.5828
Frontis-MA1-35B	OpenMLE-Evo	21.67/22	60.61%	0.7647
Frontis-MA1-35B	**OpenMLE-Evo-Max**	22.00/22	**71.21%**	**0.8126**

周杰

固定框架：训练贡献 21.22 个百分点

Qwen3.6-35B 的 39.39% 提高到 60.61%。

固定模型：搜索增强后达到 71.21%

Evo-Max 加入跨任务经验先验与异步并行搜索。

更会修改的模型与更会利用经验的搜索相互配合，获奖率达到 71.21%

Junlin Yang, Che Jiang, et al. Frontis-MA1: Training an AI4AI Model towards Recursive Self-Improvement in Machine Learning Engineering, 2026.

Frontis-MA1：学会改进方案，也能帮助科学研究

竞赛中学到的改进能力能否迁移？NatureBench Lite 用 10 个留出科学任务检验。
每任务 4 小时，分别替换模型和搜索框架，两类改进都提高了复现论文结果的能力。

Rank	Model	Agent harness	All S ↑	All M ↑
OpenMLE controlled comparisons				
–	Frontis-MA1-35B	OpenMLE-Evo NB adapter	**30.0% (3/10)**	**70.0% (7/10)**
–	Qwen3.6-35B-A3B	OpenMLE-Evo NB adapter	20.0% (2/10)	50.0% (5/10)
–	Qwen3.6-35B-A3B	Original AIRA-Evo	10.0% (1/10)	20.0% (2/10)

同一基座，换搜索框架

达到论文水平的任务由 2/10 增至 5/10。
执行经验的组织方式影响可达到的结果。

同一框架，换训练后的模型

达到论文水平的任务由 5/10 增至 7/10。
其中 3/10 超过论文结果 10% 以上。

模型训练与经验搜索都显示迁移潜力，当前证据来自 10 个留出科学任务

Junlin Yang, Che Jiang, et al. Frontis-MA1: Training an AI4AI Model towards Recursive Self-Improvement in Machine Learning Engineering, 2026.

学习条件：任务与环境也要支持持续改进

改进机制能否持续有效，也取决于任务是否有学习价值、环境能否提供有效反馈。
任务生成与环境改造提供练习机会，统一研究设施支持系统性的比较与优化。

任务生成

BigBang 改进前沿任务的合成与评价，
并用真实训练收益校准方向。

环境改造

EnvHarness 根据当前弱点改造练习环境，
让技能提炼与参数学习获得更合适的反馈。

系统协同

RSIGym 提供统一的研究执行设施，
AI4AI at Scale 在约定边界内联合改进系统。

持续成长既需要新的学习挑战，也需要把数据、训练与系统修改连接起来

BigBang：模型继续成长，训练任务也要向前走

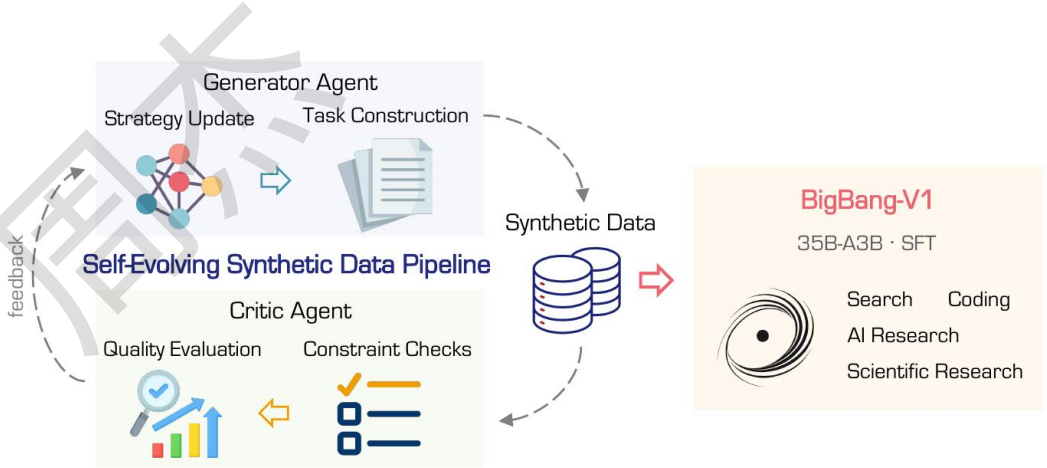
改进模型与搜索之外，还需要持续提供新挑战；固定题库和生成策略终会接近上限。
BigBang 面向可验证的前沿任务，让生成数据与评价数据的方法随模型一起改进。

前沿：持续暴露能力缺口

问题位于知识与能力边界，
也可以没有已知最优解。

可验证：让探索有据可循

通过计算、形式化规则或仿真，
客观检查候选解的质量。

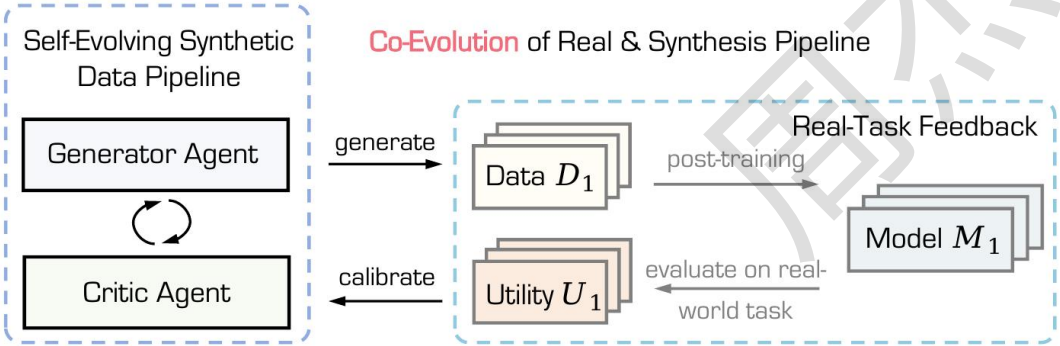


可持续的能力成长，需要不断出现新挑战，也需要可信的验证反馈

The BigBang Team. BigBang: Pursuing Open-Ended Intelligence through Self-Evolving Synthesis of Verifiable Frontier Tasks, 2026.

BigBang：任务看起来难，还要证明值得学

每次改合成策略都重新训练模型太贵，因此先用判别器快速筛查数据质量。
再周期性训练模型，用留出真实任务的收益校准判别器，修正生成与评价策略。



生成器改程序，判别器查质量

直接修改、运行和调试合成代码。
先排除工具、格式与轨迹问题，
再检查证据、矛盾和可验证性。
每轮记录经验，指导后续修改。

元判别器核对“评价与收益”

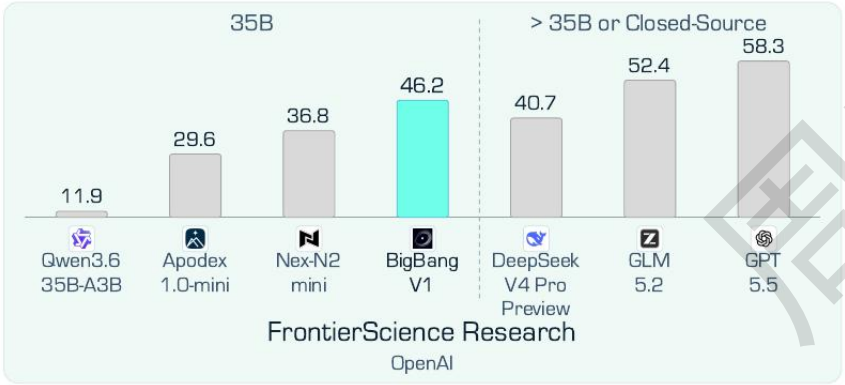
抽取方案训练模型，在留出任务上评估，
用实际收益修正判别与生成策略。

快速判别提供修改方向，真实训练收益校准数据演化的目标

The BigBang Team. BigBang: Pursuing Open-Ended Intelligence through Self-Evolving Synthesis of Verifiable Frontier Tasks, 2026.

BigBang：科研训练的收益也延伸到软件工程

BigBang-V1 在科研与通用任务上均超过同一基座，收益不限于科学知识问答。
科研推理与仓库级编程提供两个观察角度；这些结果衡量的是完整后训练流程。





FrontierScience：11.9 提高到 46.2

超过图中的 DeepSeek V4 Pro Preview。

SWE-Bench Pro：43.6 提高到 54.2

科研数据的训练收益迁移到真实代码问题。

科研与软件工程都获得提升，完整系统展现了跨任务的能力收益

The BigBang Team. BigBang: Pursuing Open-Ended Intelligence through Self-Evolving Synthesis of Verifiable Frontier Tasks, 2026.

BigBang：训练之后，搜索与核验仍能继续增益

模型变强后，证据遗漏与过早接受候选仍会限制表现。
推理时保留原路径，再增加广泛检索和反证检查，最后合并三路证据形成答案。

Model	*Long-horizon Search*		*Scientific Research*			
	BrowseComp	xbench	FS-R	HLE	BioMystery-HS	BioMystery-HD
BigBang-V1	76.5	58.4	46.2	50.3	57.5	15.7
BigBang-V1 + Search-and-Verify	**76.8**	**64.6**	**46.9**	**52.5**	**63.0**	**17.6**

xbench：58.4 提高到 64.6

更广的检索与反证检查，
对长程信息搜集带来更明显收益。

BioMystery-HS：57.5 提高到 63.0

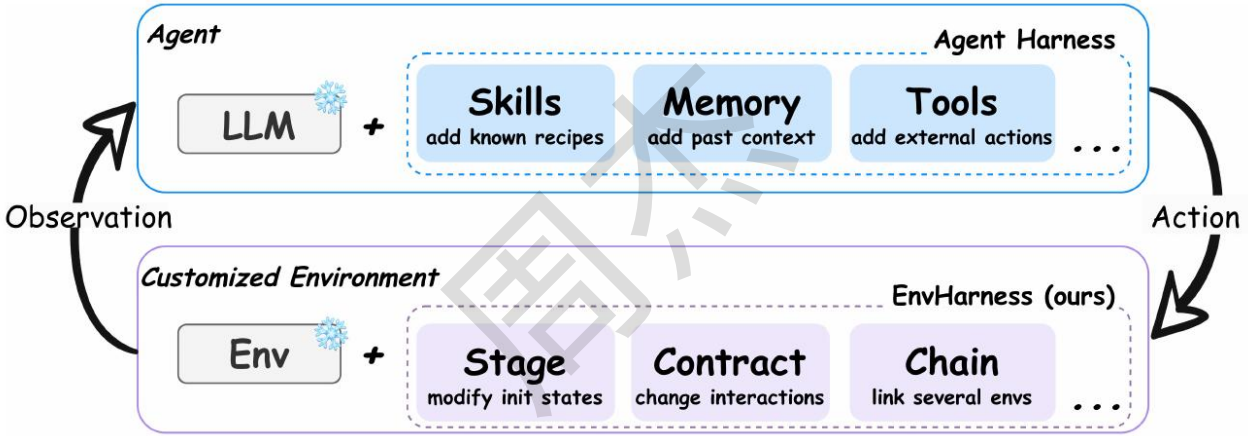
需要辨别竞争假设的生物分析，
也能从互补证据中受益。

更广的证据覆盖与更严格的候选核验，能够继续释放训练后的能力

The BigBang Team. BigBang: Pursuing Open-Ended Intelligence through Self-Evolving Synthesis of Verifiable Frontier Tasks, 2026.

EnvHarness：任务要更新，交互环境也要适配

可验证任务提供新挑战，但固定环境仍可能反复忽略智能体的具体弱点。
EnvHarness 在原环境外添加可编程组件，调整起点、交互与任务链，并沿用验证器。



Stage：改变起点

把杯子藏进抽屉，
让任务先训练寻找物品。

Contract：改变交互

限制动作或调整观察反馈，
让智能体学会必要的前置步骤。

Chain：延长任务

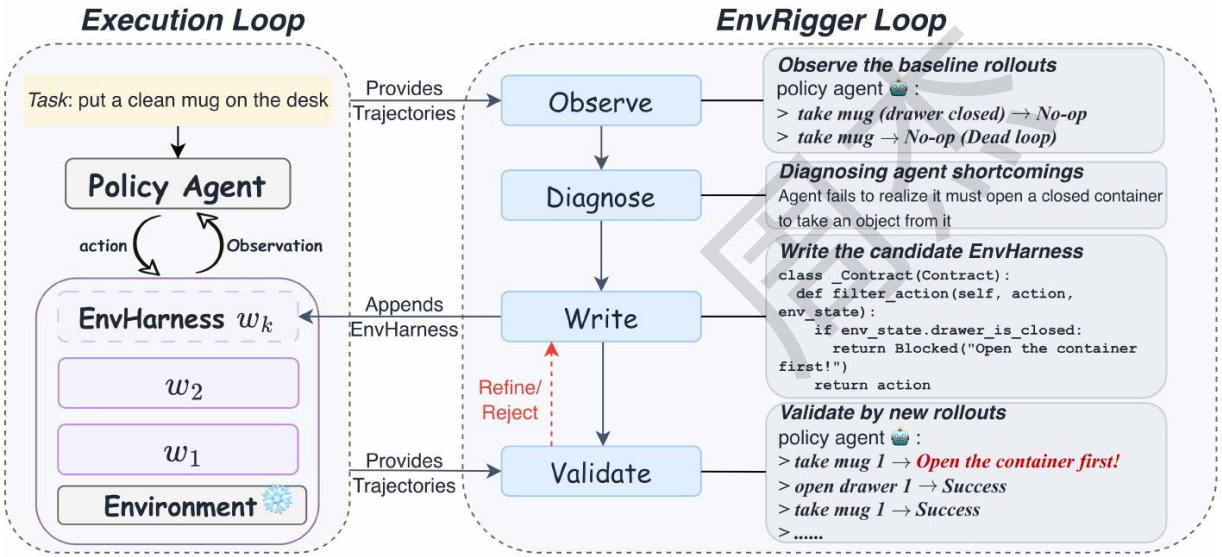
把多个任务接成一段经历，
训练长程目标保持。

保留可靠的环境与验证器，也能为当前智能体设计更有针对性的练习

Chengsong Huang, Zifeng Wang, et al. EnvHarness: Awakening Static Worlds for Agent Learning, 2026.

EnvHarness：先诊断弱点，再决定环境怎么改

盲目加难会让任务不可解，过度简化又无法形成有效练习。
EnvRigger 从成败轨迹诊断共性弱点，编写环境组件，再用新执行检验挑战性与可解性。



先判断需要哪种练习

困难过大时提供辅助，
全部成功时增加挑战，
针对重复动作、状态误读等
共性问题设计环境修改。

重新执行后才接纳组件

查看新轨迹的成功率与失败分布，
不可解或缺乏挑战时继续修订。

好的环境修改要针对能力缺口，同时让智能体有机会完成任务

Chengsong Huang, Zifeng Wang, et al. EnvHarness: Awakening Static Worlds for Agent Learning, 2026.

EnvHarness：练习更有针对性，技能也更有效

从定制环境的执行轨迹提取技能，再回到原任务的留出实例，检验收益能否迁移。
模型、环境数量与技能提取流程保持一致，成功率提高的同时，重复执行也减少了。

Skill Source	SWE-verified		OfficeQA		SpreadsheetBench	
	SR (↑)	Average Step (↓)	EM (↑)	F1 (↑)	Pass@1 (↑)	Mean Score (↑)
No Skills	47.67 0.93	53.58 2.93	54.23 2.84	55.77 2.98	46.44 0.15	61.32 0.37
Original Envs	49.88 2.59	55.01 1.69	54.40 1.84	55.77 1.59	45.88 1.19	61.47 0.59
SWE-smith	50.12 1.74	54.72 2.03	–	–	–	–
EnvHarness Envs	**52.58** 2.72	**49.61** 2.49	**56.20** 2.34	**57.73** 2.29	**49.15** 0.36	**62.48** 0.27
Improvement	+2.70	+5.40	+1.80	+1.96	+3.27	+1.01

SWE-bench：49.88% 提高到 52.58%

平均步骤由 55.01 降至 49.61，减少约 9.8%。

表格任务：45.88% 提高到 49.15%

原环境技能低于无技能基线，定制后转为提升。

针对弱点设计练习，能让提取的技能在原始任务中更有效、更省步骤

Chengsong Huang, Zifeng Wang, et al. EnvHarness: Awakening Static Worlds for Agent Learning, 2026.

EnvHarness：环境适配可支持技能与参数学习

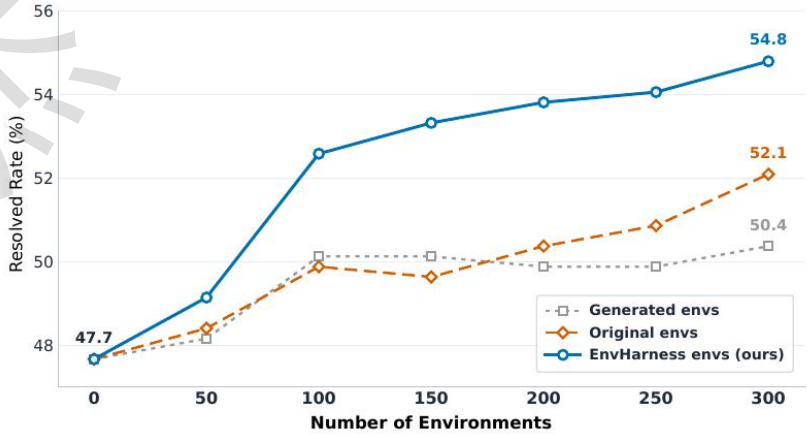
环境改造能否超出一次数据扩充？论文分别检验强化学习与持续技能积累。
定制环境带来同分布训练收益，也能随已学技能继续调整；分布外收益并非一致。

强化学习：ALFWorld 同分布收益更大

	ALFWorld			WebShop	
Training Set	In-Dist	OOD	Avg.	Score	SR
Original Envs	81.4	**89.6**	85.5	75.6	66.0
EnvHarness Envs	**87.9**	88.8	**88.4**	**79.2**	**67.4**

Qwen3-8B 经 GRPO 训练后，
同分布成功率由 81.4 提高到 87.9。
OOD 则由 89.6 小幅下降到 88.8。

技能演化：同样 300 个环境



SWE-bench 达到 54.8%，
高于原环境的 52.1% 与生成环境的 50.4%。

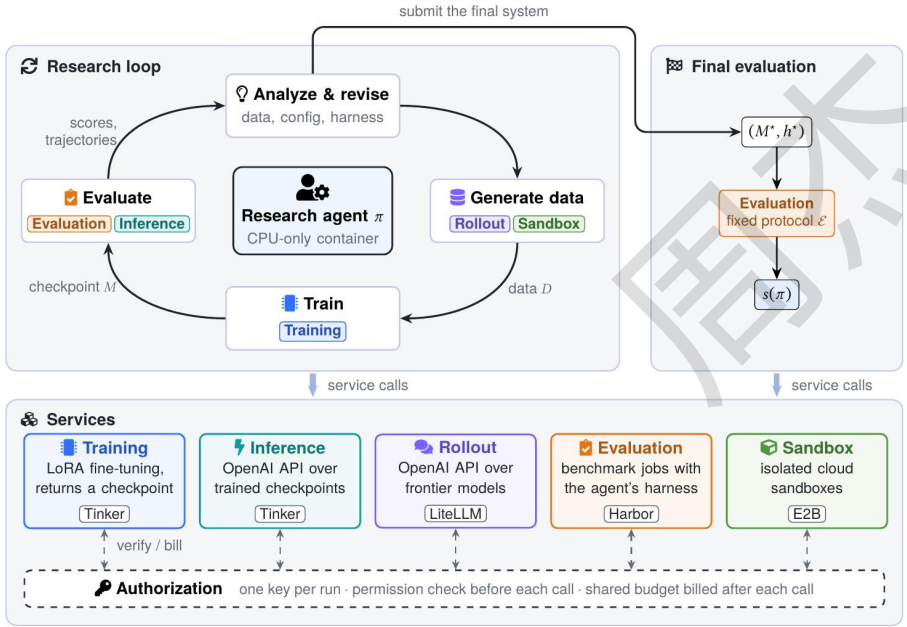
定制环境能支持两类学习，但同分布提升并不保证分布外任务也受益

周杰

Chengsong Huang, Zifeng Wang, et al. EnvHarness: Awakening Static Worlds for Agent Learning, 2026.

RSIGym：把研究设施备好，开放系统改进空间

任务、环境、模型与框架都可以改进，研究智能体却常被反复搭建实验设施拖住。
RSIGym 将训练、推理和评测封装为服务，让智能体在统一预算下探索不同干预。



反复搭建环境，消耗研究预算

训练、推理、生成数据、评测与沙箱都通过服务调用完成，智能体在轻量工作区组织实验。

每次实验返回下一步的依据

除了分数，还提供轨迹和日志，支持诊断失败、修改候选方案，再提交模型与执行框架。

复用研究设施、开放改进对象，才能系统比较不同的自主研究路径

Fanqing Meng, Lingxiao Du, et al. RSIGym: A Flexible Environment for Recursive Self-Improvement, 2026.

RSIGym：既能单独改，也能联合改

数据、训练设置和执行框架彼此影响，只开放一个环节难以检验它们如何配合。
RSIGym 用权限划分三类研究任务，用共享服务预算约束实验开销。

Data：改训练数据

训练配置与执行框架固定。
研究生成什么样的样本，
更有利于模型完成任务。

Harness：改执行框架

模型权重固定。
研究工具使用、控制流程
和异常恢复如何组织。

Joint：一起改

数据、训练配置与框架
同时开放，允许智能体
比较不同干预的组合。

每次调用统一校验权限并记录费用，智能体自行决定如何分配实验预算。

RSI-Index 衡量五类任务平均弥合了多少剩余性能差距，便于比较跨领域改进。

明确哪些可改、哪些固定，再对齐预算，才能比较不同改进方式的价值

Fanqing Meng, Lingxiao Du, et al. RSIGym: A Flexible Environment for Recursive Self-Improvement, 2026.

RSIGym：自主研究有效，优势却因任务而异

六个研究智能体从相同模型与简易框架起步，每项基准获得 500 美元服务预算。
Opus 5 的综合指标最高，但没有智能体在所有任务领先，终端与技能仍难改进。

	SWE initial 0.1767	TB2 initial 0.1049	AIME initial 0.3167	GPQA initial 0.5133	Skills initial 0.0165	RSI-Index mean share	Spend $	Time hours
Opus 5	0.5033 +40%	**0.2697** +18%	**0.9778** +97%	0.8333 +66%	0.2121 +20%	**0.4809**	1,910.39	44.92
Opus 5.5	0.5267 +43%	0.2172 +13%	0.9556 +94%	**0.8533** +70%	0.0948 +8%	0.4528	2,083.50	31.99
DeepSeek	**0.5333** +43%	0.2097 +12%	0.8778 +82%	0.8333 +66%	0.1568 +14%	0.4343	1,095.60	39.19
Astra	0.3567 +22%	0.2210 +13%	0.8833 +83%	0.8433 +68%	0.2044 +19%	0.4093	1,342.53	19.33
Sol 6.1	0.1867 +1%	0.2210 +13%	0.8889 +84%	0.8100 +61%	0.0870 +7%	0.3321	861.75	17.55
Sol 6	0.1967 +2%	0.1760 +8%	0.8278 +75%	0.5033 −2%	**0.2214** +21%	0.2079	161.73	16.03

29/30 个组合超过初始分数。终端与技能任务中，没有模型弥合超过 25% 的性能差距。

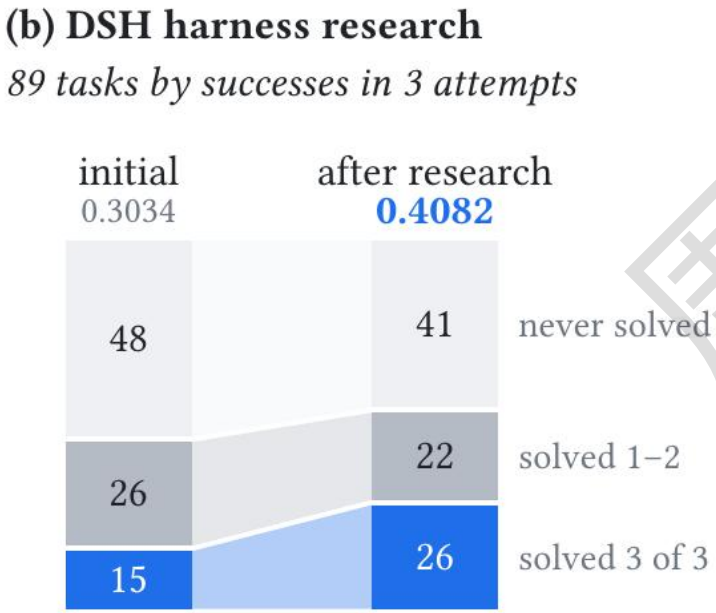
开发与最终评估使用同一任务集，这些结果反映预算内优化，尚不是独立测试集泛化。

自主研究能改善目标系统，当前证据限于同一任务集内的预算优化

Fanqing Meng, Lingxiao Du, et al. RSIGym: A Flexible Environment for Recursive Self-Improvement, 2026.

RSIGym：修好执行故障，也能释放已有能力

联合优化之外，固定模型单独改框架也有价值：Opus 5 从 DSH 日志定位执行故障。
Qwen3.6-35B 权重不变，同一组 89 个终端任务上的平均得分由 30.34% 到 40.82%。



改动针对真实执行故障

输出被截断后继续执行，
保留任务依赖的后台服务，
结束前用命令检查是否真正完成。

稳定完成的任务更多了

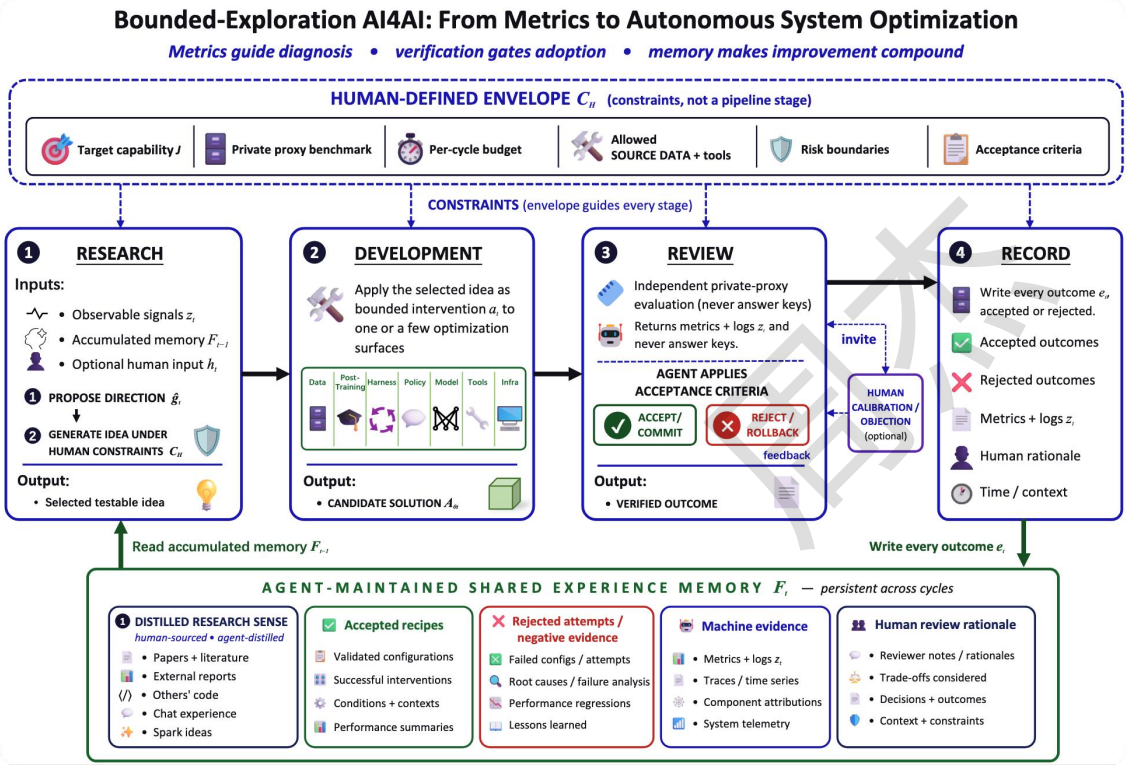
三次都成功的任务从 15 个增至 26 个。
大部分收益来自第一次重写。

减少执行故障能提高任务完成率，也说明系统进步不只来自更强的模型

周杰

Fanqing Meng, Lingxiao Du, et al. RSIGym: A Flexible Environment for Recursive Self-Improvement, 2026.

AI4AI at Scale：系统协同优化需要明确边界



全流程优化涉及数据、
训练与运行机制，
不同改动会相互影响。

人先约定改进边界

明确目标、可用资源
与候选方案的验收标准。

每轮结果都要留下

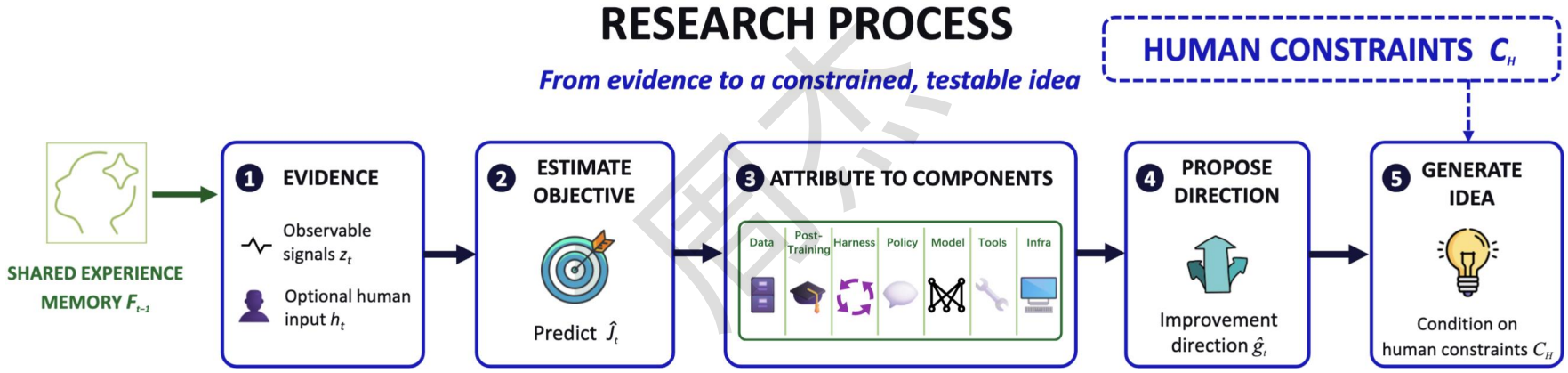
成功与被拒绝的尝试，
都进入后续可查的经验。

先约定目标与验收条件，再让每次试验留下可追溯的依据

XYZ Agentic Team. AI4AI at Scale: A Full-Pipeline System for Enhancing LLM Agentic Capabilities, 2026.

AI4AI at Scale：证据决定方向，独立评测决定采用

为避免边改边迎合答案，优化器只能读取获准的诊断证据。
从证据定位组件问题、提出可检验方案，再由独立评测支持是否采用的决定。



观察证据　估计瓶颈　定位组件　提出方向　形成方案

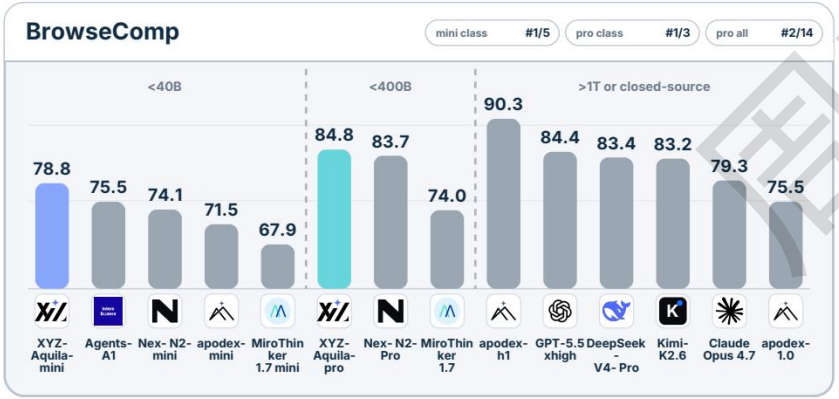
优化器负责提出改法，评测提供独立证据，验收标准决定哪些改动保留

XYZ Agentic Team. AI4AI at Scale: A Full-Pipeline System for Enhancing LLM Agentic Capabilities, 2026.

AI4AI at Scale：系统协同带来更强的搜索能力

报告中，mini 在小于 40B 参数组取得更高的 BrowseComp 成绩。
pro 在 LiveBrowseComp 达到 53.7，领先图中列出的系统。

小模型：BrowseComp 78.8



动态搜索：LiveBrowseComp 53.7



部分基线的工具、评审与测试日期不同，分数比较仍受评测条件影响。

整套系统表现出竞争力，单项改动的贡献与总研发成本仍需进一步检验

XYZ Agentic Team. AI4AI at Scale: A Full-Pipeline System for Enhancing LLM Agentic Capabilities, 2026.

长期评测：怎样确认系统确实在成长

新知识是否留住、经验是否有用、改进机制是否更强，最终都需要长期证据。
评测要区分额外预算的收益、对旧题的适应，以及对新任务的迁移。

预算与迁移

框架演化评测与简单重试等方法比较，
再用独立留出任务检查收益能否保留。

连续使用

AgentStream 让任务持续到来，
检查经验积累在同领域与跨领域中的作用。

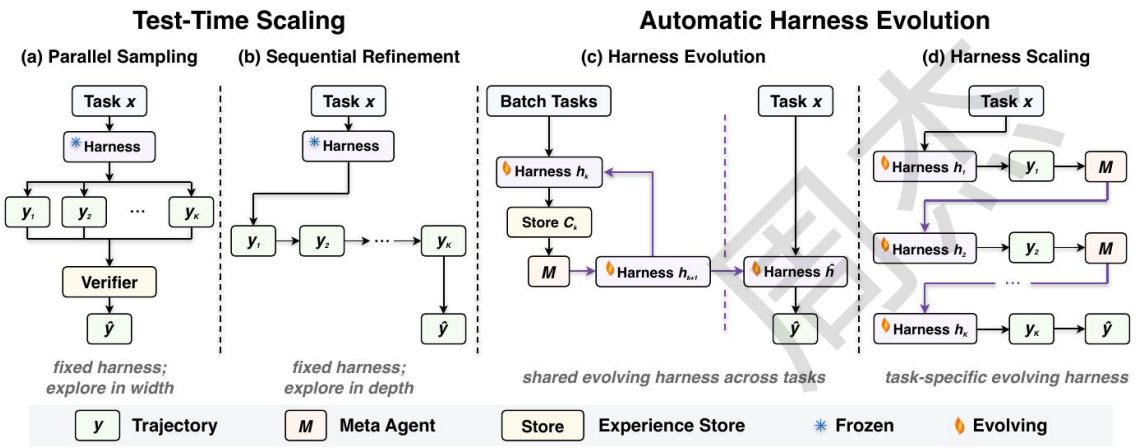
学习效率

Agent Plasticity 衡量获得进步的成本，
并追问经验是否真正影响了后续行动。

最终得分、迁移收益与学习效率，共同回答系统是否在持续成长

框架演化评测：真的变强，还是多试了几次？

系统越复杂，越要分清收益来自哪里：改框架是否优于把同样预算用于多次尝试？
这项研究同时对齐反馈与预算，并分开搜索任务和最终测试，检验可迁移的改进。



同样预算，可以花在哪里？

多次独立尝试、修改解题轨迹、
优化共享框架，或按题改框架。

用两种对照回答两个问题

相同反馈下，能否胜过简单搜索？
换一批未见任务，收益能否保留？

证明演化有效，既要胜过同预算强基线，也要在未见任务上保留收益

Yike Wang, Huaisheng Zhu, et al. Rethinking the Evaluation of Harness Evolution for Agents, 2026.

框架演化评测：终端任务中，重试更划算

Terminal-Bench 2.1 让各方法从同一框架出发，使用 5 轮执行预算与相同反馈。
并行采样的最终选择结果更好，连续修订覆盖更多任务，框架演化未超过简单基线。

Method	Claude Opus 4.6		GPT-5.4		GPT-5.4 mini		Average	
	pass@1	pass@5	pass@1	pass@5	pass@1	pass@5	pass@1	pass@5
direct sampling using initial harness	69.9	-	75.9	-	56.9	-	67.6	-
Test-Time Scaling								
Parallel Sampling	**84.8**	84.8	**87.1**	87.1	**74.1**	74.1	**82.0**	82.0
Sequential Refinement	83.1	**90.4**	85.4	**93.3**	62.1	77.6	76.9	**87.1**
Automatic Harness Evolution								
Harness Evolution	73.0	83.2	78.6	89.3	57.4	64.6	69.7	79.0
Harness Scaling	83.1	89.9	82.0	88.8	61.5	**78.2**	75.5	85.6

有测试反馈：并行采样最终得分 82.0，框架演化为 69.7；
无测试反馈：并行采样仍为 72.3，框架演化为 67.4。

额外预算带来的进步，应与同等反馈和尝试机会下的简单方法比较

Yike Wang, Huaisheng Zhu, et al. Rethinking the Evaluation of Harness Evolution for Agents, 2026.

框架演化评测：换一批题，平均收益几乎消失

45 个任务用于搜索、10 个用于选择版本，另有 34 个任务完全留给最终测试。
三个模型的测试均分仍为 63.2，说明搜索集上的进步没有转化为明确的整体泛化收益。

Method	Claude Opus 4.6	GPT-5.4	GPT-5.4 mini	Average
direct sampling using initial harness	63.3	72.1	54.4	63.2
Harness Evolution	64.5 (+1.2)	72.1 (+0.0)	53.0 (-1.4)	63.2 (+0.0)

未见任务上的变化

Opus 4.6：+1.2 个百分点
GPT-5.4：没有变化
GPT-5.4 mini：−1.4 个百分点

修改看起来合理，也未必普遍有效

轨迹分析发现，部分修改记住了
特定题目的路径、命令和已知错误。
这些经验难以自动变成通用能力。

修改看起来合理仍不够，真正值得继承的是能迁移到未见任务的改进

Yike Wang, Huaisheng Zhu, et al. Rethinking the Evaluation of Harness Evolution for Agents, 2026.

框架演化评测：长程游戏展现了适应价值

长程游戏既有改进空间，又持续提供细粒度反馈，适合在交互中调整工具、技能与记忆。
同预算下，按任务演化取得明确收益；这支持在线适应，尚不等于跨任务泛化。

	Claude Opus 4.8		GPT-5.4	
Task	**Sequential Refinement**	**Harness Scaling**	**Sequential Refinement**	**Harness Scaling**
dcss_dungeon_ai	17.1	**22.4**	9.9	**13.3**
nethack_dungeon_agent	44.3	**61.3**	**50.4**	49.0
openrct2_theme_park_ai	38.4	**47.8**	36.2	**50.0**
openttd_transport_ai	53.5	**56.2**	**51.6**	50.5
wesnoth_tactical_ai	**88.0**	86.0	81.5	**86.0**
Average	48.3	**54.7**	45.9	**49.8**

ARC-AGI-3：2,000 步预算

完整框架演化相对固定框架：
Opus 4.8：43.3 → 77.5
GPT-5.6 Terra：21.1 → 38.2
指标同时衡量通关与动作效率。

EdgeBench：同样探索 12 小时

Opus 4.8：48.3 → 54.7；GPT-5.4：45.9 → 49.8。
收益并非每个游戏都成立，表中保留了这些差异。

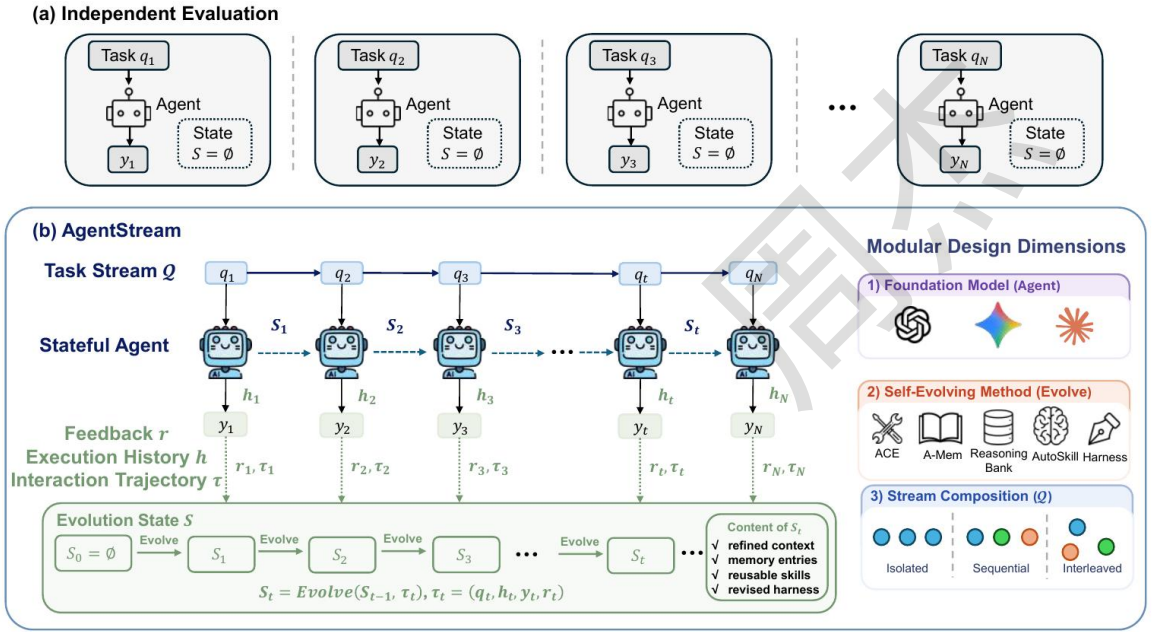
这是单任务在线适应，
不等于跨任务泛化已经解决。

框架演化更适合需要持续适应、具有改进空间且反馈充分的长程任务

Yike Wang, Huaisheng Zhu, et al. Rethinking the Evaluation of Harness Evolution for Agents, 2026.

AgentStream：做对新题，还要经得起连续使用

留出评测检验改进能否迁移，真实部署还要面对连续到来、领域不断变化的任务。
AgentStream 保留跨任务状态，让记忆、技能与框架持续更新，观察收益与干扰。



每做完一题，更新一次经验

用轨迹和自我评价改写经验，
更新记忆、技能或运行框架。
演化时不接触测试答案。

同一框架交叉比较

3 个基座模型，5 类演化方法，
6 个基准、每个 50 项任务。
与同模型不积累经验的基线比较。

自进化需要放进连续任务流检验，过去的经验既可能帮助，也可能干扰

Dong Yan, Jian Liang, et al. AgentStream: How Well Do Self-Evolving LLM Agents Perform Under Streaming Tasks?, 2026.

AgentStream：混合任务流，未必更难学

经验按领域分别积累、跨领域依次传递，或在混合任务间共享，效果可能完全不同。
三种场景改变任务顺序与状态共享范围，区分领域内积累和跨领域交互。

Isolated：各领域独立积累

同领域任务共享经验，
不同基准之间不传递状态。

Sequential：按领域依次处理

处理完一个领域，再切到下一个，
此前积累的状态一直保留。

Interleaved：跨领域交错到来

把所有领域任务混合打乱，
共同更新一份演化状态。

Scenario	Positive (> Vanilla)	Negative (< Vanilla)	Gain	Top-1 rate
Isolated	**34/45**	**11/45**	**+1.37**$\pm$0.80	**38%**
Sequential	28/45	16/45	+0.75$\pm$0.48	29%
Interleaved	28/45	17/45	+0.90$\pm$0.34	33%

独立场景 34/45 次配置获得正收益，跨域场景为 28/45；交错的平均收益高于顺序。

同领域积累最稳定，但交错任务的平均收益高于按领域依次处理

Dong Yan, Jian Liang, et al. AgentStream: How Well Do Self-Evolving LLM Agents Perform Under Streaming Tasks?, 2026.

AgentStream：会做任务，不一定善于积累经验

在本文的模型与任务设置中，较弱基座的平均演化收益为负，积累经验也可能退步。
能提炼可靠经验之后，收益仍不随模型强度单调增加：剩余改进空间同样重要。

	Isolated		Sequential		Interleaved	
Model	Positive	Gain	Positive	Gain	Positive	Gain
GPT-5.4	2/5	−0.35	1/5	−0.78	1/5	−0.62
Gemini 3.1 Pro	5/5	+2.71	4/5	+1.98	5/5	+2.41
Claude Opus 4.7	5/5	+1.75	4/5	+1.05	4/5	+0.90

GPT-5.4：三种场景平均均为负收益

15 种方法与场景组合中，仅 4 种超过基线。
经验积累本身不能保证性能改善。

Gemini 的提升大于基线更强的 Claude

平均增益分别为 2.37 与 1.23 个百分点。
这一趋势在三种场景下都出现。

经验提炼能力与剩余提升空间共同影响收益，不能只按初始分数选择基座

Dong Yan, Jian Liang, et al. AgentStream: How Well Do Self-Evolving LLM Agents Perform Under Streaming Tasks?, 2026.

AgentStream：跨领域使用，更要选择相关经验

把经验直接写入上下文，有利于领域内积累，也容易把无关策略带进新任务。
外部记忆与技能库则按当前任务检索，在混合任务流中筛选值得复用的经验。

Method	Isolated	Sequential	Interleaved
ACE	+2.28	+2.26	−1.26
Harness	+1.91	−0.86	+1.01
A-Mem	+1.93	+1.71	+2.22
ReasoningBank	+0.78	+0.46	+1.79
AutoSkill	−0.07	+0.19	+0.72

周杰

ACE：从同域收益转为混合流负收益

平均增益由 +2.28 变为 −1.26 个百分点。
直接融入上下文的经验更容易跨域干扰。

检索式方法：交错场景平均收益最高

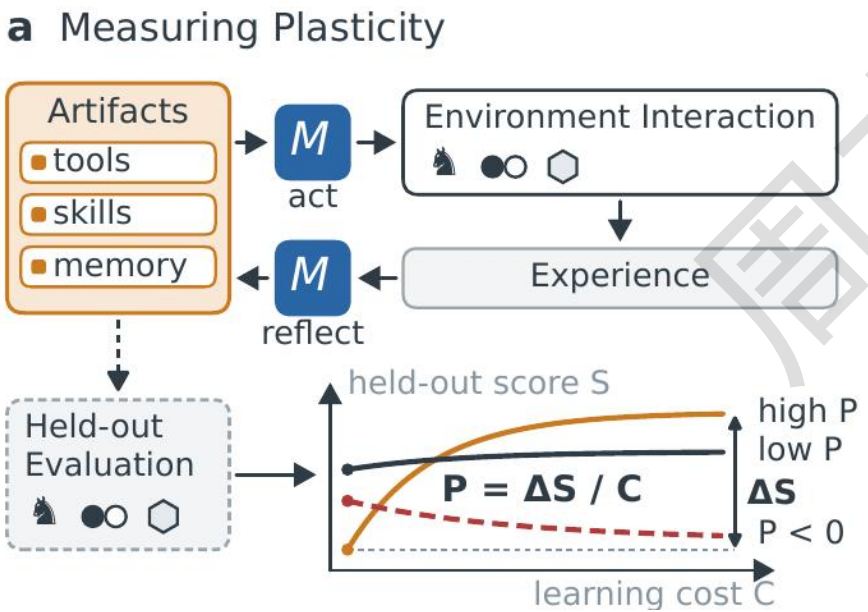
A-Mem、ReasoningBank、AutoSkill 均呈此趋势。
相关经验检索帮助筛去跨域噪声。

没有适合所有场景的方法，关键是让相关经验在合适的任务中发挥作用

Dong Yan, Jian Liang, et al. AgentStream: How Well Do Self-Evolving LLM Agents Perform Under Streaming Tasks?, 2026.

Agent Plasticity：长期进步还要看学得多高效

连续使用中能否进步只是第一问，获得这些收益花了多少成本，同样影响长期价值。
这项工作固定模型，让工具、技能与记忆持续更新，按未见交互的收益衡量学习效率。



每轮重新开始，只继承积累的经验

每局使用新上下文，模型权重固定。
根据训练交互反思，修订持久化产物，
通过结构与执行检查后交给下一轮。

用学习过程看不到的对局检验

保留同难度和更强对手的评测集，
记录每个检查点的表现与学习成本。
评测轨迹和分数不进入反思过程。

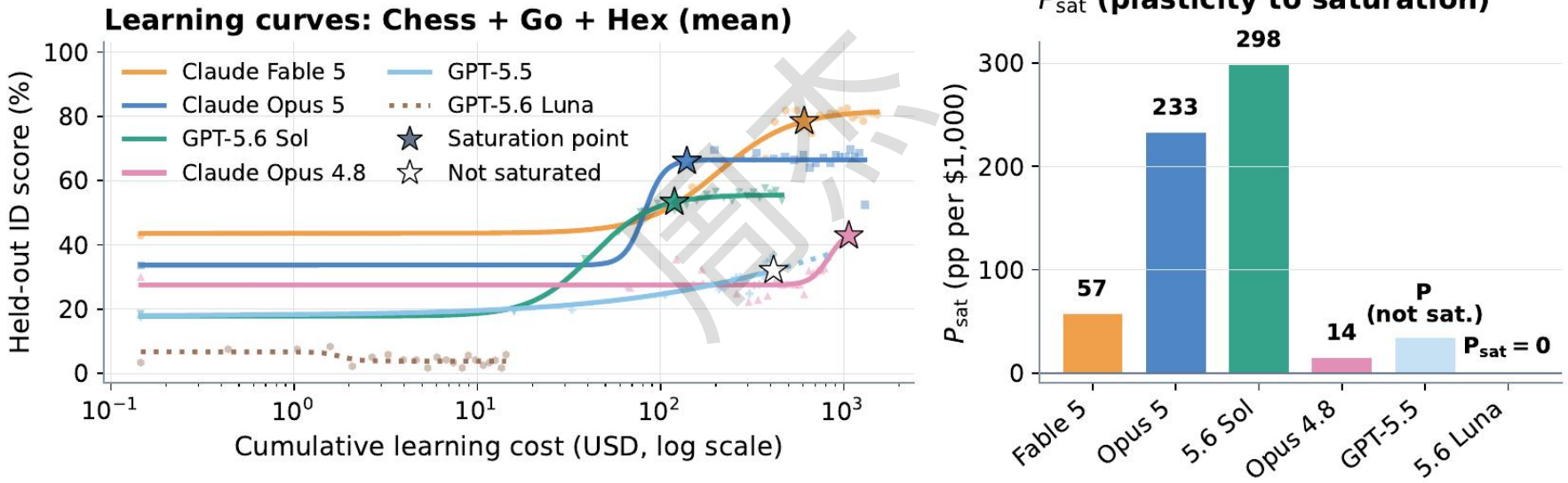
可塑性衡量经验转化为未来收益的效率，必须同时看泛化表现与学习成本

周木

Harman Singh, Anton Bakhtin, et al. Agent Plasticity: Measuring Self-Improvement Through Experience, 2026.

Agent Plasticity：终点最强，未必学得最省

最终能力会掩盖起点与学习开销的差异，因此用未见对局的得分增量除以学习成本。
主要指标统计达到估计增幅 90% 时的效率，减少饱和后继续花费对比较的影响。



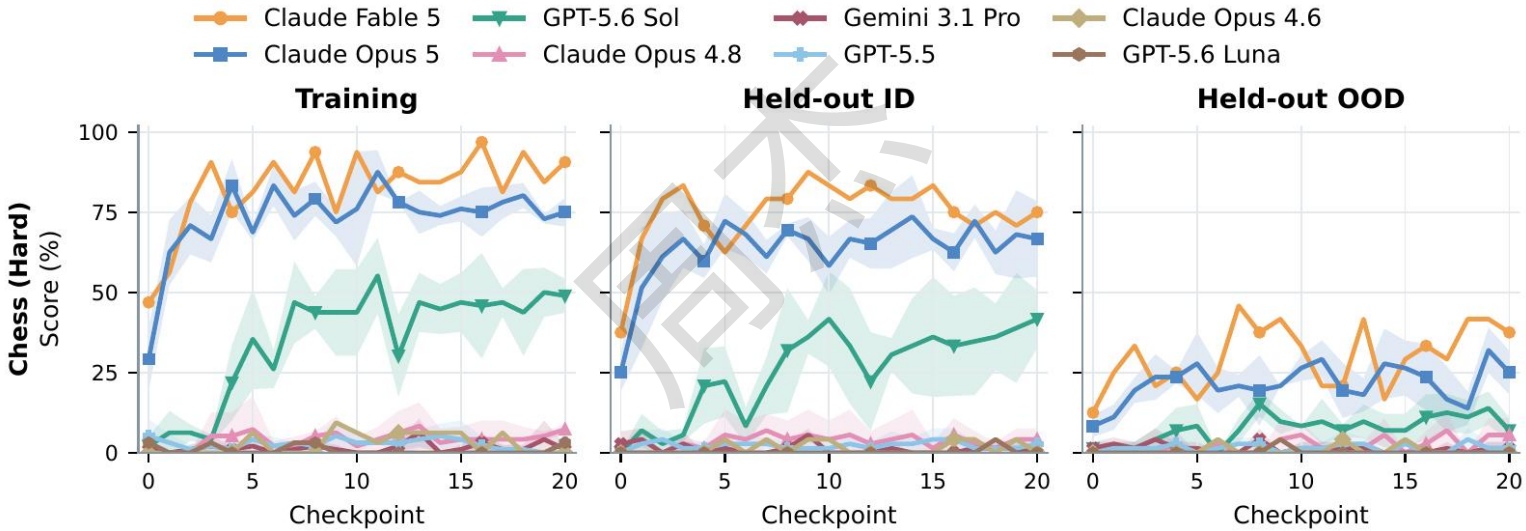
Fable 5 的拟合终点最高，GPT-5.6 Sol 的学习效率最高：298 对 57 分 / 千美元。

最终能力与学习效率是两个维度，预算不同，合适的模型也可能不同

Harman Singh, Anton Bakhtin, et al. Agent Plasticity: Measuring Self-Improvement Through Experience, 2026.

Agent Plasticity：同难度进步，不保证越级迁移

Hard Chess 中，部分模型在训练对局与未见同难度对局上持续改善。
面对更强对手时，收益通常更小或更不稳定，需要区分经验复用与难度迁移。



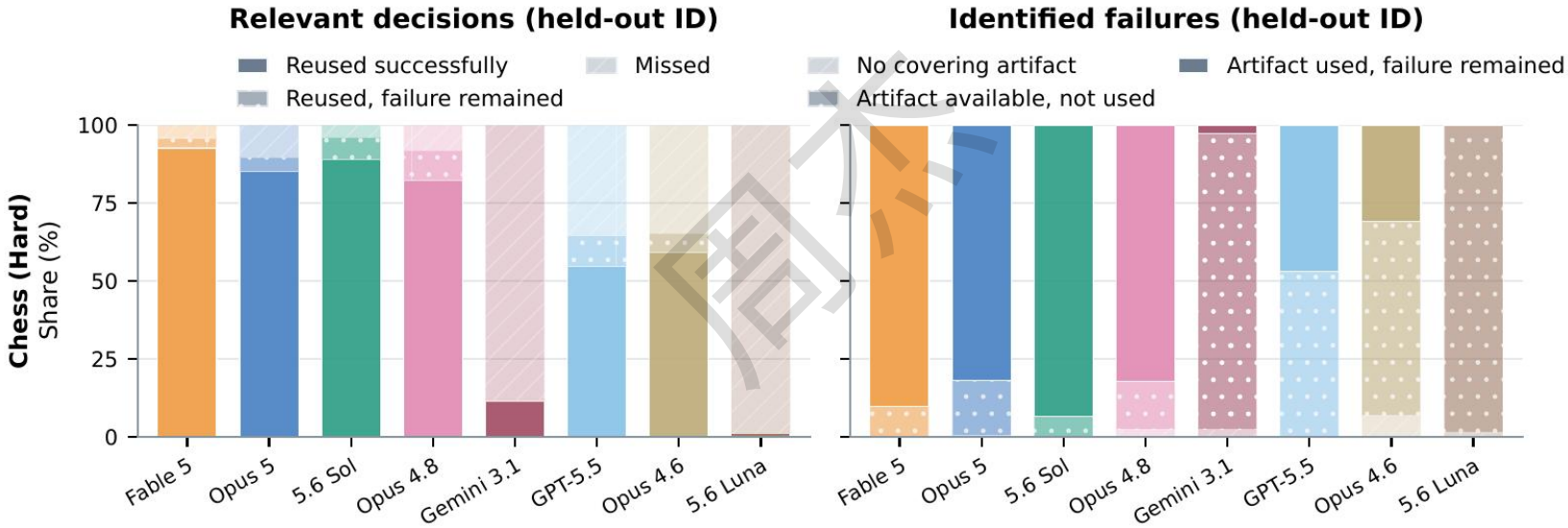
左：训练交互　中：未见同难度对局　右：未见、更强的对手

既要证明经验能用于未见对局，也要检验面对更强对手时能保留多少收益

Harman Singh, Anton Bakhtin, et al. Agent Plasticity: Measuring Self-Improvement Through Experience, 2026.

Agent Plasticity：留下经验，还要用上、用好

学习曲线说明是否进步，失败决策则帮助定位卡点：相关经验是否存在、是否被调用？
调用后仍失败，问题可能在经验质量、泛化或应用方式，而非缺少经验记录。



改进较弱的模型常跳过已有经验。高复用模型的剩余错误，则更多发生在已经使用经验时。

调用时机与应用质量，共同影响已有经验能否改善行动

Harman Singh, Anton Bakhtin, et al. Agent Plasticity: Measuring Self-Improvement Through Experience, 2026.

持续成长：既积累能力，也改进学习方法

新数据补充知识，行动经验改善做法，自动优化进一步拓展学习机制。
三类研究可以在同一系统中配合，共同支持长期成长。

数据驱动终身学习

新数据学得进，旧能力留得住。
参数更新、容量扩展与回放巩固相互配合。

经验驱动终身学习

行动留下可复用的经验。
记忆与技能辅助执行，有效做法还能进入参数。

循环自我进化

提出改法的机制也能继续更新。
执行验证筛选改动，任务与环境提供持续挑战。

长期进步要同时看新任务收益、旧能力保留与获得这些收益的成本

走向自主成长：仍有三个关键问题

能力积累与自动优化已有进展，长期自主成长仍有待更充分的证据。
下一步要把学习收益、问题价值与真实使用连接起来。

怎样提出好问题

从能力缺口与真实需求中寻找学习目标，
用实际收益持续校准任务与反馈的价值。

怎样走进现实

让数据、经验与系统更新服务持续变化的工作，
同时管理长期成本、任务干扰与能力退步。

怎样证明进步

同预算比较新任务收益与旧能力保留。
再检验更新后的系统，能否更有效地产生下次改进。

下一次学习，能否比这一次更有效?

谢谢！